

REAL-WORLD NET MAUI

SUCCINCTLY®

BY **ALESSANDRO
DEL SOLE**

Real-World .NET MAUI Succinctly[®]

Alessandro Del Sole

Foreword by Daniel Jebaraj



Copyright © 2025 by Syncfusion, Inc.

2501 Aerial Center Parkway

Suite 111

Morrisville, NC 27560

USA

All rights reserved.

ISBN: 978-1-64200-239-3

Important licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal or educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

BOLDDesk, BOLDSIGN, BOLD BI, BOLD REPORTS, SYNCFUSION, ESSENTIAL, ESSENTIAL STUDIO, SUCCINCTLY, and the 'Cody' mascot logo are the registered trademarks of Syncfusion, Inc.

Technical Reviewer: James McCaffrey

Copy Editor: Courtney Wright

Acquisitions Coordinator: Tres Watkins, VP of content, Syncfusion, Inc.

Proofreader: Jacqueline Bieringer, content producer, Syncfusion, Inc.

Table of Contents

About the Author	9
Introduction.....	10
Solving common problems	10
Sharing experience	10
Download the companion code	11
Chapter 1 Cross-Platform Device Capabilities.....	12
Platform integration in .NET MAUI.....	12
Chapter prerequisites	12
Managing network connections with snackbar views	14
Defining custom messages	15
Handling the status of the network connection.....	16
Displaying a snackbar view.....	17
Checking the battery level	19
Reacting to the energy saving status in the user interface	21
Working with files	23
Accessing files in the local app folders.....	24
Selecting files from the device	25
Development trick: Running code in the main thread.....	28
Opening the browser and sending emails.....	29
Opening the browser	29
Sending emails	31
Chapter summary.....	34
Chapter 2 Local Settings: Preferences and Secure Storage.....	35
Managing local preferences	35

Securing preferences and settings: secure storage	37
Configuring iOS to work with simulators	38
Implementing access to secure storage	38
Secure storage tips and tricks	39
Chapter summary	40
Chapter 3 Implementing Biometric Authentication	41
Setting up libraries for biometric authentication	41
Additional setup for iOS	42
Additional setup for Android	42
The logic of the sample project	43
Creating a sample user interface	44
Enforcing requirements for biometric authentication	46
Implementing biometric authentication	47
Authentication with password	47
Authentication with fingerprint and face ID	48
Running the sample app	51
Chapter summary	52
Chapter 4 Local Data Access with SQLite	53
Choosing and installing the SQLite libraries	53
Installing the SQLite NuGet packages	54
Setting up the database connection	55
Creating a data model	56
Reading, filtering, and writing data	58
Reading data	58
Writing data	59
Deleting data	60

Implementing Model-View-ViewModel logic.....	61
Creating the user interface with data binding.....	64
Running the example.....	66
Chapter summary.....	66
Chapter 5 Working with Web API and JSON.....	68
Assumptions for this chapter	68
Understanding the sample solution.....	68
Understanding the database	69
Understanding the Web API project.....	72
Understanding the .NET MAUI project.....	78
Calling Web APIs from .NET MAUI.....	81
Hints about editing existing data	84
Implementing broadcast messages.....	85
Deserializing JSON responses	85
Implementing the user interface.....	88
Chapter summary.....	91
Chapter 6 Securing Communication with Certificate Pinning	93
Describing the certificate pinning implementation	93
Certificate pinning in .NET MAUI	93
Considerations for debugging	96
Chapter summary.....	97
Chapter 7 App Center: Drive Business Decisions with Analytics	98
Analytics and diagnostic data	98
Setting up a sample project	99
Configuring the Microsoft App Center.....	100
Adding analytics and privacy consent to the app.....	102

Understanding analytics data	107
Hints about Application Insights	111
Chapter summary	111
Chapter 8 Displaying Lottie Animations with SkiaSharp	112
.NET MAUI and Lottie	112
Adding Lottie animations to the project	113
Displaying Lottie animations	114
Chapter summary	115
Chapter 9 Displaying and Sharing Documents	116
Chapter focus and sample project setup	116
Introducing the SfPdfViewer control	118
Sharing files and documents	120
Chapter summary	123
Conclusion	124

The *Succinctly*® Series of Books

Daniel Jebaraj
CEO of Syncfusion®, Inc.

When we published our first *Succinctly*® series book in 2012, *jQuery Succinctly*®, our goal was to produce a series of concise technical books targeted at software developers working primarily on the Microsoft platform. We firmly believed then, as we do now, that most topics of interest can be translated into books that are about 100 pages in length.

We have since published over 200 books that have been downloaded millions of times. Reaching more than 1.7 million readers around the world, we have more than 70 authors who now cover a wider range of topics, such as Blazor, machine learning, and big data.

Each author is carefully chosen from a pool of talented experts who share our vision. The book before you and the others in this series are the result of our authors' tireless work. Within these pages, you will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

We are absolutely thrilled with the enthusiastic reception of our books. We believe the *Succinctly*® series is the largest library of free technical books being actively published today. Truly exciting!

Our goal is to keep the information free and easily available so that anyone with a computing device and internet access can obtain concise information and benefit from it. The books will always be free. Any updates we publish will also be free.

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctlyseries@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Please follow us on social media and help us spread the word about the *Succinctly*® series!



About the Author

Alessandro Del Sole is a Xamarin Certified Mobile Developer and has been a Microsoft MVP since 2008. Awarded MVP of the Year in 2009, 2010, 2011, 2012, and 2014, he is internationally considered a Visual Studio expert and a .NET authority. Alessandro has authored many printed books and ebooks on programming with Visual Studio, including *Xamarin with Visual Studio*, [Visual Studio 2022 Succinctly](#), *Visual Basic 2015 Unleashed*, and [Xamarin.Forms Succinctly](#).

Alessandro has written tons of technical articles about .NET, Visual Studio, and other Microsoft technologies in Italian and English for many developer portals. He is a frequent speaker at Italian conferences, and he has released a number of Windows Store apps. He has also produced a number of instructional videos in both English and Italian. Alessandro works as a senior software engineer for Fresenius Medical Care, focusing on building mobile apps with .NET MAUI and Xamarin in the healthcare market.

You can follow him on [LinkedIn](#) and support his community contributions on [Patreon](#).

Introduction

In my previous ebook, [.NET MAUI Succinctly](#), I walked through the development platform with an instructional approach. I discussed common views, types, libraries, patterns, and more, with simple examples that explained how to put together different blocks of the technology from the point of view of the beginner, regardless of any C# programming expertise. When it comes to building real applications, developers working with .NET MAUI who are not very experienced with this technology will face a number of problems to solve, especially when the app has a more complex design, or if they used to work with other platforms, such as desktop or web frameworks.

Solving common problems

Developing for mobile devices implies taking care of things you will not need to care about on desktop or for a website. Checking for the battery status to ensure the user does not lose data, ensuring the user knows the app is working with a data plan, accessing files on the device, notifying the user of a problem with a nice UI—these are all examples of this different way of thinking.

In addition, developers sometimes expect that everything in cross-platform technology really works cross-platform at no cost, but unfortunately, that's not true. With .NET MAUI, the cross-platform capabilities work fine in 90 percent of cases, but a good 10 percent of situations require you to write handlers, write platform-specific code, or use third-party libraries to make a view have the exact same behavior on iOS, Android, Windows, and macOS platforms. This is totally normal because they are different operating systems, with different APIs, and with two different implementations. Addressing these differences requires effort.

Sharing experience

I have been working with Xamarin.Forms and .NET MAUI in real-world mobile apps for more than seven years, and I have been lucky enough to do this within a large agile team of professional designers, lawyers (yes, an app must comply with regulations), businesspeople, and other cool developers. During these years, I have gained a lot of experience in implementing solutions to solve both problems common to any app and custom scenarios, and I realized that sometimes the effort needed to obtain simple things is higher than expected.

I'm currently working on migrating a big Xamarin.Forms project to .NET MAUI, and I'm facing issues myself. With this in mind, I decided to write this new ebook and collect some of my experience to provide developers with insight and points of view on topics they will need to face sooner or later when working with .NET MAUI (and if they are migrating from Xamarin.Forms). This book contains only a part of that experience, due to the nature of the *Succinctly* series ebooks. However, the topics you will find in this book are of common interest and the examples will help you be faster and more productive.

Download the companion code

This ebook comes with a [companion code repository](#) that contains sample projects grouped by chapter. My suggestion is to read chapters while keeping their sample projects open in Visual Studio. I will remind you in those chapters where this approach is necessary. Figures in the ebook were captured from Visual Studio 2022 on a Windows 10 PC, but the source code will also work on Windows 11, macOS, and Visual Studio 2022 for Mac (except for targeting Windows).

There is certainly much more in .NET MAUI to explore in real-world applications, and this ebook cannot cover every possible scenario, but I am confident that it will help you solve many common problems quickly.

Chapter 1 Cross-Platform Device Capabilities

Applications in the real world need to make sure the user always has control over what's happening—especially in unexpected situations—and the user needs to be able to respond accordingly. For example, if the network connection is lost, not only must the app intercept the loss of connection, but it also needs to communicate to the user that a problem occurred and then provide one or more actions. This should all be done with a nice user interface, avoiding unnecessary modal dialogs that might be acceptable for desktop apps, but not on mobile apps where the user should never feel blocked.

This chapter will describe how to solve common problems that mobile apps might encounter, in a cross-platform approach. You will not find an explanation of how to work with sensors, which is something specific to some app categories; rather, you will find ways of handling scenarios that almost every app will face.

Platform integration in .NET MAUI

If you have experience with Xamarin.Forms, you know that accessing device capabilities in a cross-platform approach is possible via the Xamarin.Essentials library, an open-source project created and maintained by Microsoft that you can install from NuGet. This library exposes a large set of APIs that you can invoke from the shared project. It helps you avoid the need to write platform-specific code for capabilities common to all the supported systems.

In .NET MAUI, there is no external library. A similar set of APIs is already built into the core library, so you just need to add the appropriate using directives (as you will see in this chapter). This also avoids the need for an additional NuGet package in your solution.



Note: You might hear this set of APIs referred to as “*.NET MAUI Essentials*,” but *this does not really exist. It’s a common way for current and former Xamarin.Forms developers to refer quickly to the part of MAUI that allows for accessing device capabilities.*

Chapter prerequisites

To follow the examples provided in this chapter, create a new .NET MAUI solution using the default template, as shown in Figure 1.

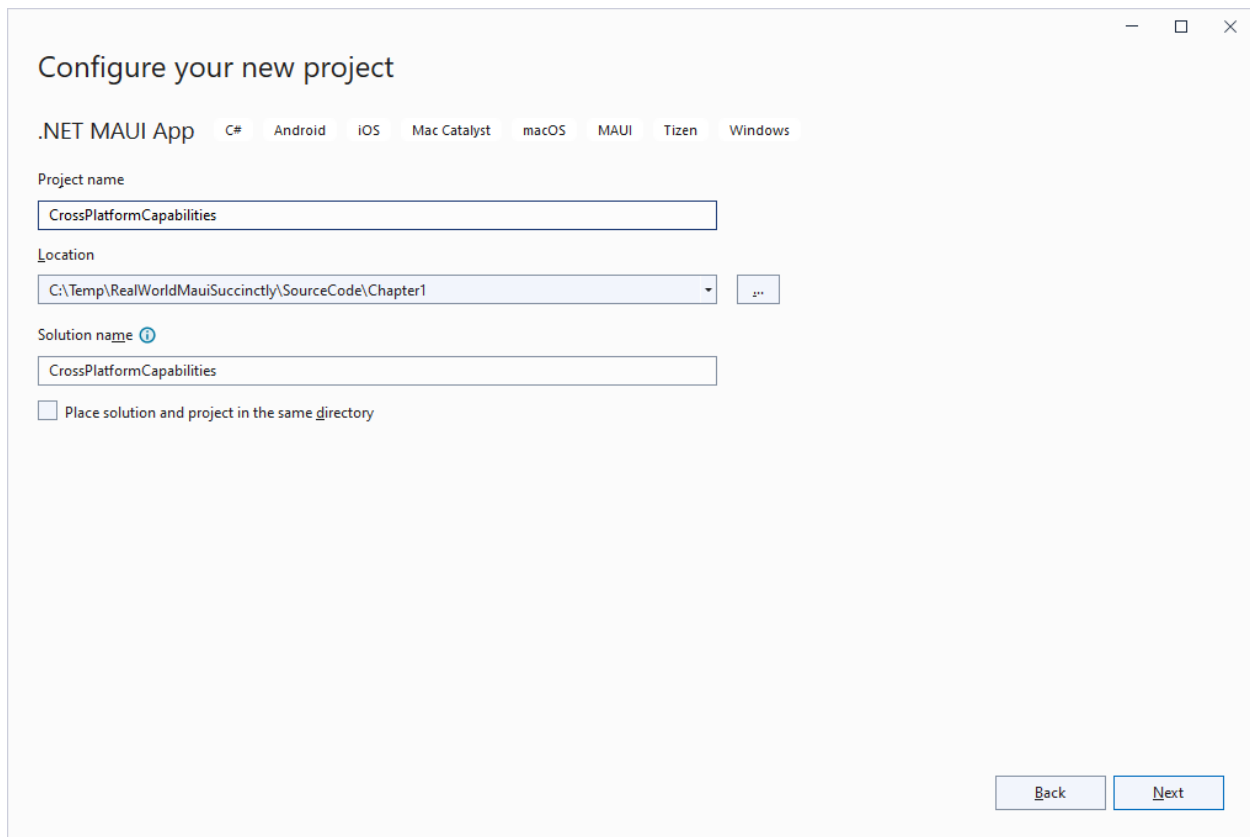


Figure 1: Creating a new .NET MAUI solution

Select **.NET 7** as the target technology. When the solution is ready, make sure you install the .NET MAUI Community Toolkit. This library is required to use snackbar views later. To install the toolkit, right-click the solution name in **Solution Explorer**, select **Manage NuGet Packages for Solution**, and then open the **Browse** tab. Search for the library (see Figure 2) and then click **Install**.

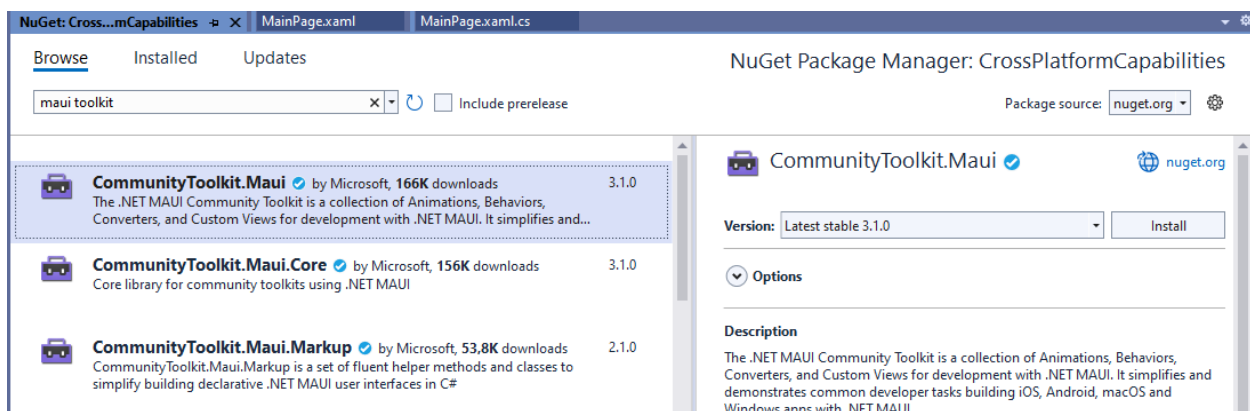


Figure 2: Installing the .NET MAUI Community Toolkit

The Community Toolkit must be initialized before you can use it. In the **MauiProgram.cs** file, change the **UseMauiApp** invocation as follows:

```
builder.UseMauiApp<App>().ConfigureFonts(fonts =>
{
    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
}).UseMauiCommunityToolkit();
```

 **Tip:** The .NET MAUI Community Toolkit ships with a new code analyzer that will highlight, with red squiggles, the lack of the invocation in the **UseMauiCommunityToolkit** method in the code editor. This is a very helpful reminder that you need to add it.

Another NuGet package that you need to install is the **CommunityToolkit.Mvvm** library from Microsoft, which is required to implement broadcast messages. As you will learn shortly, in .NET 7, the **MessagingCenter** class has been deprecated. The recommendation is to use a new class called **WeakReferenceMessenger**, provided by **CommunityToolkit.Mvvm**. This library has the wider purpose of supporting the implementation of the Model-View-ViewModel pattern in a variety of .NET applications. Figure 3 shows how it appears in the NuGet Package Manager.

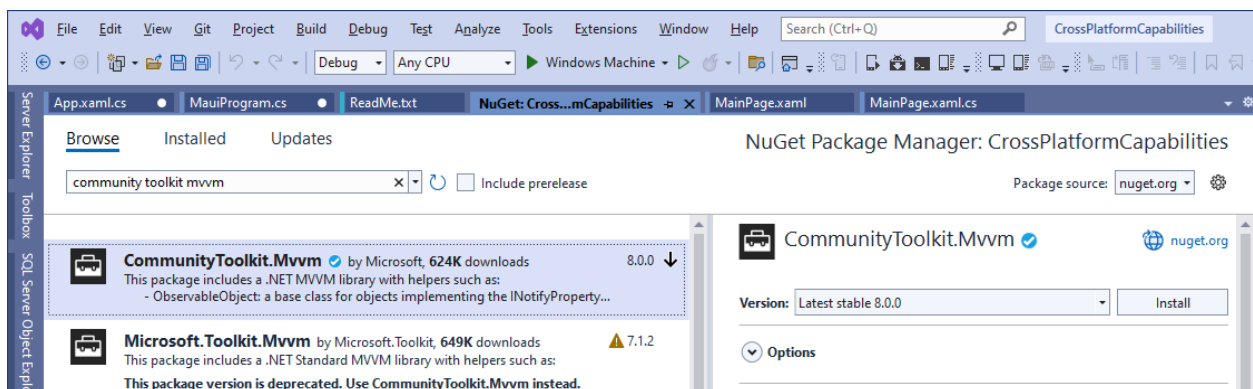


Figure 3: Installing the *CommunityToolkit.Mvvm* package

You are now ready to write some code, and you can always follow the example by opening the companion source code.

Managing network connections with snackbar views

Most applications today need an internet connection to work properly, and you, as the developer, are responsible for responding to changes to the connection status. For example, you need to inform users that the connection dropped, and depending on the scenario, you might also need to offer different actions.

You probably already know that handling changes in the status of the network connection is accomplished via the **Microsoft.Maui.Networking.Connectivity** class, but in this chapter I will show how to implement a snackbar view to inform the user.

This view does not exist in .NET MAUI, but it is offered by the .NET MAUI Community Toolkit. This kind of view has become very popular in many applications because it does not block user interaction while providing information. For a better understanding, consider Figure 4, where you see the final result of the work.

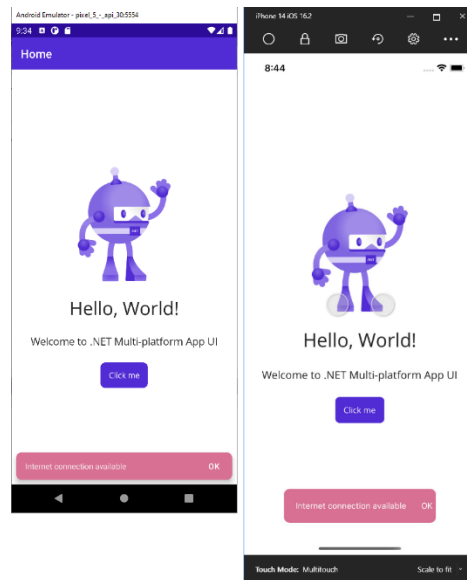


Figure 4: Displaying a snackbar when the connection status changes

As you can see, the user interface displays a nice view that informs the user when the connection status changes. The snackbar will disappear after 10 seconds. Let's now go through each step required to build this. In terms of code architecture, the sample app will need to do the following:

- Subscribe to changes in the network connection status. This is accomplished by handling the **Connectivity.ConnectionChanged** event in the **App** class so that changes are handled at the application level.
- Send a broadcast message to inform other objects in the app that a change has occurred. This is done via the **WeakReferenceMessenger** class and message implementations.
- Intercept broadcast messages about connection changes and display a snackbar via the **DisplaySnackbar** extension method.

These steps are described in detail in the next sections.

Defining custom messages

Broadcast messages allow objects in the application to notify other objects about a particular situation, such as that a task has been completed. This is very common when implementing the Model-View-ViewModel pattern, in which you cannot access views from a ViewModel, and therefore, you send a message from the ViewModel that is intercepted by a view, which takes the necessary actions.

If you have experience with Xamarin.Forms, you know that you could use the **MessagingCenter** class and its **Send**, **Subscribe**, and **Unsubscribe** methods to manage messages. However, with .NET MAUI and .NET 7, the **MessagingCenter** class is now deprecated. The official Microsoft [documentation](#) suggests using a new class called **WeakReferenceManager**, exposed by the CommunityToolkit.Mvvm library that you installed previously. The biggest difference from the old **MessagingCenter** is that, with the new approach, the message is a .NET object that contains all the information you need, whereas in the past you had to supply a message string and an instance of the object that contained the necessary information.

In the current example, the message is very simple: it just needs to let callers know if an internet connection is available once the status of the connection changes, which can be represented by just a **bool** value. To accomplish this, add a new class called **ConnectionChangedMessage** to the solution as follows:

```
// Requires using CommunityToolkit.Mvvm.Messaging.Messages;
public class ConnectionChangedMessage : ValueChangedMessage<bool>
{
    public ConnectionChangedMessage(bool value) : base(value)
    {
    }
}
```

As a general rule, a new message class inherits from the generic **ValueChangedMessage** class. The generic type parameter represents the data type that you need to exchange. In this case, you just need a Boolean value, but you can send very complex information to subscribers. You also need to supply an empty constructor explicitly that takes the specified type as an argument. The actual value, **true** or **false** in this case, will be sent along with the message. This is explained in the next section.

Handling the status of the network connection

In .NET MAUI, you get information about the network connection via the **Connectivity** class. Among others, the **Connectivity** class exposes an event called **ConnectivityChanged**, which is raised every time the status of the network connection changes. Because you might have multiple pages in your app that require a connection, the best place to handle the event is in the **App** class. First, add the following line right after the invocation to the **InitializeComponent** method, inside the constructor:

```
Connectivity.ConnectivityChanged += Connectivity_ConnectivityChanged;
```

The event handler might look like the following:

```
private void Connectivity_ConnectivityChanged(object sender,
    ConnectivityChangedEventArgs e)
{
    WeakReferenceMessenger.Default.
        Send(new ConnectionChangedMessage(
```



```

        e.NetworkAccess == NetworkAccess.Internet));
    }

```

Because the event is handled in the **App** class, and this does not have direct access to the user interface of any page, the best way to inform all the subscribers that the network status changed is to send a broadcast message. This is accomplished via the **Send** method from the **WeakReferenceManager** class, a singleton class whose instance is exposed by the **Default** property.

You need to pass an instance of the previously implemented message class as an argument, so supply a Boolean value for the constructor. This is **true** if an internet connection is available and **false** otherwise, no matter if the device is completely disconnected from any network, connected to a local network, or connected to a network with limited access. You can further manage the event using the values from the **NetworkAccess** enumeration if you need more granularity of control.

There is one additional thing to do: the network check will not happen immediately at startup, so you need to handle the **OnStart** method as follows:

```

protected override void OnStart()
{
    base.OnStart();
    WeakReferenceMessenger.Default.
        Send(new ConnectionChangedMessage(
            Connectivity.NetworkAccess == NetworkAccess.Internet));
}

```

Basically, the same check is done at startup, just once, and the same message is sent to the subscribers. The next step is notifying the app's user interface of the network status change and showing the snackbar.

Displaying a snackbar view

You can think of a snackbar as a box that appears as an overlay to provide information to the user. This can include errors, confirmations, warning messages, and so on. The benefits of using a snackbar are that it does not block user interaction with the app, and you have complete control over it.

For instance, you could animate a snackbar to appear, keep its information visible for a few seconds, and then disappear. A typical situation for this is when the user saves some changes and the app provides confirmation. Depending on the design needs, you might also build your own snackbar view, but in most situations, you can leverage the snackbar provided by the .NET MAUI Community Toolkit. Code Listing 1 shows how to implement this in the only page of the sample app (the MainPage.xaml.cs file). I'll comment on this shortly.

Code Listing 1

```

using CommunityToolkit.Maui.Alerts;
using CommunityToolkit.Maui.Core;

```

```

using CommunityToolkit.Mvvm.Messaging;

namespace CrossPlatformCapabilities;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();

        WeakReferenceMessenger.Default.
            Register<ConnectionChangedMessage>
            (this, async (sender, message) =>
            {
                await ShowConnectionSnackbarAsync(message.Value);
            });
    }

    private async Task ShowConnectionSnackbarAsync(bool value)
    {
        var options = new SnackbarOptions
        {
            BackgroundColor = Colors.PaleVioletRed,
            TextColor = Colors.White,
            ActionButtonTextColor = Colors.White,
            CornerRadius = new CornerRadius(10),
            Font = Microsoft.Maui.Font.SystemFontOfSize(14),
        };

        string message;
        switch(value)
        {
            case true:
                message = "Internet connection available";
                break;
            default:
                message = "Internet connection lost";
                break;
        }

        await this.DisplaySnackbar(message, visualOptions: options);
    }

    // Omitting Visual Studio autogenerated code...
}

```



Tip: In Code Listing 1, the code for the `OnCounterClicked` event (related to the sample button generated by Visual Studio when it created the solution) has been intentionally omitted. It is not relevant to the current discussion.

In the constructor, the page subscribes to the `ConnectionChangedMessage` via the `Register` method of the `WeakReferenceMessenger` class. This method takes two arguments: the instance of the subscriber, represented by the `this` keyword, and an object of type `MessageHandler`, represented by an asynchronous lambda, that takes the sender of the message and the instance of the message type as arguments.

When the message is intercepted, the `ShowConnectionSnackbarAsync` method is invoked, passing the Boolean value that represents the availability of an internet connection. This latter method first declares an instance of the `SnackbarOptions` class, customizing the appearance of the snackbar view via self-explanatory properties, and then it invokes the `DisplaySnackbar` extension method to display the appropriate string depending on the value of the connection status.

You can further customize the appearance of the snackbar by specifying custom text for the action button of the view, the view duration, and an action that must occur when the user clicks the action button. These are all parameters of the `DisplaySnackbar` method, explained in the [documentation](#) and in my book [.NET MAUI Community Toolkit Succinctly](#).

Checking the battery level

Checking the battery level of the device can be an important task, depending on the type of app you develop. Both iOS and Android inform the user with an alert when the battery level goes down to 20 percent, so if your app only displays data and does not require saving information, that might be enough.

Generally, it is not a good idea to bore users with additional alerts that provide the same information already provided by the system. As an alternative, you could consider the implementation of a more discrete snackbar that disappears after a few seconds if you need to make sure the user saves some information manually. This is a good idea if your app manages sensitive data and it is better to leave the final responsibility of data handling, including data persistence, to the user.

Another scenario is the need to temporarily persist some information in a silent way. To accomplish this, I will demonstrate how to use the `EnergySaverStatusChanged` event from the `Microsoft.Maui.Devices.Battery` class. This event is raised when the device enters in energy saver mode, which normally happens when the battery level falls to 20 percent and lower, but because this mode can be manually enabled by the user, I will also show how to combine this with a check on the actual battery level.

Before going into this, let me summarize what the **Battery** class has to offer. This class exposes the following properties:

- **ChargeLevel**, of type **double**, which returns the current battery charge level between 0 and 1 (where 0 means completely discharged and 1 means 100 percent charged).
- **BatteryState**, of type **BatteryState**, which describes the state of the battery and returns the information described in Table 1.
- **PowerSource**, of type **BatteryPowerSource**, which describes how the device is being powered and returns the values described in Table 2.

Table 1: BatteryState enumeration

Value	Description
Charging	The battery is currently charging.
Discharging	The battery is discharging. This is the expected value when the device is not connected to a power source.
NotCharging	The battery is not being charged.
NotPresent	A battery is not available in the device (e.g., a desktop computer).
Unknown	The state could not be detected.
Full	The battery is fully charged.

Table 2: BatteryPowerSource enumeration

Value	Description
Battery	The device is powered by the battery.
AC	The device is powered by an AC charger.
Usb	The device is powered through a USB cable.
Wireless	The device is powered via wireless charging.
Unknown	The power source could not be detected.

When one of the aforementioned properties changes, the **BatteryInfoChanged** event is raised, and you will be able to read the property's status.



Note: *On Android, you will need to select the `BATTERY_STATS` permission in the manifest before accessing information on the battery. UWP and iOS do not require additional steps.*

The next example focuses on the `EnergySaverStatusChanged` event, for which you specify a handler in the constructor of the `App` class as follows:

```
Battery.EnergySaverStatusChanged += Battery_EnergySaverStatusChanged;
```

The event handler will check if the phone entered energy saving status, but because this can be manually enabled by the user, it will also check if the battery level is equal to or lower than 20 percent. This is accomplished with the following code:

```
private void Battery_EnergySaverStatusChanged(object sender,
    EnergySaverStatusChangedEventArgs e)
{
    // Remove the ChargeLevel check if your app
    // implements background services
    WeakReferenceMessenger.Default.
        Send(new BatteryStatusChangedMessage(
            e.EnergySaverStatus == EnergySaverStatus.On
            && Battery.ChargeLevel <= 0.2));
}
```

In this case, any tasks related to persisting user settings or storing local information will occur only if there is a real need. A message is sent to subscribers, which can then react to the status change. The `BatteryStatusChangedMessage` is a custom message that you need to implement in the project as follows:

```
// Requires using CommunityToolkit.Mvvm.Messaging.Messages;
public class ConnectionChangedMessage : ValueChangedMessage<bool>
{
    public ConnectionChangedMessage(bool value) : base(value)
    {
    }
}
```

This is very similar to what you have implemented for checking the network connection in the first part of this chapter.

Reacting to the energy saving status in the user interface

The first thing that a page must do is subscribe to the message sent by the `App` class when the phone enters the energy saving status. In the current example, the following line must be added to the constructor of the `MainPage.xaml.cs` file:

```
WeakReferenceMessenger.Default.Register<BatteryStatusChangedMessage>(
    this, (sender, message) =>
    {
        ManageBatteryLevelChanged();
    });
```

The `ManageBatteryLevelChanged` method will be responsible for performing the actual actions. This is its code:

```
private void ManageBatteryLevelChanged()
{
    FileHelper.WriteData("test data");
}
```

`FileHelper` is a class that must be implemented and that exposes methods for reading and writing text files locally. Code Listing 2 shows the full code for the class.

 **Note:** *For Android, you will need to enable the `READ_EXTERNAL_STORAGE` and `WRITE_EXTERNAL_STORAGE` permissions in the app manifest.*

Code Listing 2

```
namespace CrossPlatformCapabilities
{
    public static class FileHelper
    {
        public static string ReadData()
        {
            try
            {
                string fileName = Path.Combine(Environment.GetFolderPath(
                    Environment.SpecialFolder.LocalApplicationData),
                    "appdata.txt");

                string text = File.ReadAllText(fileName);

                return text;
            }
            catch (Exception)
            {
                return null;
            }
        }
    }
}
```

```

public static bool WriteData(string data)
{
    try
    {
        string fileName = Path.Combine(Environment.GetFolderPath(
            Environment.SpecialFolder.LocalApplicationData),
            "appdata.txt");

        File.WriteAllText(fileName, data);

        return true;
    }
    catch (Exception)
    {
        return false;
    }
}
}

```

The **ReadData** and **WriteData** methods are very simple. They open a stream for reading and writing a file, respectively, and then they read or write the file content as text. The **WriteData** method is invoked by the **ManageBatteryLevelChanged** method in the user interface's code-behind file, which silently saves some user information when the battery level is very low. Such information can then be read back by invoking the **ReadData** method, which returns a string.

In the current example, everything happens silently, but you could implement a snackbar that informs the user about the battery level being low and the automatic information backup. Using a text file for local persistence is the simplest approach, but you can take this further by persisting user and app settings to the local device storage or into an SQLite database. App settings and SQLite databases will be discussed in Chapters 2 and 4, respectively. Keep an eye on the **FileHelper** class, because it will be extended in the next section with methods to work with the file system.

Working with files

Working with files is a common requirement in mobile apps. Due to the natural system restrictions on mobile devices, you will usually only work with image files, video files, and document files. With .NET MAUI, you can do this cross-platform. This section discusses two topics: accessing data in the local app folders and how to select files.

Accessing files in the local app folders

In terms of cross-platform development, you will be able to access two file locations: the cache directory where temporary files are stored, and the app data folder where app data files are usually stored. Note that the cache directory is not usually visible to the user, and the data it contains can be removed at any time due to its temporary nature.

For a better understanding of how reading files from these locations works, consider the following method that you can add to the **FileHelper** class to build a reusable class for file access:

```
public async static Task<string> ReadDataAsync()
{
    try
    {
        var mainDir = FileSystem.AppDataDirectory;
        string localData;

        using (var stream = await
FileSystem.OpenAppPackageFileAsync("appdata.txt"))
        {
            using (var reader = new StreamReader(stream))
            {
                localData = await reader.ReadToEndAsync();
            }
        }
        return localData;
    }
    catch (Exception)
    {
        return null;
    }
}
```

The **FileSystem** class exposes two properties: **AppDataDirectory** and **CacheDirectory**, both self-explanatory. It also exposes a method called **OpenAppPackageFileAsync**, which allows for reading the content of a file. Regarding this, there are some important considerations:

- Only text files are supported.
- As the name implies, **OpenAppPackageFileAsync** allows for opening a file that is part of the app resources. You can only open text files that were previously added to the platform-specific projects.
- Continuing from the previous point, for Android you will need to place the text file under the **Assets** folder and set its **Build Action** property to **AndroidAsset**. On iOS, the text file must be placed under the **Resources** folder and its **Build Action** set to **BundleResource**. On UWP, the file must be placed in the project folder and its **Build Action** set with **Resource**.

Accessing files in the local app folders is useful, but it is even more useful to learn how to select an existing file from the user's device.

Selecting files from the device

If your app needs to allow users to select files from the device, .NET MAUI has the cross-platform solution for you. Let's start by supposing your app lets users select and display an image file. You will need a button to select an image, a label to display the selected file's name, and an **Image** view to display the image. Code Listing 3 shows the XAML for this.

Code Listing 3

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="CrossPlatformCapabilities.FilePickerPage"
  Title="FilePickerPage">
  <ContentPage.Content>
    <Grid>
      <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition />
      </Grid.RowDefinitions>

      <Button x:Name="PickButton" Clicked="PickButton_Clicked"
        HorizontalOptions="Center"
        Text="Pick an image"/>

      <Label x:Name="ImageNameLabel"
        HorizontalTextAlignment="Center"
        HorizontalOptions="Center" Grid.Row="1"/>

      <Image x:Name="PickedImage" Grid.Row="2"/>
    </Grid>
  </ContentPage.Content>
</ContentPage>
```

Nothing complex at all. Because you want to display the image and its file name, you might want to implement a class that stores this information from the selected file, such as:

```
public class FileSelection
{
    public string FileName { get; set; }
    public ImageSource Image { get; set; }
}
```

Now it is time to write a method for file selection. The **FilePicker** class exposes a method called **PickAsync**, which opens the system user interface for file selection, which returns an object of type **FileResult**. An instance of **FileResult** will contain the file name, the full path, and a stream object that contains the selected file.

Code Listing 4 demonstrates how to invoke **PickAsync** and how to store the result in the **FileSelection** class we created previously.

Code Listing 4

```
private async Task<FileSelection> PickAndShowAsync(PickOptions options)
{
    try
    {
        FileResult result = await FilePicker.PickAsync(options);
        FileSelection fileResult = new FileSelection();
        if (result != null)
        {
            fileResult.FileName = $"File Name: {result.FileName}";
            if (result.FileName.EndsWith(".jpg",
                StringComparison.OrdinalIgnoreCase) ||
                result.FileName.EndsWith(".png",
                StringComparison.OrdinalIgnoreCase))
            {
                var stream = await result.OpenReadAsync();
                fileResult.Image =
                    ImageSource.FromStream(() => stream);
            }
        }
        return fileResult;
    }
    catch (Exception ex)
    {
        return null;
        // The user canceled or something went wrong.
    }
}
```

The method also takes a parameter of type **PickOptions**, but this will be discussed shortly. The key points to focus on now are:

- **PickAsync** returns the selected file as a **FileResult** object.
- PNG and JPG image files have built-in support, so other types are filtered out (see the **if** block).
- The file name and the actual image are assigned to an object of type **FileSelection** and returned as the result of the method. The image in particular is returned as a stream.

The final piece is the **Clicked** event handler for the button, which will look like the following:

```
private async void PickButton_Clicked(object sender, EventArgs e)
{
    var result = await PickAndShowAsync(
        new PickOptions { PickerTitle="Pick a file",});
    this.ImageNameLabel.Text = result.FileName;
    this.PickedImage.Source = result.Image;
}
```

The key point here is the invocation to the **PickAndShowAsync** method. Notice how an instance of **PickOptions** allows you to customize the user interface for file selection. You can actually specify different text for the picker and, as you will see in the next section, this is also the place where you add support for different file types. If you run the code and select an image, you will see a result similar to Figure 5.



Figure 5: Displaying an image selected from the device

You can support different file types, which will be explained in the next section.

Development trick: Running code in the main thread

To keep the user interface active and capable of accepting user input even when an app is busy doing something else, most operating systems (iOS, Android, and UWP are no exception) run the user interface on a dedicated thread known as the main thread, user interface thread, or UI thread.



Note: *For consistency with API naming, I will refer to this thread as the main thread.*

All code that interacts with elements of the user interface should run within this thread. Otherwise, exceptions would be raised because of invalid cross-thread calls. However, this does not mean that an app only runs on a single thread. For example, classes like **Battery** or **Connectivity** that you saw at the beginning of this chapter raise events on a separate thread to avoid a work overload on the main thread.

If part of the code that runs on a separate thread needs to interact with elements of the user interface, that piece of code needs to run on the main thread instead. For instance, an event handler might contain some logic and code that changes properties of UI elements. The logic part can stay on a separate thread, but the piece of code that works against UI elements needs to run on the main thread.

.NET MAUI simplifies running code in the main thread via the **MainThread** class. If you want to run code on the main thread, all you do is write something like this:

```
MainThread.BeginInvokeOnMainThread(() =>
{
    // Run your code on the main thread here...
});
```

The body for the **BeginInvokeOnMainThread** method is an **Action** that you can represent with a lambda expression, like in the previous snippet, or by explicitly declaring a method name as follows:

```
MainThread.BeginInvokeOnMainThread(DoSomething);
...
void DoSomething()
{
    // Run your code on the main thread.
}
```

The fact is that you might not know if a piece of code needs to run on the main thread or not. Luckily enough, the **MainThread** class exposes the **IsMainThread** static property, which returns **true** if the current piece of code is already running on the main thread. You use it as follows:

```
if (MainThread.IsMainThread)
    DoSomething();
else
```

```
MainThread.BeginInvokeOnMainThread(DoSomething);
```

The **MainThread** class is extremely useful. It will help you avoid errors and save time detecting which thread your code is running on.

Opening the browser and sending emails

Sometimes you need to open a browser on the device for navigating to a webpage, or your app might need to allow users to send emails. .NET MAUI provides APIs for both scenarios, which are discussed in this section.

For the next example, add a new **ContentPage** to the project with the XAML shown in Code Listing 5, which declares two simple buttons.

Code Listing 5

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="CrossPlatformCapabilities.BrowserEmailSample"
  Title="BrowserEmailSample">
  <ContentPage.Content>
    <StackLayout VerticalOptions="Center">
      <Button Text="Open Web page"
        x:Name="BrowserButton"
        Clicked="BrowserButton_Clicked"/>
      <Button Text="Open email client"
        x:Name="EmailButton"
        Clicked="EmailButton_Clicked"
        Margin="0,15,0,0"/>
    </StackLayout>
  </ContentPage.Content>
</ContentPage>
```

The event handlers of each button will be implemented to work with the two scenarios taken into consideration.

Opening the browser

.NET MAUI provides the **Browser** class, which makes it really simple to open an external browser pointing to the specified URI via the **OpenAsync** method. While you can display web contents, including websites, inside the **WebView** control, there might be several reasons for displaying a website with an external browser.

The first reason is certainly the fact that web browsers are more optimized to display websites than the **WebView**. I will explain other reasons with a practical example coming from my daily work. The main project I work on is an app for dialysis patients, and we had the requirement to provide links to official sources about COVID-19 from the World Health Organization (WHO) website. We made the decision to display such webpages inside an external browser for the following reasons:

- If we implemented our browsing system inside the app, the user would have the perception that contents from the WHO were offered by us, which is not true.
- In the unlucky (and unlikely) event the website was hacked, the user would have the perception that it was our fault, which is not true.
- Because of the sensitivity of the information, the user had to be very clear on the source of the information, and that would not be possible (or at least very difficult) with in-app browsing.
- Security is key for us, so we thought that delegating this responsibility to the browser was the best approach.

The simplest way to open a website inside a browser is by invoking the **OpenAsync** method as follows:

```
private async void BrowserButton_Clicked(object sender, EventArgs e)
{
    await Browser.OpenAsync("https://www.microsoft.com");
}
```

If multiple browsers are installed on your device, the system will show a user interface where you can select a browser for opening the website. With this simple line of code, a browser will be opened and will run externally. You can also open a web browser instance inside the app, changing the previous line of code as follows:

```
await Browser.OpenAsync("https://www.microsoft.com",
    BrowserLaunchMode.SystemPreferred);
```

The **BrowserLaunchMode** enumeration has two values: **External1**, which is basically the default and works exactly like the first example; and **SystemPreferred**, which makes the browser run inside the app. Figure 6 shows an example where on the right, the browser is running externally, and on the left, the browser is running inside the app, as demonstrated by the availability of the close (X) button.

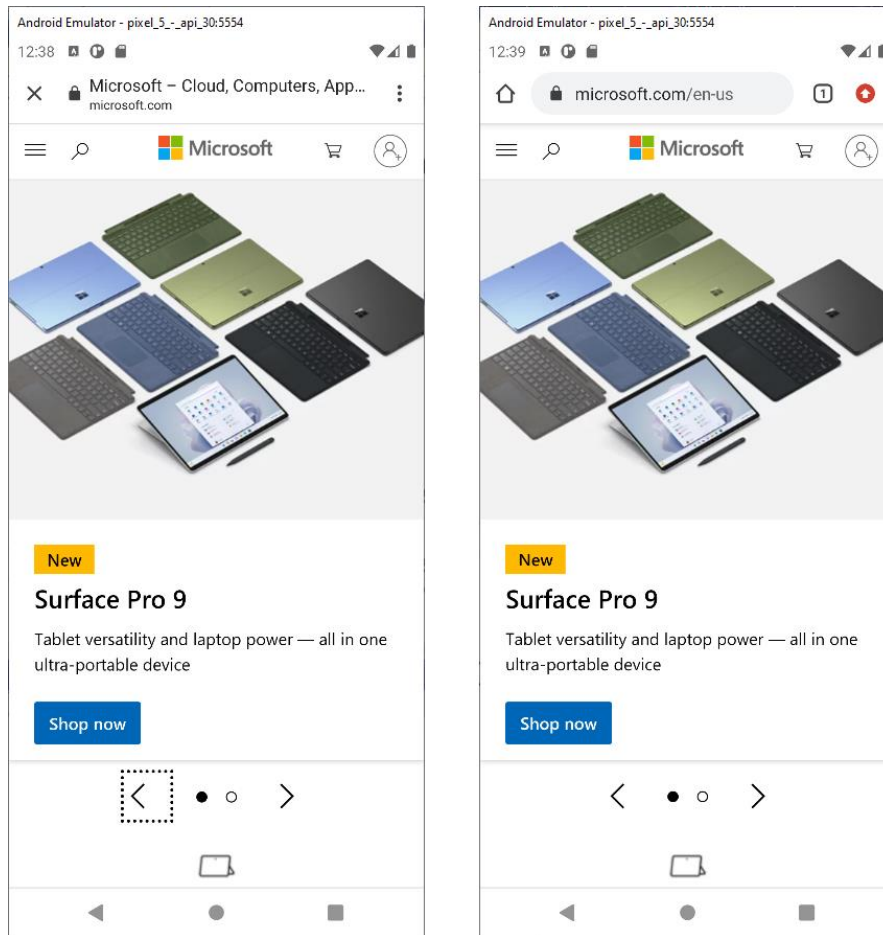


Figure 6: Opening the browser outside and inside the app

As you can see, when the browser runs externally, you will have the browser controls available. When it runs inside the app, you will have fewer controls available, but the user does not need to switch between apps. On iOS, when the browser runs externally, you will see a shortcut at the top-left corner of the screen that allows you to go quickly back to the app that opened the browser.

Sending emails

Similar to external web browsing is sending emails. Creating a dedicated user interface would take time, and you would need to care about a lot of information. .NET MAUI provides the **Email** class, which exposes the **ComposeAsync** method and allows for opening one of the installed email clients on your device. In its simplest form, the method works like this:

```
await Email.ComposeAsync();
```

This will open an email client on the device, and the user will need to supply all the email information manually. Another method overload allows for specifying a minimum set of information as follows:

```
await Email.ComposeAsync("Support request",
    "We have problems with the Internet connection",
    "support@onecompany.com");
```

The first argument is the email subject, the second argument is the email body, and the third argument is an array of **string** containing the list of recipients. However, there is a third method overload, which is much more powerful, and gives you complete control of the email. Such overload takes an argument of type **EmailMessage** and works like in the following example (which is also the event handler for the button in the sample app):

```
private async void EmailButton_Clicked(object sender, EventArgs e)
{
    EmailMessage message = new EmailMessage();
    message.Subject = "Support request";
    message.To = new List<string> { "support@onecompany.com" };
    message.Cc = new List<string> { "myboss@mycompany.com" };
    message.BodyFormat = EmailBodyFormat.PlainText;
    message.Body = "We have problems with the Internet connection";

    await Email.ComposeAsync(message);
}
```

As you can see, the **EmailMessage** class exposes self-explanatory properties and allows you to specify if the body should be plain text (**EmailBodyFormat.PlainText**) or HTML (**EmailBodyFormat.HTML**). Notice that HTML is not supported on Windows. You can define multiple recipients in the **To**, **Cc**, and **Bcc** properties via **List<string>** objects.

The **EmailMessage** class also exposes the **Attachments** properties, which allows you to programmatically attach files to the email message. The following code demonstrates how to add an attachment:

```
string attachmentPath =
    Path.Combine(Environment.GetFolderPath(
        Environment.SpecialFolder.MyPictures),
        "myimage.png");
EmailAttachment attachment = new EmailAttachment(attachmentPath);

message.Attachments = new List<EmailAttachment>();
message.Attachments.Add(attachment);
```

If you plan for the app to run on Android 11 devices and higher, you will need to add the following XML markup to the Android manifest before running the example:

```
<queries>
  <intent>
    <action android:name="android.intent.action.SENDTO" />
    <data android:scheme="mailto" />
  </intent>
```


</queries>

 **Tip:** For iOS, the `Email` class is only supported on physical devices. On the simulator, it will throw a `FeatureNotSupportedException`.

On iOS and Mac Catalyst, you instead need to add the following key to the Info.plist file:

```
<key>LSApplicationQueriesSchemes</key>
<array>
  <string>mailto</string>
</array>
```

If you run the example, you will first see the operating system asking which email client you want to use (if more than one is on the system), and then you will see a new email message with all the required information supplied and ready to be sent. Figure 7 shows an example of the new email message inside the client.

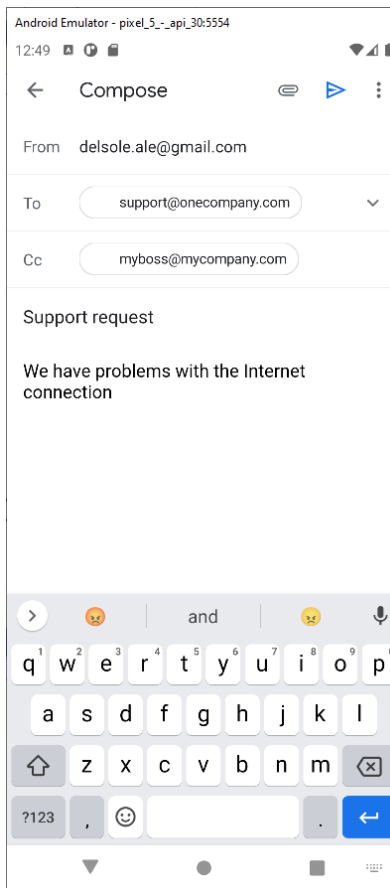


Figure 7: Email client selection and an email message ready to be sent

With a few lines of code, you can make your users able to send emails without the need to implement a custom user interface and logic.

Chapter summary

Mobile apps in the real world often need to implement features that need to work cross-platform. In this chapter, you have seen how to build a snackbar to inform users about the loss of network connection, and you have seen how to check for the battery level to make sure the app can persist important information. You have then seen how to work with local files, and how you can open the browser and send emails using system resources that avoid the need for custom user interfaces and additional effort for developers.

All these capabilities are provided by the MAUI codebase, and in the next chapter, you will learn other cross-platform capabilities related to the app settings and secure storage.

Chapter 2 Local Settings: Preferences and Secure Storage

Storing user preferences and app settings locally are very common requirements with cross-platform projects. There are tons of examples, including storing the last access date/time, the consent to use a cellular network when wi-fi is not available, color scheme preferences, and so on. These options do not generally need to be secured, but sometimes you might need to store an encrypted password or information that identifies the user. In this case, you do need to secure both preferences and settings.

The .NET MAUI code base provides types that make it extremely easy to store local settings. This chapter describes both scenarios and provides details on the classes, and their methods, you can use for these purposes. You can use the solution available inside the Chapter2\LocalSettings folder of the companion code, or you can create a new .NET MAUI project from scratch. If you do so, remove all the code generated by Visual Studio 2022: the entire **ScrollView** node in the MainPage.xaml file and the **OnCounterClicked** event handler in its code-behind file.

Managing local preferences

Local preferences in .NET MAUI can be easily managed via the **Preferences** class from the **Microsoft.Maui.Storage** namespace. The **Preferences** class stores preferences in the native local storages, more specifically:

- For Android, into [Shared Preferences](#).
- For iOS, into [NSUserDefaults](#).
- For UWP, into the application data container.

On each platform, preferences are key/value pairs. The value can be of one of the primitive .NET types. Before understanding how they work, it is important to know the methods exposed by the **Preferences** class, summarized in Table 3.

Table 3: Methods exposed by the Preferences class

Methods exposed by the Preferences class	
Set	Adds a preference to the local settings.
Get	Gets a preference value from the local settings.
Remove	Removes the specified preference.
Clear	Deletes all the preferences for the current app.

Methods exposed by the Preferences class	
Default.ContainsKey	Detects if the specified preference exists in the storage given its key.



Tip: *Uninstalling an app will also remove all local preferences.*

Let's now start with an example. Suppose you want to store the last date/time the app has been used. You can do this in the **OnSleep** method of the App.xaml.cs file as follows:

```
protected override void OnSleep()
{
    Preferences.Set("TimeOfLastUsage", DateTime.Now);
}
```

TimeOfLastUsage is the key, and the value is a **DateTime** object. Keys are always of type **string**. Retrieving a value from the local settings is still easy. In the **OnStart** method, you can write something like the following:

```
internal static DateTime timeOfLastUsage;
protected override void OnStart()
{
    timeOfLastUsage = Preferences.Get("TimeOfLastUsage",
        DateTime.Now);
}
```

The generic **Get** method returns the value for the specified key and stores the result in a variable. **Get** also requires you to specify a default value in case the key does not exist in the storage, and this is true for all the supported data types. In this example, local preferences are used to store information that can be useful for the app logic at runtime, so it's not really a user preference.

Let's make another example for this purpose: checking if the phone is connected to a cellular network only, asking the user permission to use it, and storing the result. Code Listing 6 contains the code that you want to add to the MainPage.xaml.cs file.

Code Listing 6

```
private bool CheckCellularConnection()
{
    var profiles = Connectivity.ConnectionProfiles;
    return profiles.Count() == 1 &&
        profiles.Contains(ConnectionProfile.Cellular);
}

private bool useCellularNetwork;
```

```

protected override async void OnAppearing()
{
    if(CheckCellularConnection())
    {
        if (!Preferences.ContainsKey("UseCellularNetwork"))
        {
            bool result = await DisplayAlert("Warning",
                "Do you agree on using cellular data when Wi-Fi is not available?",
                "Yes", "No");
            Preferences.Set("UseCellularNetwork", result);
            useCellularNetwork = result;
        }
        else
            useCellularNetwork =
                Preferences.Get("UseCellularNetwork", false);
    }
}

```

The **CheckCellularConnection** method checks if only one connection is available, and if this is a cellular network. The remaining code is in the **OnAppearing** method for convenience so that the code is executed at the app startup, but you will likely create a separate method that will be invoked when the **ConnectivityChanged** event of the **Connectivity** class is raised.

In this second example, the code invokes the **Preferences.ContainsKey** method to detect if the preference was already saved, then it still invokes the **Set** and **Get** methods to store and retrieve the preference value, respectively. The **Preferences** class is intended to store small pieces of information represented by primitive .NET types. If you need to save and retrieve more complex and structured data locally, your best option will be an SQLite database, as discussed in Chapter 4.

Securing preferences and settings: secure storage

Sometimes you need to store information locally, but you also need the information to be secured. Each of the supported operating systems offer a secure place to store encrypted information, and the .NET MAUI library provides the **SecureStorage** class, which exposes methods to work with secure storages in a cross-platform way.

There are a few differences with nonsecured preferences discussed in the previous section:

- While preferences are stored inside the app's own cache, the secure storage can be accessed by multiple applications.
- As an implication, data you save in the secure storage is not removed when you uninstall an app. This is extremely important to consider.
- You still work with key/value pairs, but the value can be of type **string** only.

The native implementation for the secure storage can be summarized as follows:

- For iOS, it is located in the KeyChain.
- For Android, it is located in the KeyStore.
- For UWP, data is encrypted with the **DataProtectionProvider** algorithms.

When working with iOS simulators, you will need to take a few more steps to set up secure access; otherwise, you will get an exception. This is explained in the next paragraph.

Configuring iOS to work with simulators

If you want to test and debug your code on iOS simulators, you need to explicitly enable the KeyChain in Visual Studio. This is accomplished by editing the Entitlements.plist file. However, unlike with Xamarin.Forms, in .NET MAUI this file is not added by default when the solution is generated. So, manually add a new XML file called **Entitlements.plist** to the Platforms\iOS subfolder of the project and add the following XML root node:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <dict>
  </dict>
</plist>
```

Save the file and close the code editor. At this point, follow these steps:

1. Double-click the **Entitlements.plist** file in Solution explorer.
2. When the editor opens up, locate the **Keychain** element and enable it.
3. In the project properties, select the **Bundle Signing** tab located under the iOS node at the bottom of the page.
4. Assign the **Entitlements.plist** file as the source for **Custom Entitlements**.

These entitlements should be removed when working on physical devices and before distribution.

Implementing access to secure storage

The biggest benefit of the secure storage is that it automatically encrypts the information with the highest levels of security possible offered by each operating system. The **Microsoft.Maui.Storage.SecureStorage** class exposes two simple methods: **SetAsync** and **GetAsync**. The first method saves the specified key and value, and the second method retrieves the value for the specified key. If the key does not exist, it returns **null**, but you cannot specify a default value like you do with local preferences.

To understand how the class works, suppose you have the following user interface where the user can enter a password:

```

<VerticalStackLayout VerticalOptions="CenterAndExpand"
    HorizontalOptions="FillAndExpand">
    <Label Text="Enter your password: "
        FontSize="Large"
        Margin="0,20,0,0"/>
    <Entry IsPassword="True"
        x:Name="PasswordEntry"
        Margin="0,10,0,0"/>
    <Button Text="OK" Margin="50,20,50,0" x:Name="OkButton"
        Clicked="OkButton_Clicked"/>
</VerticalStackLayout>

```

The **Clicked** event handler for the button contains the code that stores the password in the local storage:

```

private async void OkButton_Clicked(object sender, System.EventArgs e)
{
    // Perform password validation here...

    await SecureStorage.SetAsync("Password", PasswordEntry.Text);
}

```

Obviously, you will implement your logic for password validation, but for the sake of simplicity, the code stores the password directly. The key points are the two string parameters, key and value, for the **SetAsync** method, and the usage of the **await** operator, as this is an asynchronous call. The code you need to write to retrieve information from the secure storage looks like the following, related to the current example:

```

// Assuming you have declared a bool field called hasPassword
string password = await SecureStorage.GetAsync("Password");
if (password != null)
    hasPassword = true;

```

GetAsync returns the value for the specified key if the key exists in the secure storage. Otherwise, it returns **null**, so a null check is necessary to avoid exceptions. As you can see, reading and writing data against the secure storage is very straightforward, though I recommend you use this feature only with really sensitive information.

Secure storage tips and tricks

As I mentioned previously, when you uninstall an app that stores information in the secure storage, such information is not removed because the secure storage is at the operating system level, not at the app level (though the OS knows what app is associated with the key). You certainly cannot control when an app is uninstalled, but if the app is running on the device for the first time (which includes a reinstall), you can check whether a key already exists in the secure storage, so that it can be removed and a fresh environment provided to the user. The following method demonstrates how to accomplish this:

```
private async Task CheckAppFirstRun()
{
    if(VersionTracking.IsFirstLaunchEver)
    {
        string password = await SecureStorage.GetAsync("password");
        if (password != null)
            SecureStorage.Remove("password");
    }
}
```

The **VersionTracking** class from the **Microsoft.Maui.ApplicationModel** namespace provides properties and methods that help you work with the app's versions and builds. **IsFirstLaunchEver** returns true if the app is running for the first time on the device. The **SecureStorage.Remove** method deletes the key and its value from the secure storage. You can avoid this code if you instead want to reuse the value of an existing key.



Note: Remember to invoke *VersionTracking.Track* the very first time the app runs; otherwise, the class will not be able to generate version information.

Chapter summary

Storing preferences and user settings is a very frequent task in mobile applications, and the .NET MAUI library provides the **Preferences** class for it, which allows for saving key/value pairs in the app's local storage. Supported data types are .NET primitive types. If the information you need to store locally requires encryption, you can leverage the secure storage associated with each OS, for which .NET MAUI offers the **SecureStorage** class, which supports key/value pairs of type **string**.

In the next chapter, you will see a very practical example about using the secure storage, which is the implementation of biometric authentication.

Chapter 3 Implementing Biometric Authentication

Biometric authentication is the option to log in to an application using a fingerprint sensor or face recognition feature, also known as face ID. This helps you avoid typing a username and password every time. This option is very common and popular in modern applications that require authentication, and you will likely need to implement it yourself in your apps. There are several reasons for doing this.

The first reason is simplifying the user experience by leveraging sensors on devices. The second reason, which is connected to the first one, is that typing credentials every time on a small keyboard might be annoying. Therefore, the user might quickly become tired of using your app—even if it is the best app on the market. This assertion might seem strange, but it's not. It actually comes from research on analytics and on user experience. With the incredible number of apps available, you might build a killer app on a certain topic, but if it's not easy, fast, and fun to use, people will move to another app on the same topic.

Login features are the users' first contact with an app, and entering into the app must be as simple, fast, and smooth as possible. You will agree that using the fingerprint or face ID is much faster than typing a username and password every time. This chapter explains how to implement biometric authentication in .NET MAUI, adding some steps for local password storage, with some logic that you will be able to reuse against web services, custom APIs, or even Azure.

Setting up libraries for biometric authentication

.NET MAUI does not include a built-in API that allows for implementing biometric authentication in a cross-platform way, so in theory you should write platform-specific code. However, within the increasing ecosystem of third-party libraries, you can use the [Biometric/Fingerprint plugin for Xamarin](#), an open-source project by Sven-Michael Stübe, which is now also supporting .NET MAUI. It is free, easy to use, and offers a cross-platform approach to leveraging all the biometric authentication features on even the most recent devices.

With this in mind, create a new .NET MAUI project. When ready, open the NuGet Package Manager user interface, then locate the Plugin.Fingerprint package. At the time of this writing, you need to enable the **Include prerelease** flag and select version **3.0.0-beta 1** (see Figure 8).

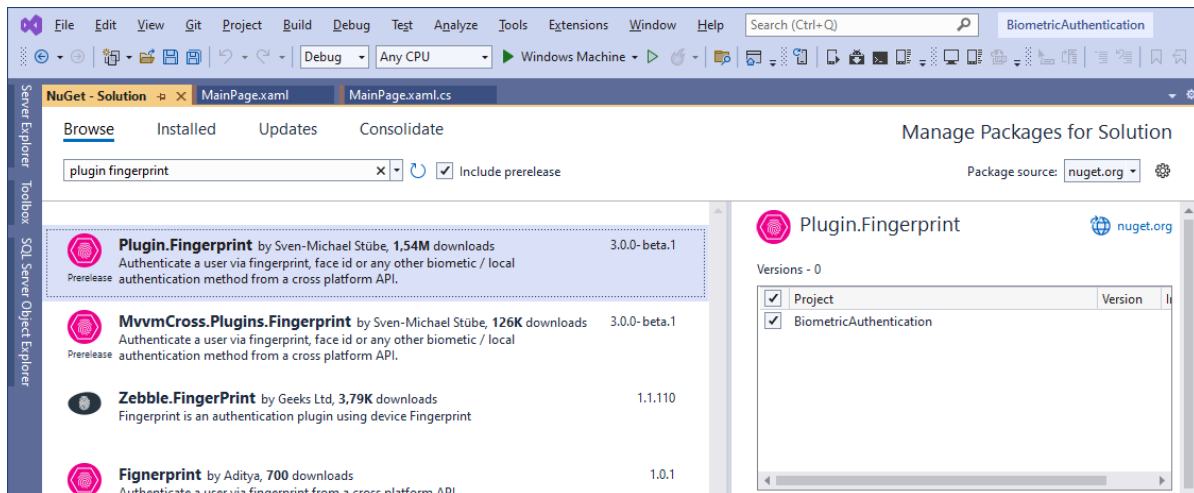


Figure 8: Adding the *Plugin.Fingerprint* library to the solution

When you're ready, click **Install** to complete the installation of the library.

Additional setup for iOS

You explicitly need to enable biometric authentication on iOS devices by adding the following key to the Info.plist file:

```
<key>NSFaceIDUsageDescription</key>
<string>Biometric authentication</string>
```

The fastest way is by opening the Info.plist file with Visual Studio's XML editor. To accomplish this, right-click the file, select **Open With**, and then **XML Editor**. You can replace the text for the previous **string** node with a custom one.

Additional setup for Android

The Android API has supported biometric authentication since Level 23 (which corresponds to Android 6.0). It requires explicit user permission. So, open the project properties and select the **Manifest** tab located under the **Android** node (see Figure 9). Make sure the **Minimum Android Version** is set to at least Android 6.0 (API Level 23—Marshmallow), as shown in Figure 9. Obviously, later versions are okay, but earlier versions do not support biometric authentication.

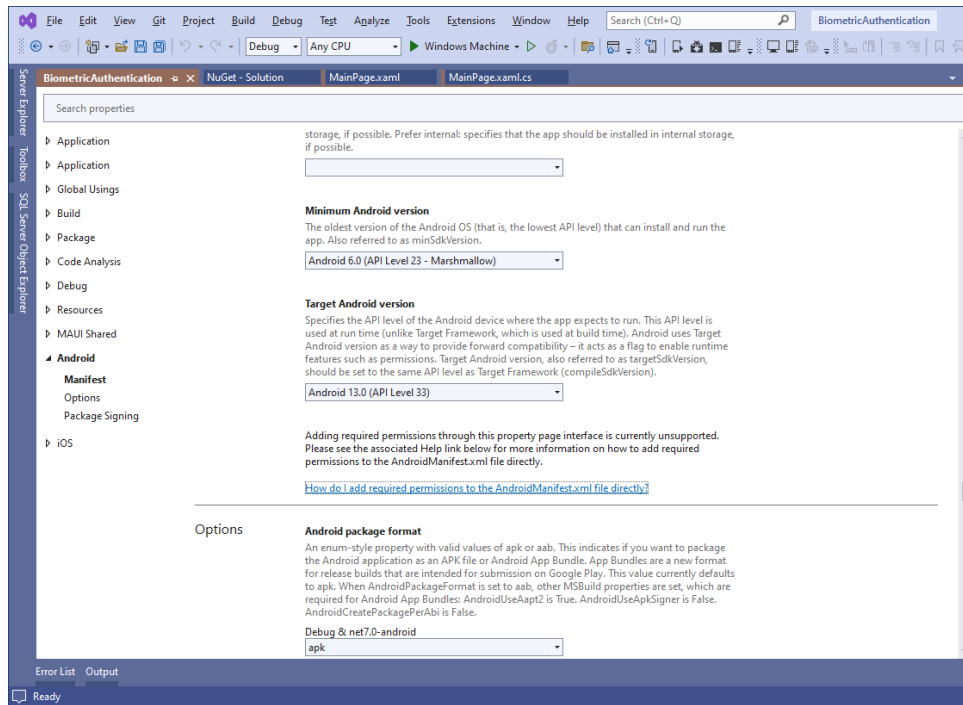


Figure 9: Setting Android requirements

In addition, the **USE_FINGERPRINT** permission is required. Double-click the **AndroidManifest.xml** file in Solution Explorer (located under Platforms\Android) and add the following key:

```
<uses-permission android:name="android.permission.USE_FINGERPRINT" />
```

The last step for Android is initializing the plugin. This is accomplished by adding the following invocation inside the **OnCreated** method of the **MainActivity.cs** file:

```
// Requires a using Plugin.Fingerprint; directive
protected override void OnCreate(Bundle savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    CrossFingerprint.SetCurrentActivityResolver(() => this);
}
```

The **SetCurrentActivityResolver** method will initialize the plugin, and the **this** qualifier represents the current activity as the object that is initializing the plugin itself.

The logic of the sample project

The biometric authentication APIs in each system tell developers whether a user is recognized by the device based on the fingerprint or face ID. Being recognized by the device does not mean being recognized, authenticated, and authorized by your application or your web services.

In real-world development, the flow would be the following:

1. The user registers, providing a username and a password. These credentials are usually validated against a web service and stored inside a database, on-premises or in-cloud, and are passed by the app every time it needs to access secured information.
2. The credentials supplied at registration are also usually encrypted and stored locally on the device.
3. When the user wants to manually log in, after registration, the app looks for a secured password on the device. If not found, the user is not registered. If found, the app makes a first match between the password typed at login with the secured, local one. If they match, the password is included in the calls to APIs, web services, or any remote service and undergoes a server-side validation before getting the information.
4. With biometric authentication, the app looks for a secured password on the device. If not found, the user is not registered. If found, it means the user is registered, so the biometric authentication user interface is shown. If the device recognizes the user, the local, secure password is sent to any API call, web services, or remote service and undergoes a server-side validation before getting the information.

There are an infinite number of scenarios and web services, APIs, and cloud services you might be using in your daily development, so the perfect server-side example does not exist and would be out of the scope of this book. For this reason, the sample application will simulate the inclusion of the password to be sent to a service, but will still show the entire flow of registration, login, and biometric authentication with password management.

Creating a sample user interface

Based on the considerations of the previous paragraph, the user interface of the sample applications consists of three pages, more specifically:

- The auto-generated main page will serve as the welcome page and will offer two buttons: one for registration and one for login.
- One page for registration, where the user will be able to supply and save a password. For the sake of simplicity, no username will be required.
- One page for the login part, with options to log in by typing the password manually and with biometric authentication.

Now, let's add two **ContentPage** items to the project: one called **RegisterPage.xaml** and one called **LoginPage.xaml**. In addition, remove all the UI code generated by Visual Studio in the **MainPage.xaml** file, and the **OnCounterClicked** event handler in the **MainPage.xaml.cs** file.



Note: As you will see in the following code snippets and listing, this example is focusing on features, not on the beauty of the user interface. A professional designer will provide you with specific icons for biometric authentication features, which you can use instead of a regular button.

For the **MainPage.xaml** file, the code is the following:

```
<?xml version="1.0" encoding="utf-8" ?>
```

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="BiometricAuthentication.MainPage">

    <StackLayout>
        <Button Text="Register" x:Name="RegisterButton"
                Clicked="RegisterButton_Clicked"/>
        <Button Text="Login" x:Name="LoginButton"
                Clicked="LoginButton_Clicked"/>
    </StackLayout>
</ContentPage>

```

The event handlers for the two buttons are extremely simple:

```

private async void RegisterButton_Clicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new RegisterPage());
}

private async void LoginButton_Clicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new LoginPage());
}

```

Then, inside the constructor of the **App** class, replace the assignment of the **MainPage** property with a **NavigationPage**, as follows:

```

MainPage = new NavigationPage(new MainPage());

```

In the **RegisterPage.xaml** file, replace the content of the **VerticalStackLayout** with the following:

```

<Label Text="Set your password:" Margin="10"/>
<Entry IsPassword="True" x:Name="PasswordEntry" Margin="10"/>
<Button Text="Save password" x:Name="SavePasswordButton"
        Clicked="SavePasswordButton_Clicked" Margin="10"/>

```

In the **LoginPage.xaml** file, replace the content of the **VerticalStackLayout** with the following:

```

<Label Text="Enter your password:" Margin="10"/>
<Entry x:Name="PasswordEntry" IsPassword="True" Margin="10"/>
<Button x:Name="PasswordLoginButton"
        Text="Login with password"
        Clicked="PasswordLoginButton_Clicked" Margin="10"/>
<Button x:Name="BiometricLoginButton"
        Text="Login with biometric authentication"
        Clicked="BiometricLoginButton_Clicked" Margin="10"/>

```

There is really nothing complex in the XAML for both pages, and you are totally free to rearrange their position and layout inside the page. The C# event handlers for both pages are discussed in the next sections, as part of the implementation logic.

Enforcing requirements for biometric authentication

As I mentioned previously, biometric authentication is useful for understanding whether the device recognizes the user but does not guarantee that the user is also registered or authenticated on an application. Therefore, it's a good idea to enforce the requirements for the user to be able to use biometric authentication. To accomplish this, when the user registers with an app, the credentials can be stored inside the device's secure storage. As you learned in Chapter 2, the secure storage provides a high level of encryption and security, but nothing prevents you from encrypting the password with an algorithm and storing the resulting string in the secure storage (which is not done in this chapter).



Note: *Because the logic relies on the secure storage, if you work with an iOS simulator, remember to set up the Keychain entitlement, as you learned in Chapter 2.*

You might argue that saving a password in the local secure storage is not a good idea. Most of the apps that require authentication and that also offer biometric authentication options do this, and the reason is simple: if the user does not enter credentials manually, but these are required by API calls or web services, the app needs to take these from somewhere, even for the very first call.

With this in mind, let's open the `RegistrationPage.xaml.cs` file and add the following event handler for the `SavePasswordButton` object:

```
private async void SavePasswordButton_Clicked(object sender, EventArgs e)
{
    // Add more validation logic here...
    if (!string.IsNullOrEmpty>PasswordEntry.Text))
    {
        await SecureStorage.SetAsync("P", PasswordEntry.Text);
        await DisplayAlert("Success", "Password saved", "OK");
        await Navigation.PopAsync();
    }
}
```

You can perform your own validation logic on the entered password, such as length and minimum-security requirements. It is saved inside the secure storage with a key that is not very readable, and this is intentional. We trust the secure storage, but some more security is never enough, so calling the key `P` instead of `Password` can be a good idea. When done, an alert is shown, and the code navigates back to the welcome page. This was the easy part. Now, let's go into more complex logic.

Implementing biometric authentication

It is not possible to provide only biometric authentication options, and this makes sense. Older devices might not have sensors, or the user might not have configured the hardware. If the sensor gets broken, for example, or if the user wears gloves (in the case of fingerprints), they must have an option to log in manually. This example will show how to work with both manual login and biometric login.

Open the LoginPage.xaml.cs. There is some code common to both scenarios. For example, the following code detects whether a password was securely stored previously, which means the user had registered:

```
private async Task<bool> IsPasswordSetAsync()
{
    string result = await SecureStorage.GetAsync("P");
    return result != null;
}
```

This will be invoked in the next specific paragraphs.

Authentication with password

In the case of a login scenario where the users type their credentials manually, you will need to validate the entered password and then send it to a web API or web service. In the LoginPage.xaml.cs, add the following method:

```
private async Task<bool> IsLocalPasswordValidationPassing(string password)
{
    string localPassword = await SecureStorage.GetAsync("P");
    return localPassword == password;
}
```

Given the specified password, the method checks if it matches the password stored in the secure storage and returns the result of the comparison.



Note: A password, as well as regular data, should be validated on the frontend, on the backend, and in the data store. It's something that is not possible to demonstrate here but certainly keep it in mind.

This method is invoked by the event handler for the PasswordLoginButton object, whose code is the following:

```
private async void PasswordLoginButton_Clicked(object sender, EventArgs e)
{
    bool isPasswordSet = await IsPasswordSetAsync();
    if (!isPasswordSet)
    {
        await DisplayAlert("Error", "Password not set, register first",
```

```

        "OK");
    await Navigation.PopAsync();
    return;
}

if (!string.IsNullOrEmpty(PasswordEntry.Text))
{
    bool localValidation =
        await IsLocalPasswordValidationPassing(PasswordEntry.Text);
    if (localValidation)
    {
        await DisplayAlert("Success", "Authenticated!", "OK");
        // Do login here...
    }
}
}

```

If the password is not set, as per the **IsPasswordSetAsync** method explained in the previous section, it means the user is not registered, so the code returns to the main page. Otherwise, if a formal validation of the password passes, **IsLocalPasswordValidationPassing** checks if the entered password matches the one stored in the secure storage. If they match, the password is accepted and sent to a real login service.

Authentication with fingerprint and face ID

For biometric authentication, the **CrossFingerprint** singleton class, through its **Current** property, provides methods and types that make it very easy to work with the biometric hardware. The first thing to do is check if this is available on the device. Availability depends on several factors, such as hardware sensors available on the device, sensors enabled and configured, and API support. This is accomplished via a method called **GetAvailabilityAsync**, which returns an object of type **FingerprintAvailability**.

The following method uses both to detect when biometric authentication is available:

```

// Requires a using Plugin.Fingerprint.Abstractions; directive
private async Task<bool> CheckIfBiometricAuthIsAvailableAsync()
{
    FingerprintAvailability isBiometricAuthAvailable =
        await CrossFingerprint.Current.GetAvailabilityAsync();
    return isBiometricAuthAvailable == FingerprintAvailability.Available;
}

```



Tip: The *FingerprintAvailability* enumeration also allows you to understand if face ID is available, not just fingerprint. Do not get confused by its name.

GetAvailabilityAsync checks if the operating system's API supports biometric authentication, if the user has given permissions, if the device has sensors, and finally, if the user has enabled and configured the sensor. Table 4 lists values from the **FingerprintAvailability** enumeration.

Table 4: *FingerprintAvailability* enumeration

FingerprintAvailability enumeration	
Available	Biometric authentication can be used.
NoImplementation	The plugin has no implementation for the current platform.
NoApi	The operating system's API does not support biometric authentication.
NoPermission	The app is not allowed to access biometric hardware.
NoSensor	The device has no biometric hardware.
NoFingerprint	Biometric authentication has not been set up.
NoFallback	The fallback user interface has not been set up.
Unknown	An error occurred and could not be mapped to any of the other values.
Denied	The user has denied the usage of the biometric authentication.

The next step is writing the code for the **BiometricLoginButton** object, which is the following:

```
private async void BiometricLoginButton_Clicked(object sender, EventArgs e)
{
    bool isPasswordSet = await IsPasswordSetAsync();
    if (!isPasswordSet)
    {
        await DisplayAlert("Error", "Password not set, register first",
            "OK");
        await Navigation.PopAsync();
        return;
    }

    bool biometricAuthAvailability = await
        CheckIfBiometricAuthIsAvailableAsync();
}
```

```

    if (!biometricAuthAvailability)
    {
        await DisplayAlert("Error",
            "Biometric authentication is not available.", "OK");
        return;
    }

    await BiometricAuthenticationAsync();
}

```

Like for the manual login button, first a check is made to see whether the user is registered; if not, the app goes back to the main page. If the user is registered, the code checks if biometric authentication is available by invoking the **CheckIfBiometricAuthIsAvailableAsync** method explained previously. If it's available, a new method called **BiometricAuthenticationAsync** is invoked. The following is the code for this method:

```

private async Task BiometricAuthenticationAsync()
{
    var authRequest = new AuthenticationRequestConfiguration
        ("Biometric authentication", "Login with fingerprint or face ID");
    FingerprintAuthenticationResult result =
        await CrossFingerprint.Current.AuthenticateAsync(authRequest);
    if (result.Authenticated)
    {
        await DisplayAlert("Success", "Authenticated!", "OK");
        // Do login here...
    }
    else
        await DisplayAlert("Error", $"Reason: {result.ErrorMessage}", "OK");
}

```

The **CrossFingerprint.AuthenticateAsync** method is invoked to prompt the user with the system user interface for biometric authentication. Depending on the hardware installed on your device, the API will know which sensor to invoke. This method takes an object of type **AuthenticationRequestConfiguration**, whose constructor needs you to specify the title and text to be displayed in the user interface, where supported. This object allows for further authentication configuration, but most of the time you will not need to do this.

AuthenticateAsync returns the result of the biometric authentication with an object of type **FingerprintAuthenticationResult**. The **Authenticated** property, of type **bool**, is true if the user was recognized. If not, you can get a description of the error through the **ErrorMessage** property, of type **string**, and the reason behind the error via a property called **Status**, of type **FingerprintAuthenticationResultStatus**. Values might be **Failed**, **Canceled**, **TooManyAttempts**, or **Denied**. **Status** is set with **Succeeded** when **Authenticated** is true.

Detecting the available hardware

If you need to understand what kind of hardware is supported on the device, you can invoke the `GetAuthenticationTypeAsync` method as follows:

```
AuthenticationType authTypes =  
    await CrossFingerprint.Current.GetAuthenticationTypeAsync();
```

It returns an object of type `AuthenticationType`, an enumeration whose values can be `None`, `Fingerprint`, and `Face`.

Running the sample app

After much work, you can run the sample app and see how it looks. First of all, you will need to set and save a password in the registration page. Otherwise, an error message will be displayed. Next, if you try to log in with biometric authentication, you will see the operating user interface for this feature. Figure 10 shows an example based on fingerprint on an Android device.

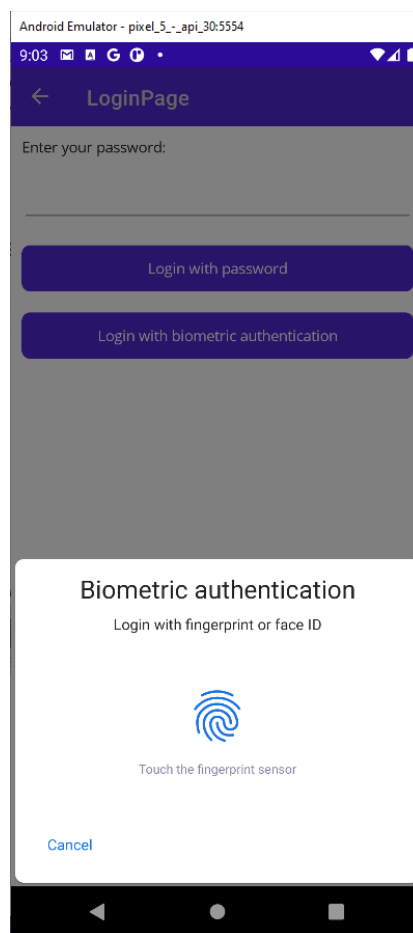


Figure 10: Biometric authentication on Android

Remember that the biometric authentication user interface can vary depending on your operating system version and system theme, especially on Android.

Chapter summary

Biometric authentication improves the user experience by leveraging the device hardware and avoiding the need to type user credentials manually every time. With the `Plugin.Fingerprint` library, it's easy to implement biometric authentication using the **CrossFingerprint** class and its methods, such as **GetAvailabilityAsync** and **AuthenticateAsync**. The complexity of the logic depends on your specific scenario, but now you know how to cover a topic that .NET MAUI does not natively.

The secure storage, widely used in this chapter, is not the only secure place where you can store passwords and data in general. In the next chapter, you will learn how to work with secured SQLite local databases.

Chapter 4 Local Data Access with SQLite

More often than not, applications work with data, and mobile apps are no exception. In many cases, mobile apps exchange data over networks and take advantage of cloud storage and services such as push notifications. However, there are situations in which mobile apps only need to store data locally. For app settings, you can use the techniques described in Chapter 2. In the case of complex, structured data, applications need a different way to store information.

The good news is that you can easily include local databases in your mobile app using [SQLite](#). This is an open-source, lightweight, serverless database engine that makes it simple to create local databases and perform operations on data. Information is stored inside tables, and data operations can be performed writing C# code and LINQ queries. SQLite is perfectly suited for cross-platform development because it's a portable database engine that's preinstalled on both iOS and Android, and it can be easily deployed to Windows.

In this chapter, I will describe how to implement local data access with SQLite and a step further: securing the database with a password.



Note: *The code example provided in this chapter is based on the Model-View-ViewModel pattern. If you are not familiar with it, you can read an introduction from the [.NET MAUI Succinctly](#) ebook. This is important advice, because I will assume you already know some concepts.*

Choosing and installing the SQLite libraries

SQLite libraries targeting C# and .NET for your apps are available through NuGet. There are several editions, each targeting different platforms. For .NET MAUI, you have a couple of options:

- **sqlite-net-pcl** is a cross-platform, [open-source](#) library that enables creating databases, reading data, and writing data. Don't be confused by the pcl (portable class library) literal; the library works with .NET MAUI and .NET standard projects, as well.
- **sqlite-net-sqlcipher**, also known as SQLCipher, works like sqlite-net-pcl, but also adds the possibility to secure the database with a password assigned in the connection string.

Securing the local database is a very good idea, especially if you store sensitive data, or simply because your company might want to add an additional level of security. However, the sample project in this chapter is based on the regular sqlite-net-pcl so that every reader can follow along with the code, regardless of company requirements.



Note: *[SQLCipher](#) used to be a paid library, and has been recently open-sourced. A paid version still exists, however, and the subscription offers technical support, custom builds, and additional options for enterprises.*

Now, assuming you have created a new .NET MAUI project and removed all the unnecessary code for the MainPage.xaml and MainPage.xaml.cs pages as we discussed previously, it is time to install the appropriate NuGet packages. Whether your choice is `sqlite-net-pcl` or `sqlite-net-sqlcipher`, you also need to install the `SQLitePCLRaw.bundle_green` package. This is necessary to expose the SQLite capabilities to all platforms. The sample project will allow you to show and edit a list of customers retrieved from and saved to a SQLite local database.

Installing the SQLite NuGet packages

You can install the following packages from the NuGet Package Manager:

- `sqlite-pcl-net`
- `SQLitePCLRaw.bundle_green` (minimum version is 2.0.4)

However, many developers (including myself) have experienced the following runtime error when a connection to the SQLite database was being established:

The type initializer for 'SQLite.SQLiteConnection' threw an exception

For some unclear reasons, this might depend on how NuGet packages are installed. If you experience this exception, you can follow these steps:

1. Uninstall both libraries.
2. Install the `sqlite-pcl-net` package via the Package Manager Console with the following command:

> Install-Package sqlite-net-pcl -Version 1.8.116
3. The previous command will also automatically install version 2.0.4 of the `SQLitePCLRaw.bundle_green` package, but you might want to update this to the latest version as follows:

> Install-Package SQLitePCLRaw.bundle_green
4. Save the solution and close Visual Studio.
5. Locate the project folder and delete the **bin** and **obj** subfolders.
6. Re-open Visual Studio and the solution.

In this way, you should no longer experience the exception previously mentioned.

Setting up the database connection

Your code will access an SQLite database through a connection string, exactly like it would do with any other database. So, the connection string is the first thing you need to build. Because an SQLite database is a file that resides in a local folder, constructing the connection string requires the database pathname. Unlike in Xamarin.Forms, where you had to do this in the platform projects, in .NET MAUI you can write the code for the connection string just once.

To accomplish this, add a new class called **DatabaseConnection**. Code Listing 7 shows the first part of the code for this new class.

Code Listing 7

```
using SQLite;

namespace LocalDataAccess
{
    public class DatabaseConnection
    {
        public const string DatabaseFilename = "Customers.db3";

        public const SQLiteOpenFlags Flags =
            // Open the database in read/write mode.
            SQLiteOpenFlags.ReadWrite |
            // Create the database if it doesn't exist.
            SQLiteOpenFlags.Create |
            // Enable multi-threaded database access.
            SQLiteOpenFlags.SharedCache;

        public static string DatabasePath
        {
            get
            {
                return Path.Combine(FileSystem.AppDataDirectory,
                    DatabaseFilename);
            }
        }

        SQLiteAsyncConnection Database;

        public DatabaseConnection()
        {
        }

        async Task Init()
        {
            if (Database is not null)
                return;
        }
    }
}
```

```

        Database = new SQLiteAsyncConnection(DatabasePath, Flags);
        var result = await Database.CreateTableAsync<Customer>();
    }
}
}

```

The **DatabaseFilename** constant contains the filename for the database. By convention, SQLite database files have .db3 extensions. The **Flags** constant, of type **SQLiteOpenFlags**, combines values from this enumeration in order to establish which operations are allowed over a database (see the comments in the code).

The **DatabasePath** read-only property returns the database pathname. This is made of the app directory and the database filename. In .NET MAUI, you can retrieve the app directory via the **FileSystem.AppDataDirectory** property without dealing with platform-specific code.

The **Database** field, of type **SQLiteAsyncConnection**, represents the actual connection string. It is assigned inside the **Init** method, which will be invoked by data access methods that will be implemented shortly. This approach is called *lazy initialization*, because access to the database is only initialized when required. The method creates a new connection, passing the pathname of the database and the flags to the constructor of the **SQLiteAsyncConnection** class. If the database does not exist, it is created.

Finally, the **CreateTableAsync** method exposed by the **SQLiteAsyncConnection** class creates a table in the database if it does not exist; otherwise, it is simply ignored. This is a generic method, and the type parameter is the data object that you use to represent your entities. This is not implemented yet, but it will be shortly.

Creating a data model

Let's imagine the app provides a piece of user interface to manage a list of your customers. First of all, you need a **Customer** class to model your data. Code Listing 8 demonstrates this in a way that satisfies the needs of SQLite.

Code Listing 8

```

using SQLite;
using System.ComponentModel;
namespace LocalDataAccess
{
    [Table("Customers")]
    public class Customer : INotifyPropertyChanged
    {
        private int _id;
        [PrimaryKey, AutoIncrement]

```



```

public int Id
{
    get
    {
        return _id;
    }
    set
    {
        this._id = value;
        OnPropertyChanged(nameof(Id));
    }
}
private string _companyName;
[NotNull]
public string CompanyName
{
    get
    {
        return _companyName;
    }
    set
    {
        this._companyName = value;
        OnPropertyChanged(nameof(CompanyName));
    }
}
private string _physicalAddress;
[MaxLength(50)]
public string PhysicalAddress
{
    get
    {
        return _physicalAddress;
    }
    set
    {
        this._physicalAddress = value;
        OnPropertyChanged(nameof(PhysicalAddress));
    }
}
private string _country;
public string Country
{
    get

```

```

        {
            return _country;
        }
        set
        {
            _country = value;
            OnPropertyChanged(nameof(Country));
        }
    }
    public event PropertyChangedEventHandler PropertyChanged;
    private void OnPropertyChanged(string propertyName)
    {
        this.PropertyChanged?.Invoke(this,
            new PropertyChangedEventArgs(propertyName));
    }
}

```

The first thing you notice is the **Table** attribute at the class level, which maps the class to a table name. Because the SQLite libraries act as an ORM, the logic is that if the library maps the class's properties to table columns in the database. So, at property levels, you will notice annotations such as **Autoincrement** and **PrimaryKey** for the **Id** property (which maps the same-named column) and other annotations that apply constraints such as **NotNull** and **MaxLength**. The implementation of the **INotifyPropertyChanged** interface is useful to notify the user interface of changes on the data, but it will be ignored by the SQLite library so this interface implementation and the data annotation can coexist.

Now that you have a data model, you need to implement methods that execute operations against data.

Reading, filtering, and writing data

The SQLite and SQLCipher libraries offer all the methods you need to read and write data, and you can use LINQ to filter your query results. In this section, you learn how to query and write data to the local database via the appropriate methods from the **SQLiteAsyncConnection** class. Open the **DatabaseConnection** class and add here all the code that you find in this section.

Reading data

Suppose you want to read the full list of records from the **Customers** table, and that you want to retrieve an individual record. The first task is accomplished via the following, simple method:

```
public async Task<List<Customer>> GetCustomersAsync()
{
    await Init();
    return await Database.Table<Customer>().ToListAsync();
}
```

The **Table<T>** method returns a **AsyncTableQuery<T>** object, which is a collection that implements the **IEnumerable** interface. However, an explicit conversion via **ToListAsync** is done to ensure maximum compatibility outside of the SQLite environment. Notice how the **Init** method is invoked first, to ensure that a connection to the database is established.

Retrieving an individual record is still simple, and it can be accomplished with the following code:

```
public async Task<Customer> GetCustomerAsync(int id)
{
    await Init();
    return await Database.Table<Customer>().
        Where(i => i.Id == id).FirstOrDefaultAsync();
}
```

Here you get a first example of data filtering with LINQ. In fact, the **FirstOrDefaultAsync** method is invoked on the collection to retrieve the first record that matches the supplied **id**, with filtering that happens via a lambda expression.

More complex LINQ queries can be performed over the result of **Table<T>** to filter the list based on a given criterion. The following method demonstrates how to filter the list of customers based on their country of residence:

```
public async Task<List<Customer>> GetFilteredCustomersAsync(
    string countryName)
{
    var query = from cust in Database.Table<Customer>()
                where cust.Country == countryName
                select cust;
    return await query.ToListAsync();
}
```

It is worth mentioning that SQLite also supports the SQL language directly. For example, you could rewrite the method body as follows:

```
await Database.QueryAsync<Customer>($"SELECT * FROM Item WHERE Country =
    '{countryName}'");
```

Writing data

SQLite supports both insert and update operations. You can easily detect if an object instance must be inserted or updated if its **Id** is equal to zero, like in the following method that saves an individual **Customer** instance:

```

public async Task<int> SaveCustomerAsync(Customer item)
{
    await Init();
    if (item.Id != 0)
        return await Database.UpdateAsync(item);
    else
        return await Database.InsertAsync(item);
}

```

The code is very simple: if the value for the **Id** property of the **Customer** instance is zero, it means that it has never been persisted to the database, so **InsertAsync** is invoked. Otherwise, the record just needs to be updated, and therefore, **UpdateAsync** is called.

The **SQLiteAsyncConnection** class also exposes the **InsertAllAsync** and **UpdateAllAsync** methods, which allow for inserting and updating all the objects inside an **IEnumerable<T>** collection. Though this is very convenient, the problem happens when you have a mixed list of new and updated objects. The solution is then represented with the following method, whose purpose is saving an entire **IEnumerable<Customer>** collection:

```

public async Task SaveAllCustomersAsync(IEnumerable<Customer> items)
{
    await Init();
    foreach(var item in items)
    {
        if (item.Id != 0)
            await Database.UpdateAsync(item);
        else
            await Database.InsertAsync(item);
    }
}

```

Basically, you apply the same logic of the previous method inside a **foreach** loop.

Deleting data

Deleting data is also an easy task, keeping in mind it's a one-way operation that cannot be reverted. The following code deletes an individual record from the table:

```

public async Task<int> DeleteCustomerAsync(Customer item)
{
    await Init();
    return await Database.DeleteAsync(item);
}

```

The **DeleteAsync** method requires the ID of the record you want to delete. Another method called **DeleteAllAsync** allows for deleting all the objects in a table. You can also completely delete an entire table via the **DropTableAsync** method. You will need to invoke the **CreateTableAsync** method shown at the beginning to create the table again.

If your app handles very sensitive data, I would recommend you add a Boolean column to the table called **IsDeleted**, or similar, which you set as **true** if you want to mark the record as deleted, and then you update the record instead of physically deleting it. This allows you to keep a history of the data, even if some records will not be used anymore.



Note: There is more in the *SQLite database engine*; for example, it supports *transactions, indexes, and backups*, and the *SQLiteAsyncConnection class* also exposes the *TableChanged event* that is raised when a table is changed. For further details, have a look at the [GitHub repository](#) for the library.

Implementing Model-View-ViewModel logic

A good approach when working with databases is implementing separation between data models, data access, business logic, and the user interface. The Model-View-ViewModel pattern is perfect for accomplishing this. In the previous section, you wrote a data model and a class for data access—now it's time to write a ViewModel.

In summary, the ViewModel needs to expose the following objects that will be data-bound to the user interface:

- A collection of **Customer** objects.
- An individual **Customer** object (useful to represent the selection in a list).
- **Command** objects that invoke methods in the **DataAccess** class and execute operations against data.

Code Listing 9 shows the full code for the ViewModel, and comments on the relevant points will follow.

Code Listing 9

```
using System.Collections.ObjectModel;
using System.ComponentModel;

namespace LocalDataAccess
{
    public class CustomerViewModel : INotifyPropertyChanged
    {
        private ObservableCollection<Customer> _customers;
        public ObservableCollection<Customer> Customers
        {
            get
            {
                return _customers;
            }
            set
            {
                _customers = value;
                OnPropertyChanged(nameof(Customers));
            }
        }
    }
}
```

```

    }
}

private Customer _selectedCustomer;
public Customer SelectedCustomer
{
    get
    {
        return _selectedCustomer;
    }
    set
    {
        _selectedCustomer = value;
        OnPropertyChanged(nameof(SelectedCustomer));
    }
}

private DatabaseConnection dataAccess;

public CustomerViewModel()
{
    dataAccess = new DatabaseConnection();
    Customers = new ObservableCollection<Customer>();

    Task.Run(async () =>
    {
        var customers = await dataAccess.
            GetCustomersAsync();
        if (customers.Any())
        {
            Customers =
                new ObservableCollection<Customer>(customers);
        }
        else
            await AddCustomerAsync();
    });
}

private async Task AddCustomerAsync()
{
    var newCustomer = new Customer
    {
        CompanyName = "Company name...",
        PhysicalAddress = "Address...",
        Country = "Country..."
    };

    Customers.Add(newCustomer);
}

```

```

        await dataAccess.SaveCustomerAsync(newCustomer);
    }

    public Command SaveAllCommand
    {
        get
        {
            return new Command(async () =>
                await dataAccess.
                    SaveAllCustomersAsync(Customers));
        }
    }

    public Command AddNewCommand
    {
        get
        {
            return new Command(async () =>
                await AddCustomerAsync());
        }
    }

    public Command DeleteCommand
    {
        get
        {
            return new Command(async () =>
            {
                if (SelectedCustomer != null)
                {
                    Customers.Remove(SelectedCustomer);
                    await dataAccess.
                        DeleteCustomerAsync(SelectedCustomer);
                }
            }));
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    private void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this,
            new PropertyChangedEventArgs(propertyName));
    }
}

```

Here are some relevant points to note about Code Listing 9:

- The implementation of the **INotifyPropertyChanged** interface allows for notifying the bound views in the user interface so that they can automatically reflect changes.
- The constructor loads existing data and adds a new customer if no data is found. Notice that the **Task.Run** method is used to invoke an asynchronous method from synchronous code (as the constructor is).
- The **AddCustomerAsync** method adds a new **Customer** object to the **Customers** collection and inserts the new object into the database at the same time. This is also to demonstrate how updating an existing **Customer** works.
- **Command** objects, which will be bound to buttons in the user interface, call methods in the **DatabaseConnection** class, according to the MVVM principles.

The final step is implementing a user interface. This is going to be very simple, but with all the pieces required to work with the database.

Creating the user interface with data binding

The user interface for the sample app is defined inside the **MainPage.xaml** file. A good choice for displaying the list of customers is the **CollectionView** control. It has a simple **DataTemplate** where a **Label** displays the value of the **Customer.Id** property, so that it cannot be edited, and where **Entry** views allow for editing the values of the other properties. Code Listing 10 shows the full XAML code.

Code Listing 10

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="LocalDataAccess.MainPage">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition Height="40"/>
        </Grid.RowDefinitions>

        <CollectionView x:Name="CustomersView"
                        SelectedItem="{Binding SelectedCustomer,
                                                Mode=TwoWay}"
                        ItemsSource="{Binding Customers}"
                        SelectionMode="Single">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <StackLayout Orientation="Vertical"
                                Margin="0,10,0,0">
                        <Label Text="{Binding Id}"
                             FontSize="Medium"/>
                        <Entry Text="{Binding CompanyName}"
```



```

        FontSize="Medium" />
        <Entry Text="{Binding PhysicalAddress}"
        FontSize="Medium"/>
        <Entry Text="{Binding Country}"
        FontSize="Medium"/>
    </StackLayout>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

<StackLayout Orientation="Horizontal"
    Spacing="5" Grid.Row="1">
    <Button Text="Add"
        Command="{Binding AddNewCommand}"
        WidthRequest="90"/>
    <Button Text="Remove"
        Command="{Binding DeleteCommand}"
        WidthRequest="90"/>
    <Button Text="Save all"
        Command="{Binding SaveAllCommand}"
        WidthRequest="90"/>
</StackLayout>
</Grid>
</ContentPage>

```

For the **CollectionView**, the **SelectedItem** property is bound to the **SelectedCustomer** property of the ViewModel; **ItemsSource** is bound to the **Customers** property, representing the full list of records coming from the database; and **SelectionMode** is **Single** to allow for selection of one object. The **Label** and the **Entry** views in the data template are bound to related properties in each instance of the **Customer** object. Finally, there are three buttons, each bound to a command in the ViewModel, that allow for performing operations against the data with an approach based on layers separation. The final step is instantiating the ViewModel and using this as the page's data context. Code Listing 11 shows the C# code that you will add to the code-behind file for the page.

Code Listing 11

```

namespace LocalDataAccess;

public partial class MainPage : ContentPage
{
    private CustomerViewModel ViewModel { get; set; }

    public MainPage()
    {
        InitializeComponent();
        ViewModel = new CustomerViewModel();
    }
}

```

```

        BindingContext = ViewModel;
    }
}

```

Now you can run the sample project and see the result of the work done so far.

Running the example

When you run the app for the first time, there will be no records in the database, so the code will add a new **Customer** instance, which is added to the **Customers** collection and automatically saved to the Customers table in the SQLite database. You can edit the new instance and add new instances, as demonstrated in Figure 11.

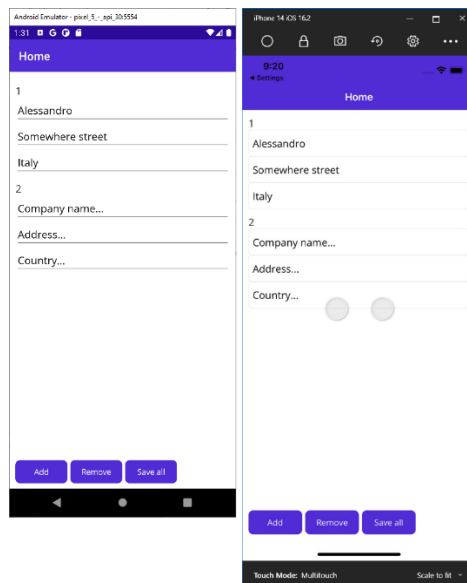


Figure 11: The sample app running and showing data from the database

Based on the code, when a new instance is added, it is automatically inserted into the database—but if you make changes to the values, you will need to save the data manually. This will perform an update operation.

Chapter summary

SQLite is an open-source, serverless database that applications written with .NET MAUI can leverage to store complex data locally within tables. In this chapter, you have seen how to create tables and read/write data, and how to display them in the user interface with data-binding.

The Model-View-ViewModel pattern has been used for true separation between data, data access, business logic, and the user interface; this is the approach you should also use in the real world for better code maintenance. There are certainly points for improvement, which depend on the needs of you and your company, but you have seen how simple it is to implement local data access securely.

Managing data is certainly important, and data does not only mean databases. There are other sources for data in mobile apps, such as Web API and JSON, which are discussed in the next chapter.

Chapter 5 Working with Web API and JSON

It's very common for applications to exchange data with remote services, either on-premises or on the cloud. .NET MAUI supports several communication protocols and data exchange formats, but certainly the most common scenario is to work with data in a platform-independent and API-driven approach.

For instance, a web service might serve different types of applications (desktop, mobile, web clients). In an ideal world, it should not know in advance what the caller application is or with which development platform it has been written, so data exchange should happen with standardized cross-platform and cross-device formats. From the point of view of development with Microsoft technologies, this can be done by developing Web API solutions that expose data as JSON and that can be consumed by any kind of application, including cross-platform projects created with .NET MAUI.

This chapter explains how to consume Web APIs and how to exchange data with a Web API service using JSON as the standard format.

Assumptions for this chapter

The companion solution for this chapter is more complex and articulated than all the other ones in the book. In fact, it includes a complete ASP.NET Web API project and a .NET MAUI project based on the Model-View-ViewModel pattern. For this reason, I assume you will open the sample solution for Chapter 5 in Visual Studio 2022, and that you will follow the reading while looking at the source code in the IDE. I will only highlight and list the code snippets that are relevant to the topic of this chapter.

Based on this consideration, some knowledge of the Model-View-ViewModel (MVVM) pattern is also required. You can read an introduction in my book, [.NET MAUI Succinctly](#). If you wish to fully try the solution, and you want to publish the Web API project to Azure, I recommend you do the following:

1. Create an Azure account. A free [account](#) is available for 12 months (please carefully read the terms of use).
2. Read the documentation about [creating a SQL database](#) on Azure.
3. Read the documentation about [publishing an ASP.NET Core project](#) to Azure.

Once you have set this up, you are ready to walk through the sample solution and this chapter.

Understanding the sample solution

The companion example for this chapter is made of three main components, described in more detail in the next paragraphs:

- A SQL database called Books hosted on Azure. This database contains a simplified list of books.
- A Web API project, deployed to Azure, that exposes calls to read, write, update, and delete books from the database.
- A .NET MAUI project that displays the list of books and allows for adding new books to the database.

Creating the database and publishing the Web API project in Azure has been the perfect choice for the current example, since I do not own a physical web server, and I can work with a real remote environment rather than showing local debugging. Let's take a closer look at the individual components of the architecture.

Understanding the database

The companion solution for this chapter works with a database called Books, which contains a simplified list of books. For each book, the database contains the title, the author's name, and the ISBN code. You can easily recreate the database on Azure using SQL Server Management Studio or Visual Studio 2022, connecting to the Azure database engine and running the script shown in Code Listing 12. Notice how a Basic profile is selected, which allows for creating a free database, limited to 1 GB of storage.

Code Listing 12

```
CREATE DATABASE [Books]
(EDITION = 'Basic',
 SERVICE_OBJECTIVE = 'Basic',
 MAXSIZE = 1 GB)
WITH CATALOG_COLLATION = SQL_Latin1_General_CP1_CI_AS;
GO

ALTER DATABASE [Books] SET COMPATIBILITY_LEVEL = 100
GO

ALTER DATABASE [Books] SET ANSI_NULL_DEFAULT OFF
GO

ALTER DATABASE [Books] SET ANSI_NULLS OFF
GO

ALTER DATABASE [Books] SET ANSI_PADDING OFF
GO

ALTER DATABASE [Books] SET ANSI_WARNINGS OFF
GO

ALTER DATABASE [Books] SET ARITHABORT OFF
```

```
GO
```

```
ALTER DATABASE [Books] SET AUTO_SHRINK OFF  
GO
```

```
ALTER DATABASE [Books] SET AUTO_UPDATE_STATISTICS ON  
GO
```

```
ALTER DATABASE [Books] SET CURSOR_CLOSE_ON_COMMIT OFF  
GO
```

```
ALTER DATABASE [Books] SET CONCAT_NULL_YIELDS_NULL OFF  
GO
```

```
ALTER DATABASE [Books] SET NUMERIC_ROUNDABORT OFF  
GO
```

```
ALTER DATABASE [Books] SET QUOTED_IDENTIFIER OFF  
GO
```

```
ALTER DATABASE [Books] SET RECURSIVE_TRIGGERS OFF  
GO
```

```
ALTER DATABASE [Books] SET AUTO_UPDATE_STATISTICS_ASYNC OFF  
GO
```

```
ALTER DATABASE [Books] SET DATE_CORRELATION_OPTIMIZATION OFF  
GO
```

```
ALTER DATABASE [Books] SET ALLOW_SNAPSHOT_ISOLATION ON  
GO
```

```
ALTER DATABASE [Books] SET PARAMETERIZATION SIMPLE  
GO
```

```
ALTER DATABASE [Books] SET READ_COMMITTED_SNAPSHOT ON  
GO
```

```
ALTER DATABASE [Books] SET MULTI_USER  
GO
```

```
ALTER DATABASE [Books] SET QUERY_STORE = ON  
GO
```

```

ALTER DATABASE [Books] SET QUERY_STORE
  (OPERATION_MODE = READ_WRITE, CLEANUP_POLICY =
  (STALE_QUERY_THRESHOLD_DAYS = 7),
  DATA_FLUSH_INTERVAL_SECONDS = 900,
  INTERVAL_LENGTH_MINUTES = 60,
  MAX_STORAGE_SIZE_MB = 10,
  QUERY_CAPTURE_MODE = AUTO,
  SIZE_BASED_CLEANUP_MODE = AUTO)
GO

ALTER DATABASE [Books] SET READ_WRITE
GO

```

You can enter some sample data manually, or you can run the script shown in Code Listing 13 to add a couple of books to the table.

Code Listing 13

```

USE Books
GO
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Books](
  [Id] [int] IDENTITY(1,1) NOT NULL,
  [Title] [nvarchar](50) NOT NULL,
  [Author] [nvarchar](50) NOT NULL,
  [ISBN] [nvarchar](50) NULL,
  CONSTRAINT [PK_Books] PRIMARY KEY CLUSTERED
(
  [Id] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
GO
SET IDENTITY_INSERT [dbo].[Books] ON
GO
INSERT [dbo].[Books] ([Id], [Title], [Author], [ISBN])
VALUES (1, N'Xamarin.Forms Succinctly',
N'Alessandro Del Sole', N'978-1-64200-175-4')

```

```
GO
INSERT [dbo].[Books] ([Id], [Title], [Author], [ISBN])
VALUES (2, N'Visual Studio 2019 for Mac Succinctly',
N'Alessandro Del Sole', N'978-1-64200-177-8')
GO
SET IDENTITY_INSERT [dbo].[Books] OFF
GO
```

You could improve the table by adding the publication date or a Boolean column to mark a record as logically deleted, without actually removing the record from the database, but this is left to you as an exercise.

Understanding the Web API project

The Web API project is responsible for working against the database using Entity Framework, and it returns JSON responses depending on the request. For example, it will return a JSON array when a **GET** call is made to retrieve the list of books. The project has been created following this very detailed [article](#) published by Syncfusion®, and I recommend you do the same if you want to recreate the sample project on your own. One important thing to note is about securing the API, which I will also recall in the next paragraphs, when appropriate.

The article explains how to implement authorization and authentication based on JSON Web Tokens (JWT) and is a way to include an authorization code inside the API calls. Authorization and authentication implementations can be very different between companies' policies, customer requirements, and app architecture. For example, your requirements might be implementing custom authentication with a username and password; or you might need an Active Directory authentication system if your application works within a domain; or you might work with bearer tokens. For this reason, the sample Web API service does not require any authentication.

In this way, you will be able to implement your own system based on your requirements. I will point out, where appropriate, how to add tokens and user credentials inside the API calls. In addition, the companion project also implements the **TokenController** class discussed in the Syncfusion® [article](#) for your convenience, so it will be very easy for you to implement the JWT tokens discussed in the article itself. The next paragraphs highlight the most relevant parts of the project.

Project setup

The BooksApi project that you find under the Chapter5 folder of the companion code is a regular ASP.NET Core Web API project created with Visual Studio 2022 and targeting .NET 7. It works with the following NuGet packages:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer

These are required to work against the remote database. The appsettings.json file contains the connection string. In the sample code, you can find a placeholder that needs to be replaced with the actual connection string, identified by the **DefaultConnection** variable as follows.

```
"ConnectionStrings": {  
  "DefaultConnection": "YOUR_CONNECTION_STRING_GOES_HERE"  
},
```

For SQL Azure databases, it can be retrieved in the Azure portal. The connection string must be added to the application services, so the following lines must be added in the Program.cs file right after the declaration of the **builder** variable:

```
// requires a using Microsoft.EntityFrameworkCore; directive  
var connection = builder.Configuration.  
    GetConnectionString("DefaultConnection");  
builder.Services.AddDbContextPool<BooksContext>(options =>  
    options.UseSqlServer(connection));
```



Tip: The sample project is based on .NET 7, and therefore, it is using the so-called implicit statements feature introduced with .NET 5. In the Program.cs file, you do not see the usual namespace and class definitions, which are inferred by the compiler.

Another relevant part is the BooksController.cs file, whose implementation is discussed along with the data model in the next paragraphs.

Data model implementation

The data model consists of a **Book** class that maps the **Books** table in the database, and looks like the following:

```
public partial class Book  
{  
    public int Id { get; set; }  
    public string Title { get; set; }  
    public string Author { get; set; }  
    public string Isbn { get; set; }  
}
```

A specialized instance of the **DbContext** class is then needed to connect the database to the data model, as represented in Code Listing 14.

Code Listing 14

```
using Microsoft.EntityFrameworkCore;  
  
public partial class BooksContext : DbContext  
{
```

```

public BooksContext(DbContextOptions<BooksContext> options)
    : base(options)
{
}

public virtual DbSet<Book> Books { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Book>(entity =>
    {
        entity.Property(e => e.Author)
            .IsRequired()
            .HasMaxLength(50);

        entity.Property(e => e.Isn)
            .HasColumnName("ISBN")
            .HasMaxLength(50);

        entity.Property(e => e.Title)
            .IsRequired()
            .HasMaxLength(50);
    });

    OnModelCreatingPartial(modelBuilder);
}

partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}

```

As you can see, the **OnModelCreating** method performs the actual binding between the table and the data model, including some validations.



Tip: *Don't forget to replace the sample connection string in the appsettings.json file with the proper one.*

API calls implementation and testing

The API calls are implemented inside a controller class called **BooksController**. The first thing to do in the controller is implement a constructor that will receive an instance of the context class via dependency injection:

```
private readonly BooksContext _context;
```

```
public BooksController(BooksContext context)
{
    _context = context;
}
```

The next piece of code is related to retrieving the list of books from the database. There are two **GetBooks** methods: one to get the full list, and one to get an individual book based on its ID:

```
[HttpGet]
public async
    Task<ActionResult<IEnumerable<Book>>> GetBooks()
{
    return await _context.Books.ToListAsync();
}

[HttpGet("{id}")]
public async Task<ActionResult<Book>> GetBooks(int id)
{
    var books = await _context.Books.FindAsync(id);

    if (books == null)
    {
        return NotFound();
    }
    return books;
}
```

The code is executed asynchronously, and data is obtained via methods from the **DbContext** class: **ToListAsync** for the full list and **FindAsync** for the individual object. Adding a new **Book** object to the database is done via the **POST** verb and the **PostBook** method, as follows:

```
[HttpPost]
public async Task<ActionResult<Book>> PostBook(Book book)
{
    _context.Books.Add(book);
    await _context.SaveChangesAsync();

    return CreatedAtAction("GetBooks",
        new { id = book.Id }, book);
}
```

The **Book** object that will be added to the database is received by the API call in the form of a JSON object, and you will see how to package this in .NET MAUI shortly. Notice how the API call returns a **CreatedAtAction** object, which matches the 201 HTTP status code (**Created**) with the ID of the new record.

Updating an existing **Book** object can be performed via the **PutBook** method, which leverages the **PUT** verb and works as follows:

```

public async Task<IActionResult> PutBook(int id, Book book)
{
    if (id != book.Id)
    {
        return BadRequest();
    }
    _context.Entry(book).State = EntityState.Modified;
    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!BooksExists(id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return NoContent();
}
private bool BooksExists(int id)
{
    return _context.Books.Any(e => e.Id == id);
}

```

Like for adding an object, the **Book** entity instance is still received from the caller clients. One difference is that the entity state changes to **EntityState.Modified**, and the code also checks for concurrency exceptions. This method returns a **NoContent** response, which matches the 204 HTTP status code. The very last method is called **DeleteBook**, which receives only the ID for the **Book** object you want to delete, and whose code is the following:

```

[HttpDelete("{id}")]
public async Task<ActionResult<Book>> DeleteBook(int id)
{
    var books = await _context.Books.FindAsync(id);
    if (books == null)
    {
        return NotFound();
    }
    _context.Books.Remove(books);
    await _context.SaveChangesAsync();
    return books;
}

```

```
}
```

The code is very easy, and this is a standard implementation. However, if you want to store the full history of the data, I recommend you add a **BIT** column called **IsDeleted** to the database, add a same-named property of type **bool** to the **Book** class, and replace the **DbContext.Remove** invocation as follows:

```
Book.IsDeleted = true;
```

In this way, the code will not physically remove the record from the database; rather, it will mark it as logically deleted. If you go for this option, you might also want to change the **GetBooks** method body as follows:

```
return await _context.Books.Where(b=>b.IsDeleted == false).ToListAsync();
```

Once the most relevant pieces of code are clear, it's time to test the API calls. You can do this by starting the Web API project for debugging on the local machine, or you can deploy it to Azure (or to an on-premises server) for the most realistic experience. Whichever option you choose, you need a tool to make and test API calls. One of the most popular and free tools is [Postman](#). With Postman, you can make API calls with any of the HTTP verbs, and you can include all the information needed for the request through a simple user interface. Whatever your deploy option is, take note of the Web API address.

In Postman, open a new tab to make a **GET** call, and paste the URL of your Web API service and the **/api/books** suffix. When you're ready, simply click **Send**. After a few seconds, you will see the list of books in the results window, in the form of a JSON array, as shown in Figure 12.

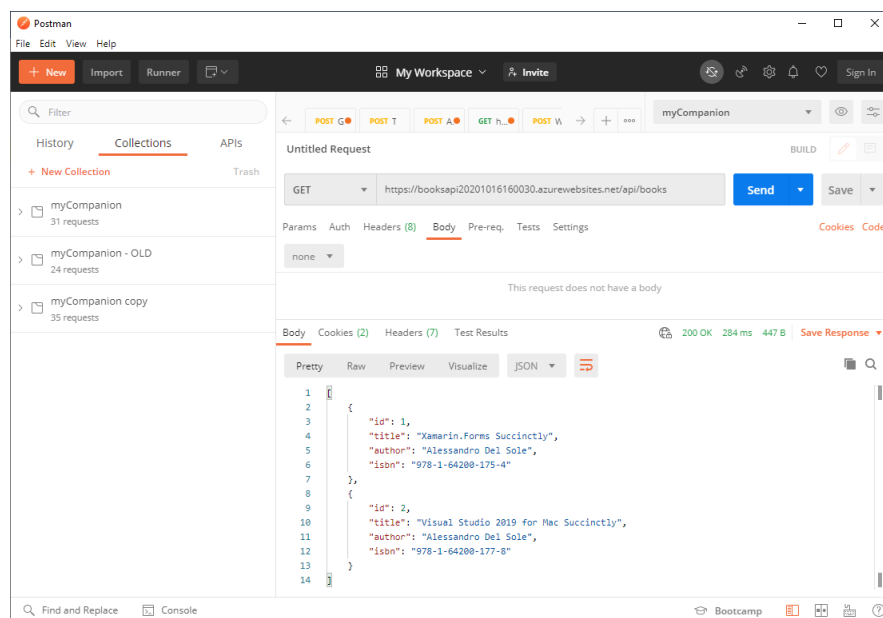


Figure 22: Getting the list of books with Postman

Adding a new book can be done by changing the HTTP verb to **POST** and specifying the data in the **Body** tab. If you select the **raw** view, you can write the JSON directly, as shown in Figure 13.

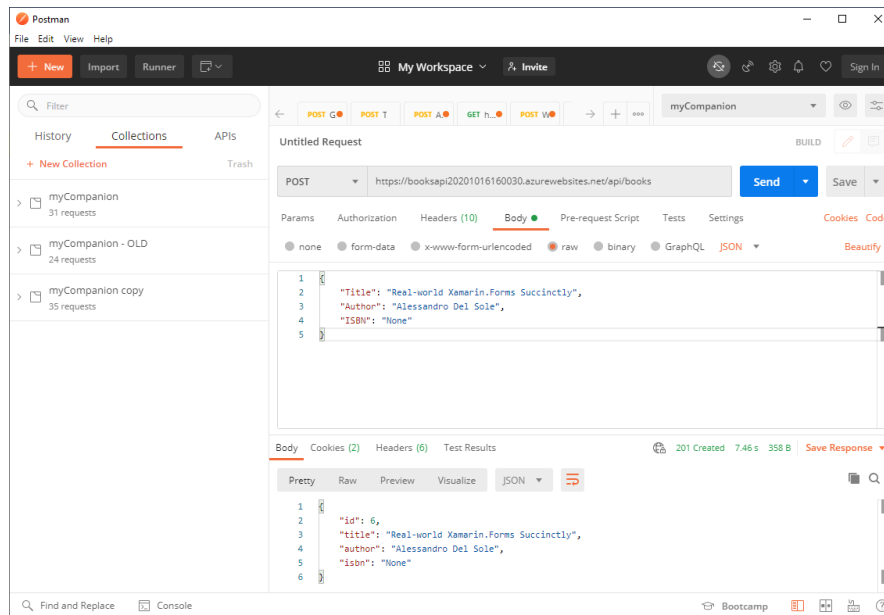


Figure 33: Adding a new object with a POST request

Another required step is opening the **Headers** tab and adding a new header tag of type **Content-Type** in the **KEY** column, with a value of **application/json**. When you click **Send**, the **PostBook** API call will be invoked, and the new object written into the database. The response contains the full properties for the newly added object, and the most important one is the new record ID.

Tip: If the Web API implements a custom authentication mechanism with a username and password, these will be included in the JSON sent to the API. If the implementation relies on a token, this will instead be included in the Authorization tab, where you will find a dropdown list of the most popular token types.

If you want to double-check, make the **GET** call again to see the list of books, and you will see that the new one is available. Updating a book will work very similarly: you will need to write the full list of **Book** properties and change the HTTP verb from **POST** to **PUT**.

The URL of the API call will also need to include the book ID in the following way: **/api/books/1**, where 1 is the record ID. Deleting a book instance is also easy. You must change the HTTP verb to **DELETE** and pass the record ID into the URL like you did for the **PUT** call. Then in the **Body** tab, you select **none**, as the request has no body. Once you have ensured the API calls work outside of any application and development environment, you can move on to understanding how to consume the API in .NET MAUI.

Understanding the .NET MAUI project

The .NET MAUI project provides the following points of interest:

- The user interface displays a list of books with a **CollectionView** and allows for loading, adding, and deleting books with specific buttons. All these views are bound to properties and commands in a ViewModel called **BookViewModel**.
- The ViewModel calls methods to work with data in a service class called **WebApiService** and send broadcast messages to the user interface, which will react by informing the user about the result of data operations.
- Data with the API is exchanged by serializing and deserializing JSON objects. This is done via the [Newtonsoft.Json](#) library, the *de facto* standard library for .NET and C#. Actually, starting with .NET Standard 2.0, Microsoft has introduced the [System.Text.Json](#) API, which provides built-in objects to work with JSON data, without the need for third-party libraries. You are totally free to use this API instead of Newtonsoft.Json, but the fact is that, in the real world, you will still find millions of projects that use the latter. If you are starting a new project, you can still make a choice, but if you are maintaining an existing project, you will probably work with Newtonsoft.Json.



Tip: Due to the popularity of Newtonsoft.Json, Microsoft has also created some [documentation](#) to help migrate to System.Text.Json.

The final result of the work is shown in Figure 14, where you can see a list of books, and in Figure 15, where you can see the insertion of a new book. You can keep both figures as a reference, since I will not list the XAML code (which you can instead follow through the companion source code).

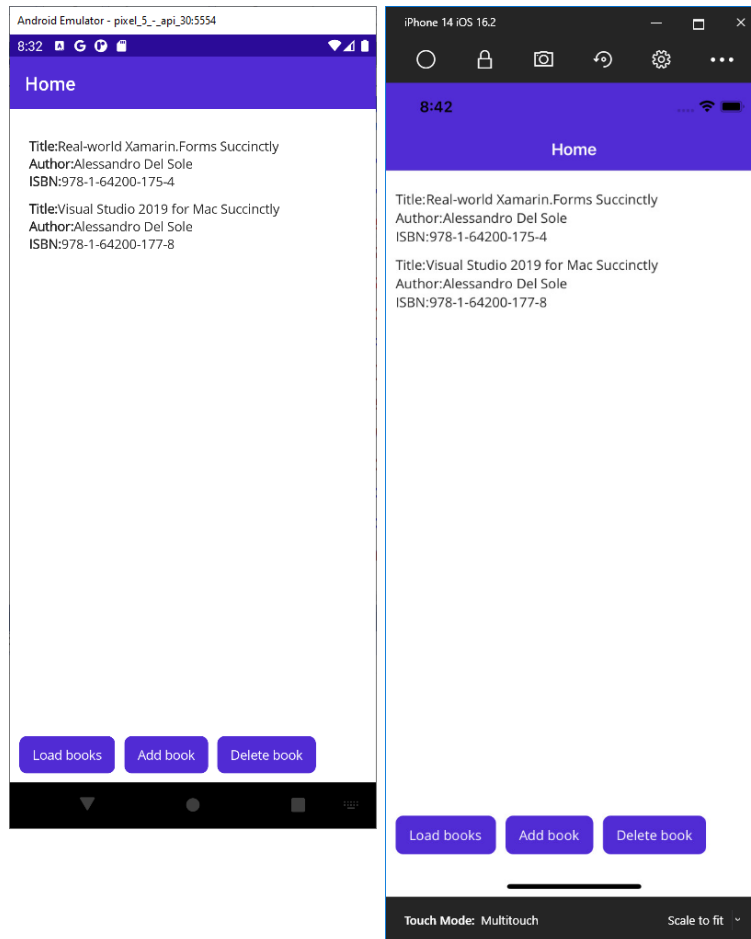


Figure 14: The sample app displays a list of books

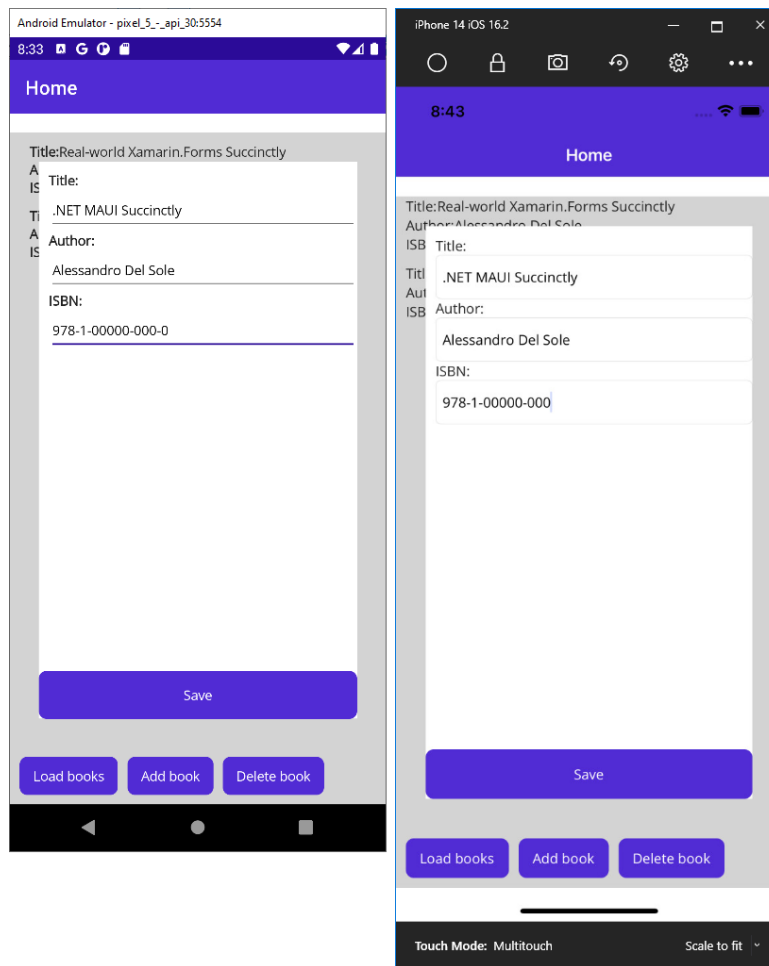


Figure 45: The sample app allows for adding and saving a new book

Calling Web APIs from .NET MAUI

Calling Web APIs from .NET MAUI is not difficult and is accomplished via the **HttpClient** class and its methods. It is good practice to group methods that work with a Web API into a separate class. In the sample project, it's called **WebApiService**, and it exposes methods to read, write, and delete data.

Another good practice is to write reusable methods, avoiding code that targets only a specific data type. With this in mind, a method called **GetDataAsync** sends a **GET** request to the specified endpoint and is implemented as follows:

```
public static async Task<HttpResponseMessage> GetDataAsync(string url,
    string id = null)
{
    HttpResponseMessage response;
    try
    {
```

```

        using (var client = new HttpClient())
        {
            if (id != null) url = $"{url}/{id}";
            response = await client.GetAsync(url);
        }
    }
    catch (HttpRequestException)
    {
        response = new
HttpResponseMessage(HttpStatusCode.ServiceUnavailable);
    }
    catch
    {
        response = new
HttpResponseMessage(HttpStatusCode.InternalServerError);
    }
    return response;
}

```

The reason for the **id** optional parameter is that, with a single method, you can obtain either the full list of objects from the specified endpoint (in our example, a list of books) or an individual object from the specific API overload. The actual API call is done by the **HttpClient.GetAsync** method, and the resulting response is returned as it is to the caller (the **BooksViewModel** class, in our case), which will be responsible for data processing. Multiple **catch** statements return a different HTTP error. This is to demonstrate how you can control the different error types in your logic that you might get from a Web API service.

As I mentioned previously, the companion source code provides all the necessary code files to implement JWT tokens to secure the API calls, but this is left to you as an exercise. API calls can be currently invoked with the so-called anonymous authentication. This is because it is not possible to demonstrate all the possible authentication solutions and at the same time to offer a working example. However, a few hints can be provided. If your Web API requires a token, you can add the following line after instantiating the **HttpClient** class:

```

client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", "Your Oauth token");

```

The constructor of the **AuthenticationHeaderValue** receives the **Bearer** literal and the authorization token and is assigned to the **Authorization** property of the request's headers. If your Web API services rely on Windows domain authentication, you can create an instance of the **HttpClient** class, specifying it needs to use the default Windows user credentials:

```

var client =
    new HttpClient(new HttpClientHandler() { UseDefaultCredentials = true });

```

The following is a method that invokes a Web API to delete data and is very similar to the previous method:

```

public static async Task<HttpResponseMessage>

```

```

        DeleteDataAsync(string url, int id)
    {
        HttpResponseMessage response;
        try
        {
            using (var client = new HttpClient())
            {
                string fullUri = $"{url}/{id}";
                response = await client.DeleteAsync(fullUri);
            }
        }
        catch (HttpRequestException)
        {
            response = new
        HttpResponseMessage(HttpStatusCode.ServiceUnavailable);
        }
        catch
        {
            response = new
        HttpResponseMessage(HttpStatusCode.InternalServerError);
        }
        return response;
    }

```

The only difference is that this method invokes **HttpClient.DeleteAsync** to send a **DELETE** request. The next method, called **WriteDataAsync**, is very interesting. Its purpose is to send a **POST** request to a Web API, and its implementation takes advantage of .NET generics, so it can be reusable:

```

public static async Task<HttpResponseMessage>
    WriteDataAsync<T>(T data, string url, string id = null)
{
    HttpResponseMessage response;
    try
    {
        using (var client = new HttpClient())
        {
            client.DefaultRequestHeaders.Accept.Clear();
            client.DefaultRequestHeaders.Accept.Add(
                new MediaTypeWithQualityHeaderValue("application/json"));

            string json = JsonConvert.SerializeObject(data);
            var content =
                new StringContent(json, Encoding.UTF8, "application/json");
            if (id != null) url = $"{url}/{id}";
            response = await client.PostAsync(url, content);
        }
    }
}

```

```

    }
    catch (HttpRequestException)
    {
        response = new
HttpResponseMessage(HttpStatusCode.ServiceUnavailable);
    }
    catch
    {
        response = new
HttpResponseMessage(HttpStatusCode.InternalServerError);
    }

    return response;
}

```

For a better understanding of the code, you can focus on two steps you manually made in Postman. The first step was adding the **application/json** content-type information to the request's headers. In the code, this is done by adding to the **DefaultRequestHeaders** collection a new object of type **MediaTypeWithQualityHeaderValue**, which receives the specified MIME type as an argument.

The second step you made in Postman was specifying the JSON markup for the new book you wanted to add to the database via API. In the code, this takes two lines. The first line serializes the object you want to write into JSON via the **JsonConvert.SerializeObject** method. The second line creates an instance of the **StringContent** class based on the serialized data, the encoding, and the MIME type. In fact, it's not enough for the API to receive the serialized object, but it is also necessary for the API to know which text encoding and data format is used. Finally, the API call is made by invoking **HttpClient.PostAsync**.

Hints about editing existing data

Existing data can be updated by sending a **HTTP PUT** request to the API via the **HttpClient.PutAsync** method. You have different alternatives:

- You can create a method that works like **WriteDataAsync**, but that invokes **PutAsync** instead.
- You can keep everything in one method, adding a bool parameter to **WriteDataAsync**, such as **IsUpdate**, and invoking **PutAsync** if **IsUpdate** is **true**, or **PostAsync** if it is **false**.
- You could handle the update scenario inside the **Post** method of the API controller (in our case, it's **PostBook**). The client could still send a **POST** request, and then the API decides how to dispatch the request, which would need the **WriteDataAsync** method to send a bool additional parameter.

I would personally choose the second option, which allows for having less code and doesn't create confusion in the API code.

Implementing broadcast messages

In the next section, you will see how to implement methods that work against data in the ViewModel. When such methods complete their tasks, they need to notify the user interface. This is accomplished via broadcast messages. In Xamarin.Forms you would have done this via the **MessagingCenter** class, but in Chapter 1, you learned that this has been deprecated in .NET MAUI when targeting .NET 7.

So, following the lesson learned in Chapter 1, first install the **CommunityToolkit.Mvvm** NuGet package, and then add the code represented in Code Listing 15.

Code Listing 15

```
using CommunityToolkit.Mvvm.Messaging.Messages;

namespace WorkingWithWebApi.Messages
{
    public class DataStatusMessage :
        ValueChangedMessage<DataStatus>
    {
        public DataStatusMessage(DataStatus value) : base(value)
        {
        }
    }

    public enum DataStatus
    {
        BookSaved,
        BookDeleted,
        BookError
    }
}
```

The **DataStatusMessage** class handles an object of type **DataStatus**. This is an enumeration that represents the possible status for the data handled by the ViewModel, for example, when the data is saved or deleted, or when there is an error.

The methods described in the next section will send messages via the **Send** method of the **WeakReferenceMessenger** class, whereas the user interface will subscribe to these messages via the **Register** method of the same class.

Deserializing JSON responses

All the methods in the **WebApiService** class return an object of type **HttpResponseMessage**. Among others, this object exposes a **StatusCode** property, of type **HttpStatusCode**, which contains the code for the result of the API call and the **Content** property, of type **HttpContent**, which contains the actual response from the API.

The responsibility for analyzing the status code and for the response content is of any **ViewModel**. In the sample app, one **ViewModel** called **BooksViewModel** is available. Here, a method called **LoadBooksAsync** is invoked by a command called **LoadBooksCommand** and gets the lists of books from the Web API as follows:

```
private async Task LoadBooksAsync()
{
    string url = "https://yourdomainname.azurewebsites.net/api/books";
    var result = await WebApiService.GetDataAsync(url);

    switch (result.StatusCode)
    {
        case HttpStatusCode.OK:
            string resultString =
                await result.Content.ReadAsStringAsync();

            var deserialized =
                JsonConvert.DeserializeObject<List<Book>>(resultString);

            Books = new ObservableCollection<Book>(deserialized);
            return;
        default:
            WeakReferenceMessenger.Default.
                Send(new DataStatusMessage(DataStatus.ServerError));
            return;
    }
}
```

As you can see, the **ViewModel** handles the status code returned by the **GetDataAsync** method, and in case of errors, sends a broadcast message that the user interface will catch to display a warning to the user (see the **MainPage.xaml.cs** file in the companion source code for the implementation). In the case of success, the response of the API call is first converted into a plain string. Then, because the code expects a list of books as the response, the **JsonConvert.DeserializeObject** converts the obtained string into a specialized **List<Book>** object.

The next line of code creates an **ObservableCollection<Book>** instance, because this type of collection is used to data-bind collections to the user interface. The behavior is similar for the **AddBookAsync** method, which invokes the **WebApiService.PostAsync** method to write a new book into the database:

```
private async Task AddBookAsync()
{
    if (NewBook != null)
    {
        string url = "https://yourdomainname.azurewebsites.net/api/books"
        var result = await WebApiService.WriteDataAsync(NewBook, url);

        switch (result.StatusCode)
        {
```

```

        case HttpStatusCode.OK:
        case HttpStatusCode.Created:
            string resultString = await result.Content.
                ReadAsStringAsync();

            var deserialized = JsonConvert.
                DeserializeObject<Book>(resultString);

            Books.Add(deserialized);
            WeakReferenceMessenger.Default.
                Send(new DataStatusMessage(DataStatus.BookSaved));
            return;
        default:
            WeakReferenceMessenger.Default.
                Send(new DataStatusMessage(DataStatus.ServerError));
            return;
    }
}

```

The behavior is similar to the **LoadBooksAsync** method. Here the code also handles the **Created** HTTP status code. In this case, **JsonConvert.DeserializeObject** returns the individual **Book** instance that has been added to the database, and this will be added to the **Books** collection of the ViewModel so that the user interface can update accordingly.

The **DeleteBookAsync** in the ViewModel is also similar:

```

private async Task DeleteBookAsync()
{
    string url =
        "https://yourdomainname.azurewebsites.net/api/books";
    var result =
        await WebApiService.DeleteDataAsync(url, SelectedBook.Id);
    switch (result.StatusCode)
    {
        case HttpStatusCode.OK:
            string resultString = await result.Content.ReadAsStringAsync();

            // Do anything you need with the deleted object...
            Book deserializedBook =
                JsonConvert.DeserializeObject<Book>(resultString);

            Books.Remove(SelectedBook);

            WeakReferenceMessenger.Default.
                Send(new DataStatusMessage(DataStatus.BookDeleted));
            return;
        default:
            WeakReferenceMessenger.Default.
                Send(new DataStatusMessage(DataStatus.ServerError));
    }
}

```

```

        return;
    }
}

```

The **WebApiService.DeleteDataAsync** method returns a **Book** object representing the deleted one, in case you need to display information to the user. The corresponding instance represented by the **SelectedBook** property is also removed from the **Books** collection.

All the work done so far needs to be bound to the user interface. This is the topic of the next section.

Implementing the user interface

Keeping Figures 14 and 15 as reference, the XAML code that defines the user interface is represented in Code Listing 16.

Code Listing 16

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="WorkingWithWebApi.MainPage" Padding="0,20,0,0">
    <Grid x:Name="LayoutRoot" BackgroundColor="White">
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition Height="60"/>
        </Grid.RowDefinitions>
        <CollectionView ItemsSource="{Binding Books}" Margin="10,0,10,0"
            SelectionMode="Single"
            SelectedItem="{Binding SelectedBook,
                Mode=TwoWay}">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <StackLayout Margin="10,10,0,0">
                        <StackLayout Orientation="Horizontal">
                            <Label Text="Title:" FontAttributes="Bold"/>
                            <Label Text="{Binding Title}" />
                        </StackLayout>

                        <StackLayout Orientation="Horizontal">
                            <Label Text="Author:" FontAttributes="Bold"/>
                            <Label Text="{Binding Author}" />
                        </StackLayout>

                        <StackLayout Orientation="Horizontal">
                            <Label Text="ISBN:" FontAttributes="Bold"/>
                            <Label Text="{Binding Isbn}" />
                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>
        </Grid>
    </ContentPage>

```



```

        </StackLayout>
    </DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

<Grid x:Name="NewBookGrid" IsVisible="False"
      BindingContext="{Binding}"
      BackgroundColor="White" Margin="30">
    <Grid.RowDefinitions>
        <RowDefinition />
        <RowDefinition Height="50"/>
    </Grid.RowDefinitions>
    <StackLayout Margin="10,10,0,0"
      BindingContext="{Binding NewBook}">
        <StackLayout>
            <Label Text="Title:" FontAttributes="Bold"/>
            <Entry Text="{Binding Title}" />
        </StackLayout>

        <StackLayout>
            <Label Text="Author:" FontAttributes="Bold"/>
            <Entry Text="{Binding Author}" />
        </StackLayout>

        <StackLayout>
            <Label Text="ISBN:" FontAttributes="Bold"/>
            <Entry Text="{Binding Isbn}" />
        </StackLayout>
    </StackLayout>
    <Button Grid.Row="1" Text="Save"
      Command="{Binding AddBookCommand}"/>
</Grid>

<StackLayout Grid.Row="1" Orientation="Horizontal">
    <Button Text="Load books" Margin="10,0,0,0"
      Command="{Binding LoadBooksCommand}" />
    <Button Text="Add book" x:Name="AddBookButton"
      Clicked="AddBookButton_Clicked" Margin="10,0,0,0"/>
    <Button Text="Delete book" x:Name="DeleteBookButton"
      Command="{Binding DeleteBookCommand}"
      Margin="10,0,0,0"/>
</StackLayout>
</Grid>
</ContentPage>

```

The following is a list of relevant points in Code Listing 16:

- The **Books** collection is the data source for the **CollectionView**. Each **Label** in the data template is bound to a property of the **Book** class.
- The second **Grid** acts like a pop-up when adding new objects, and it contains **Entry** views that are bound to properties of a new **Book** instance.
- The **Button** views at the bottom are bound to commands in the ViewModel, except for the **AddBookButton** object, which handles a **Clicked** event for the sake of simplicity.

The code-behind for the page is instead represented in Code Listing 17.

Code Listing 17

```
using CommunityToolkit.Mvvm.Messaging;
using WorkingWithWebApi.Messages;
using WorkingWithWebApi.ViewModel;

namespace WorkingWithWebApi
{
    public partial class MainPage : ContentPage
    {
        private BookViewModel ViewModel { get; set; }

        public MainPage()
        {
            InitializeComponent();
            WeakReferenceMessenger.Default.Register<DataStatusMessage>(
                this, (sender, message) =>
                {
                    ManageDataStatusChanged(message);
                });
            ViewModel = new BookViewModel();
            BindingContext = ViewModel;
        }

        private async void ManageDataStatusChanged(DataStatusMessage
            message)
        {
            var value = message.Value;
            switch(value)
            {
                case DataStatus.BookDeleted:
                    await DisplayAlert("Deleted",
                        "The specified book was deleted", "OK");
                    break;
                case DataStatus.BookSaved:
                    LayoutRoot.BackgroundColor = Colors.White;
                    NewBookGrid.IsVisible = false;
                    break;
                case DataStatus.BookError:
```

```

        await DisplayAlert("Error",
            "An error has occurred", "OK");
        LayoutRoot.BackgroundColor = Colors.White;
        NewBookGrid.IsVisible = false;
        break;
    default:
        break;
    }
}

private void AddBookButton_Clicked(object sender, EventArgs e)
{
    LayoutRoot.BackgroundColor = Colors.LightGray;
    NewBookGrid.IsVisible = true;
    ViewModel.NewBook = new Model.Book();
}
}
}

```

The following is a list of relevant points for Code Listing 17:

- The constructor of the page subscribes to messages sent by instances of the **DataStatusMessage** class and initializes an instance of the ViewModel.
- The **ManageDataStatusChanged** method is invoked when broadcast messages are intercepted and shows the appropriate user interface depending on the message.
- The **Clicked** event handler for the **AddBookButton** object shows the user interface to add a new **Book**.

When in doubt, keep Figures 14 and 15 as references, as well as the companion source code. Working with Web APIs and JSON is really easy and fast in .NET MAUI, and with a bit of knowledge of the Model-View-ViewModel pattern, you can create a very flexible and solid architecture.

Chapter summary

This chapter discussed a key topic in real-world development, which is invoking Web API from a .NET MAUI project and deserializing JSON responses, based on a SQL database.

You have seen how Web API makes it easy to work with data, taking advantage of the Entity Framework, and of the implementation of HTTP verbs (**GET**, **POST**, **PUT**, and **DELETE**). You've also seen how easy it is to invoke a Web API service from .NET MAUI via the **HttpClient** class, and how to customize the request. Finally, you have seen how to deserialize a response using the *de facto* standard library called **Newtonsoft.Json**, and how to turn the returned JSON into a .NET data model.

The communication between the mobile app and the Web API shown in this chapter relies on the HTTPS protocol, which offers high-level security. However, this might not be enough, and you might be required to take a step further by detecting certificate pinning attacks. This is covered in the next chapter.

Chapter 6 Securing Communication with Certificate Pinning

Securing communications between applications and services is extremely important, and mobile apps are no exception. Even if you use an encrypted channel based on HTTPS, you should never completely trust the identity of the target. For example, an attacker could easily discover the URL your application is pointing to and put a fake certificate in the middle of the communication between an application and the server, thus intercepting the communication. This is extremely dangerous—especially if the application handles sensitive data.

In order to avoid this, you can use a technique called certificate pinning to reduce the risk of this kind of man-in-the-middle attack dramatically. This chapter describes how to implement certificate pinning in .NET MAUI, making your mobile apps more secure.

Describing the certificate pinning implementation

You should implement certificate pinning every time you build apps that handle sensitive data and that call HTTPS URLs. Additionally, many enterprises make strict security checks before validating and distributing an app, even if for internal use only, including penetration tests. Penetration tests search for security holes in an application and simulating a man-in-the-middle attack with a fake certificate is a common test scenario. By implementing certificate pinning, you reduce the risk of certificate replacement, and this penetration test will pass.

In order to implement certificate pinning, you will need the valid certificate's public key. This can be provided by your system administrator. For demonstration or development purposes, you can also create a self-signed certificate and retrieve the public key on your own. This scenario will not be covered in this chapter, since the [documentation from Microsoft](#) already covers it thoroughly.

Certificate pinning in .NET MAUI

As you saw in Chapter 5, in .NET MAUI (and, more generally, in .NET 7) you typically use the **System.Net.Http.HttpClient** class to send requests over the network, using methods such as **GetAsync**, **PostAsync**, **PutAsync**, and **DeleteAsync**. Under the hood, **HttpClient** needs a message handler, which is responsible for managing some of the connection properties such as (but not limited to) the usage of credentials, proxy information, SSL information, and the maximum number of connections.

The **HttpClient** class has a default handler, represented by the **HttpClientHandler** object. This object and derived ones can be instructed to check what kind of security protocol is being used and to validate the certificate at every web request. For a better understanding, create a new .NET MAUI project in Visual Studio and add a new file called **EndpointConfiguration.cs**. Code Listing 18 shows the content for the new class.

Code Listing 18

```
namespace CertificatePinning
{
    public class EndpointConfiguration
    {
        // Replace with the public key of your company's certificate.
        public const string
            PUBKEY = "Y O U R   V A L I D   K E Y   G O E S   H E R E";

        // Replace with a fake key you want to use for testing.
        public const string
            PUBKEYFAKE = "Y O U R   F A K E   K E Y   G O E S   H E R E";
    }
}
```

In this class, you can store the string for two keys: a valid certificate public key, and a fake certificate key that you can use for debugging purposes. You can also consider encrypting the real public key for further security, or you might consider other options to store it, such as the secure storage.

The next step is creating an instance of the **HttpClientHandler** and assigning the following properties:

- **SslProtocol**, of type **System.Security.Authentication.SslProtocols**, which allows for specifying the security protocol used by the handler.
- **ServerCertificateCustomValidationCallback**, assigned with a callback method that is invoked at every request and that checks for certificate validity.

Now add a new class called **ServiceClient.cs** to the project and enter the code shown in Code Listing 19.

Code Listing 19

```
using System.Net;
using System.Net.Security;
using System.Security.Cryptography.X509Certificates;

namespace CertificatePinning
{
    public class ServiceClient
    {
        private HttpClient Client { get; set; }

        private HttpClientHandler clientHandler;

        public ServiceClient()
        {

```

```

        clientHandler = new HttpClientHandler();
        clientHandler.SslProtocols =
            System.Security.Authentication.
                SslProtocols.Tls12;
        clientHandler.ServerCertificateCustomValidationCallback =
            ValidateServerCertificate;
        Client = new HttpClient(clientHandler);
    }

    public async Task MakeCallAsync(string url)
    {
        HttpResponseMessage response;
        try
        {
            response = await Client.
                GetAsync(url);
        }
        catch (HttpRequestException ex)
        {
            if (ex.InnerException is WebException e && e.Status ==
                WebExceptionStatus.TrustFailure)
            {
                response = new
                    HttpResponseMessage(
                        HttpStatusCode.MethodNotAllowed);
            }
            else
            {
                response = new
                    HttpResponseMessage(
                        HttpStatusCode.ServiceUnavailable);
            }
        }
        catch (Exception ex)
        {
            response = new
                HttpResponseMessage(
                    HttpStatusCode.InternalServerError);
        }
    }

    private bool ValidateServerCertificate(object sender,
        X509Certificate certificate, X509Chain chain,
        SslPolicyErrors sslPolicyErrors)
    {
        string currentCert = certificate?.GetPublicKeyString().
            ToUpper();
        return EndpointConfiguration.PUBKEY.ToUpper() == currentCert;
    }
}

```

The class constructor generates an instance of the **HttpClientHandler** class and assigns its **SslProtocols** property with **SslProtocols.Tls12** for demonstration purposes. You can certainly assign a different security protocol, depending on your network requirements. The description for all the values in the enumeration is available in the [documentation](#).

The code then assigns the **ServerCertificateCustomValidationCallback** property with a delegate that points to a method called **ValidateServerCertificate**, and that is defined at the bottom of the listing. Among the parameters, this method receives an instance of the **X509Certificate** class that represents the certificate and that is inferred by the underlying **HttpRequest** instance making the network call.

The **GetPublicKeyString** method returns the public key of the certificate, which you must compare to the key that has been stored inside the **EndpointConfiguration** class. The handler that has been defined in this way is assigned to the instance of the **HttpClient** class. Notice that this is marked as **private**, so that no caller code can interact with the **Client** property directly. Rather, it needs to invoke the **MakeCallAsync** method. Here the code invokes the **GetAsync** method to retrieve data from a server. Every time this method is invoked, the **HttpClient** instance will also invoke the **ValidateServerCertificate** method.

If the certificate public key retrieved by **ValidateServerCertificate** is the same as the one you stored in the **EndpointConfiguration** class, validation succeeds. If they are different, validation will fail and cause an **HttpRequestException** to be thrown. However, this specific exception can happen for many reasons, so we won't understand whether it was thrown because of the certificate validation failure or because of other network issues. Luckily enough, you can check its **InnerException**; if this is of type **WebException** and the value of its **Status** property is **TrustFailure**, it means that a server certificate could not be validated; therefore, it can perfectly fit in the current implementation.

Considerations for debugging

You might want to avoid checks for certificate pinning while debugging, at least once you have made sure and tested that it works with your app. There are several reasons for doing this. For example, your development and production environments have different certificates, and therefore, it's not useful to check for the development certificate every time. Or, you simply want to avoid overhead while debugging other features.

You can disable these checks while debugging with a simple **#if** preprocessor directive, which is added to the **ValidateServerCertificate** method, as follows:

```
private bool ValidateServerCertificate(object sender,
                                     X509Certificate certificate,
                                     X509Chain chain, SslPolicyErrors sslPolicyErrors)
{
    #if DEBUG
        return true;
    #endif
    string currentCert = certificate?.GetPublicKeyString().ToUpper();
```



```
        return serverCertificate.ToUpper() == currentCert;  
    }
```

This simple addition will simplify the way you do debugging, especially on simulators, which might not support all the protocols.

Chapter summary

When your app works with sensitive data, security is even more important, and implementing checks for certificate pinning is an easy way to secure data. In this chapter, you have seen how to configure the **HttpClient** and **HttpClientHandler** classes, and how to leverage the **X509Certificate** class and its methods to make security checks. You have also seen how to avoid checks while debugging to prevent overhead.

Security is an important topic for everyone on the team—certainly for developers, but also for the stakeholders who need to promote or sell apps. Another important topic for stakeholders is making decisions and investments based on app usage. This is the topic of Chapter 7.

Chapter 7 App Center: Drive Business Decisions with Analytics

Every project depends on a budget. Decision-makers in a company invest a budget in a project to make money. That's how business works, said in two sentences. Projects are an offer to customers, and for stakeholders, investors, and decision-makers, it's extremely important to understand how to improve the quality of the product (an app) and make customers happy.

People who invest money in the project you are working on will often ask about how many active users the app has, how much time they spend on it, the most-used features, the least-used features, and how reliable the app is. Generally speaking, they will ask you about numbers. This is where analytics come in to help.

Analytics allow developers to collect information about the usage of an app and will help you understand what features are most used and how reliable the app is. The Microsoft App Center offers an extremely powerful analytics engine that will help you provide answers to decision-makers and investors, so that they can drive their business decisions in the direction of success.

This chapter explains what the Microsoft offering is for analytics and how to set up both the App Center and apps to collect analytic data about app usage, without forgetting considerations about privacy.



Note: *This chapter describes everything that is useful for analyzing the app usage, but the App Center can do much more. You can bookmark the [documentation](#) for further studies.*

Analytics and diagnostic data

Among all the possibilities offered by the Microsoft App Center, this chapter focuses on two features:

- **Analytics:** this allows us to understand the use a user makes of the app by collecting events. An event can be, for example, a tap gesture, navigating to a page, and so on. Analytics also include the phone model of the user, the operating system, the OS version, the country of the user (based on the phone carrier), the number of sessions per day and week, and much more (see the section called “Understanding analytic data” later in this chapter).
- **Diagnostics:** this tracks errors and crashes that happen during the application lifetime, storing the .NET exception information for a full understanding of a problem, for example, reading the stack trace.

The App Center also collects common information that is independent from analytics and diagnostics. For example, it generates a session every time a device connects to the analytics engine, assigning a GUID to the session to make this identifiable. It also collects other information such as the IP address, the session duration, and any information that helps us understand how much time the user spent on the app without any details on the actions taken.

Setting up a sample project

Because the focus of the chapter is on analytics and collecting usage data, the sample application can be very simple. Create a new .NET MAUI project and replace the auto-generated XAML code with the following code shown in Code Listing 20 for the MainPage.xaml file.

Code Listing 20

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="AnalyzingActions.MainPage">
    <ScrollView>
        <VerticalStackLayout
            Spacing="25"
            Padding="30,0"
            VerticalOptions="Center">
            <StackLayout VerticalOptions="Center">
                <Button x:Name="FirstPageButton"
                    Text="Open first page"
                    Clicked="FirstPageButton_Clicked"/>

                <Button x:Name="SecondPageButton"
                    Text="Open second page"
                    Clicked="SecondPageButton_Clicked"/>

                <Button x:Name="HelpButton"
                    Text="Help"
                    Clicked="HelpButton_Clicked"/>
            </StackLayout>
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

The app will simply allow for navigating between two pages and will allow us to show a help dialog. The reason behind this is that we want to track page navigation, how many times a page is opened instead of another one, and how many times the user is tapping a button (in this case, asking for help).

Next, add two content pages to the project, one called **FirstPage.xaml** and one called **SecondPage.xaml**, leaving the auto-generated XAML markup. In the `App.xaml.cs`, enable navigation, changing the assignment of the `MainPage` property as follows:

```
MainPage = new NavigationPage(new MainPage());
```

You will need to add other pieces, such as event handlers for the buttons, but we'll do that after setting up the Microsoft App Center.

Configuring the Microsoft App Center

The Microsoft App Center is a complete solution for build automation and continuous integration, automated tests, app distribution, analytics, and diagnostics. In this chapter, I will demonstrate only the analytics and diagnostics features, but it's really worth mentioning that you can take full advantage of the App Center to automate builds from a code repository, with automated testing and distribution of the app to the selected people for testing and creating different kinds of releases, such as alpha, beta, production, or enterprise.

You will first need to log in to the [App Center](https://appcenter.ms). You can register using a Microsoft, GitHub, Facebook, or Google account. When logged in, you will see the welcome message shown in Figure 16.

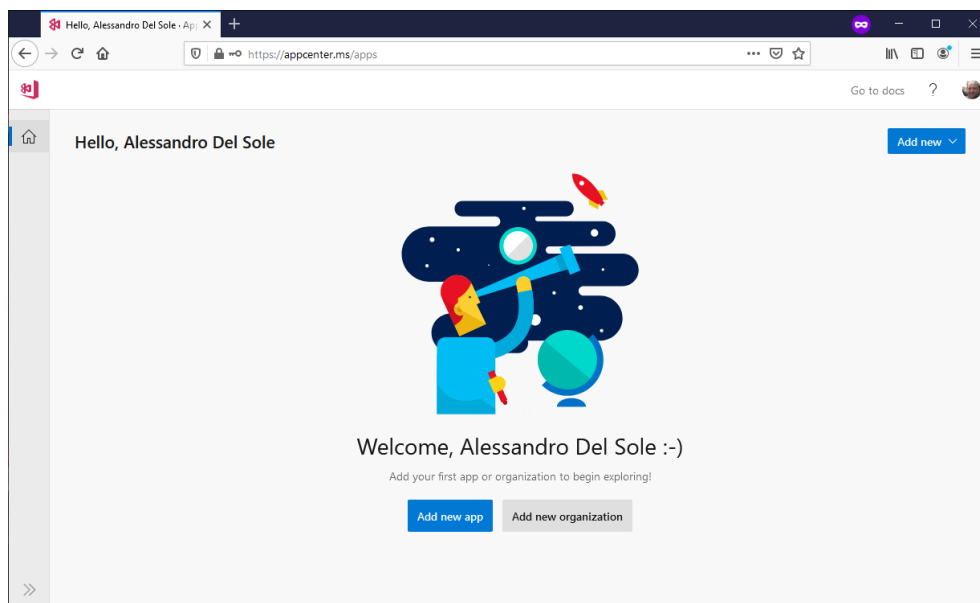


Figure 16: Logging in to the App Center

Click **Add new app**. As you can see in Figure 17, you will be asked to enter an app name, the target operating system, and the platform used for development. At the time of writing, there is no option for .NET MAUI as a platform, but you can select **Xamarin**.

For the operating system, you will need to select which one you want to target, for example iOS, like in Figure 17.

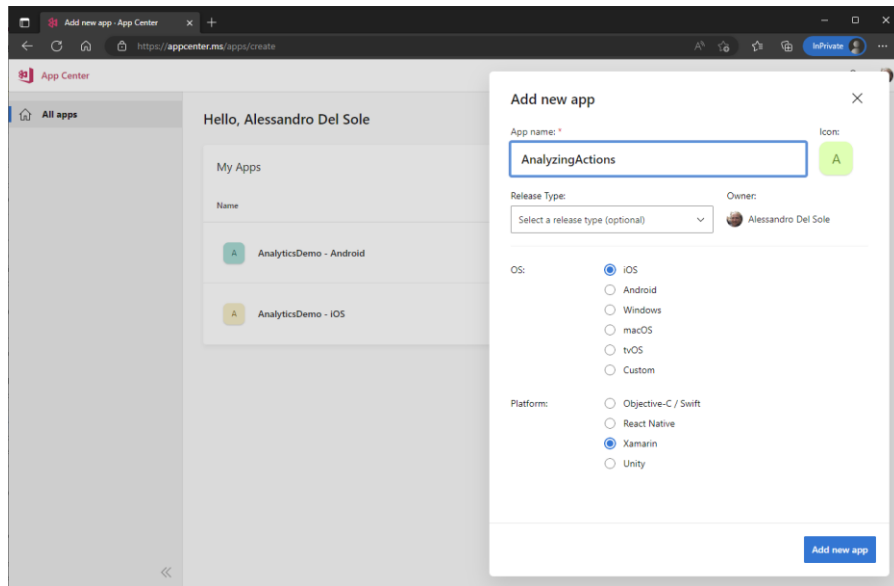


Figure 17: Adding an app to the App Center

You will need to add an app for each OS you want to target, so if you are targeting iOS and Android, you will repeat the same steps to add an app, specifying Android as the OS and Xamarin as the platform. In summary, you will have at least two apps, and you can have more if you target, for example, UWP.

When the app has been added, the App Center should automatically display the instructions to set up a Xamarin project (see Figure 18). Luckily enough, the same instructions work well with .NET MAUI, so everything will be easy.

 **Tip:** *If the instructions do not automatically appear, you can click the Overview button (the one below the app icon on the toolbar at the left side of the page).*

If this does not happen, you can click the **Overview** button (the one below the app icon on the toolbar at the left side of the page). Figure 18 shows the instructions to set up a Xamarin.Forms project to work with analytics, keeping in mind that the same applies to .NET MAUI.

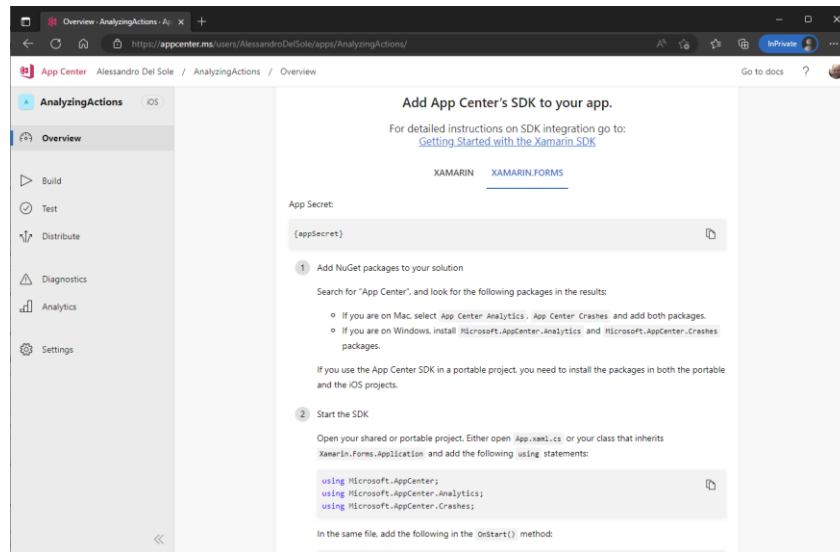


Figure 18: The instructions to set up a Xamarin.Forms project to work with analytics

As you can see, the instructions explain how to include both analytics and diagnostics information in the code. The next paragraph walks through the complete steps to set up the project.

Adding analytics and privacy consent to the app

Before writing code, you need to install a few NuGet packages. More specifically, you need the following (see Figure 19):

- Microsoft.AppCenter
- Microsoft.AppCenter.Analytics
- Microsoft.AppCenter.Crashes

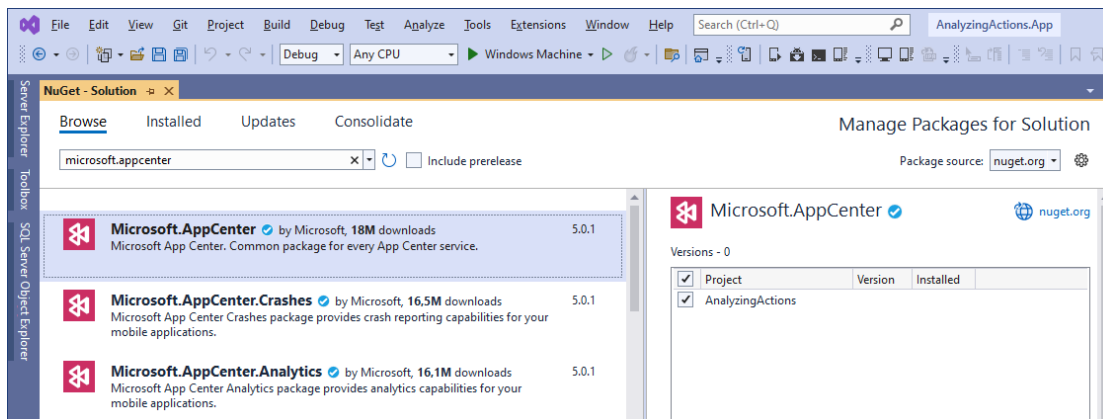


Figure 19: Analytics NuGet packages



*Tip: Because the **Microsoft.AppCenter** is a dependency for the other packages, you can start installing one between **Microsoft.AppCenter.Analytics** and **Microsoft.AppCenter.Crashes**, and **Microsoft.AppCenter** will be automatically installed.*

When you created the iOS and Android apps in the App Center, a so-called app secret was created. An app secret is a GUID that connects your app to your App Center account, and one has been generated for all the apps you added. You can find the app secret in the code presented in the instructions inside the App Center.

Once you have taken all the app secrets, you can add the following code in the **OnStart** method of the **App** class, replacing the app secrets with yours:

```
protected override void OnStart()
{
    AppCenter.Start("android=ANDROID_SECRET_GOES_HERE;" +
                    "ios=iOS_SECRET_GOES_HERE;",
                    "uwp=UWP_SECRET_GOES_HERE;",
                    typeof(Analytics), typeof(Crashes));
}
```

Actually, an app secret is connected to a specific release. In the current example, I only got a default release when I added the apps, but if you create more app releases, such as alpha, beta, and production, you will get a different app secret for each kind of release. As an implication, you will need to add as many **AppCenter.Start** invocations as you have releases (or you might implement your own detection mechanism for releases and switch between app secrets with an **if** or **switch** block).

Tracking events and errors

With analytics, you can track events and .NET exceptions occurring during the app lifetime. Events can be any actions the users do while using the app, such as tapping buttons, opening pages, and scrolling lists. Events can be tracked using the **Microsoft.AppCenter.Analytics** class, which exposes the **TrackEvent** method. In order to understand how it works, in the **MainPage.xaml.cs**, add the following event handler for the Help button:

```
// Requires a using Microsoft.AppCenter.Analytics; directive
private async void HelpButton_Clicked(object sender, EventArgs e)
{
    await DisplayAlert("Help",
        "This app will help you understand how App Center Analytics work",
        "OK");
    Analytics.TrackEvent("Help button pressed");
}
```

As you can see, when the user presses a button, other than performing a task (in this case, showing an alert), an event is sent to the analytics engine in a very detailed way. **TrackEvent** is very flexible, as it allows you to specify additional, custom information to the event. This is made possible by adding a **Dictionary<string, string>** as an argument, like in the following example, which demonstrates how to group events by category:

```
Analytics.TrackEvent("Help button pressed",  
    new Dictionary<string, string> { { "Category", "UI" } });
```

In this way, you can divide events by category and filter them more easily in the App Center. Let's keep the simpler way for now. First of all, add the following event handlers for the buttons that open the two pages:

```
private async void FirstPageButton_Clicked(object sender, EventArgs e)  
{  
    await Navigation.PushAsync(new FirstPage());  
}  
  
private async void SecondPageButton_Clicked(object sender, EventArgs e)  
{  
    await Navigation.PushAsync(new SecondPage());  
}
```

In the `FirstPage.xaml.cs`, add the following code:

```
protected override void OnAppearing()  
{  
    base.OnAppearing();  
    Analytics.TrackEvent("First Page opened");  
}  
  
protected override void OnDisappearing()  
{  
    base.OnDisappearing();  
    Analytics.TrackEvent("First Page closed");  
}
```

It is useful to track both the opening and closing events to understand how much time the user spends on a page. In the `SecondPage.xaml.cs` file, the code will also simulate an exception for error tracking:

```
protected override void OnAppearing()  
{  
    base.OnAppearing();  
    Analytics.TrackEvent("Second page opened");  
}  
protected override void OnDisappearing()
```



```

{
    try
    {
        base.OnDisappearing();
        Analytics.TrackEvent("Second page closed");
        throw new InvalidOperationException();
    }
    catch (Exception ex)
    {
        Crashes.TrackError(ex);
    }
}

```

The `Microsoft.AppCenter.Crashes.Crashes` class exposes the `TrackError` method, which receives an object of type `Exception` (or derived) as an argument. `TrackError` will send the full exception information, including the stack trace, to the analytics engine, and this will be crucial for helping developers debug any possible error. You will shortly see the result of event and error tracking in the App Center website, but an important piece is still missing: the user consent for analytics.

Managing user consent for analytics

You cannot use analytics and track a user's behavior without the user's explicit permission. This is required by laws and regulations, and it's something you cannot forget. You should not track any information that could explicitly identify the user, and actually, it is not necessary for the purpose of analytics, which are intended for usage and behavior analysis only.



Note: *Privacy is a crucial topic for mobile apps in general, and even more if an app uses analytics. I would recommend you ask a lawyer or consult with specialized companies about privacy and consent before implementing analytics, and before writing and presenting to the user any privacy policy text and analytics consent text. It is important that you do not underestimate this point.*

In the current example, the user's explicit permission is asked in a very easy way, through an alert. In your apps, you will want to present the full text of the privacy policy and store the acceptance or rejection inside the app settings or inside a database. Having said that, add the following code to the `MainPage.xaml.cs`:

```

bool? isAnalyticsConsentSet = null;
protected override async void OnAppearing()
{
    base.OnAppearing();
    if (!isAnalyticsConsentSet.HasValue)
    {
        bool result = await DisplayAlert("Consent",
            "Do you provide your consent for analytics?", "Yes", "No");
        await Analytics.SetEnabledAsync(result);
        await Crashes.SetEnabledAsync(result);
    }
}

```

```

        isAnalyticsConsentSet = result;
    }
}

```

The user's choice is stored inside an in-memory variable for the sake of simplicity, but do not forget my considerations mentioned previously. The key point here is the **SetEnabledAsync** method exposed by both the **Analytics** and **Crashes** classes. You can enable or disable analytics and crash tracking by simply passing **true** (enabled) or **false** (disabled) to this method and leaving all the existing tracking code you wrote unchanged.

Running the app and doing actions

If you run the sample app, the first thing you will see is the analytics consent request, as shown in Figure 20.

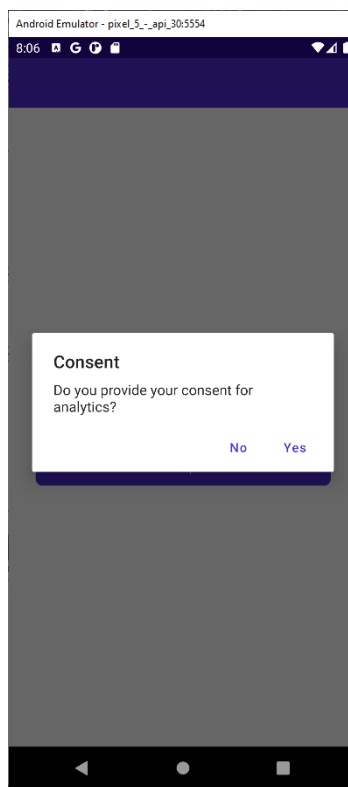


Figure 20: Asking users' permission for analytics

Tap **Yes**, then open the two pages, go back, and tap **Help** several times. This will generate a number of analytics events.

 **Tip:** A nice way to collect sample analytics data would be using the app on different devices, on different days, and setting different languages on the devices. This will help create more structured views in the App Center.

After running the app a few times, some analytics data should be available. The next section describes analytics data that you can find in the App Center.

Understanding analytics data

All the analytics data is available in the [Microsoft App Center](https://appcenter.ms/users/AlessandroDelSole/apps/AnalyticsDemo-Android/Analytics/Overview). From the App Center, select one of the apps you created previously (such as Android or iOS), and then click **Analytics** in the toolbar at the left side of the screen.

The first kind of information you get is about users. More specifically, unique active users, shown in a graph that shows the development of user access over time; **Daily sessions per user** shows how many times a user opened the app every day; **Session duration** shows how much time the user spent in the app every time they used it; and **Top devices** shows the list of the most-used devices for the selected operating system. Figure 21 shows this information.

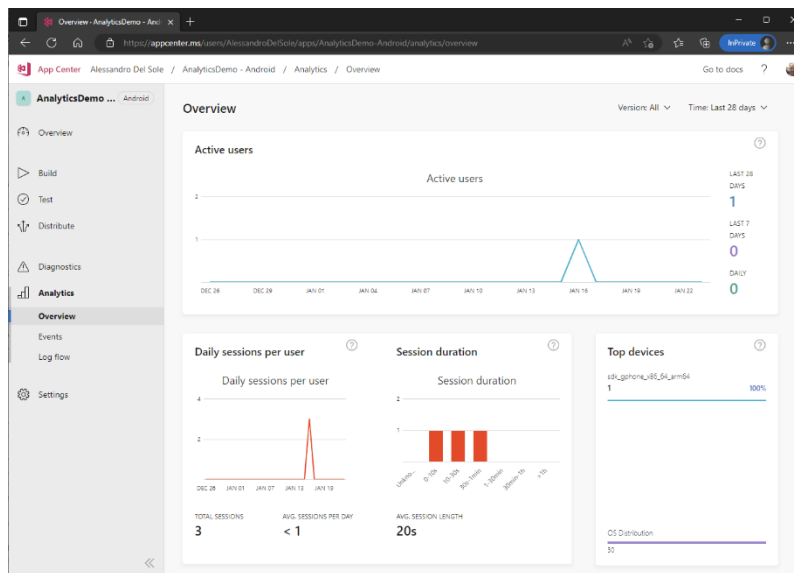


Figure 21: Analyzing user access

Scrolling down the page (see Figure 22), you will see additional information about countries and languages. It can take time for the analytics engine to collect information, so the **Countries** area might stay empty for a while, like in Figure 22, but here you will see where your users come from and what language they have set on their device.

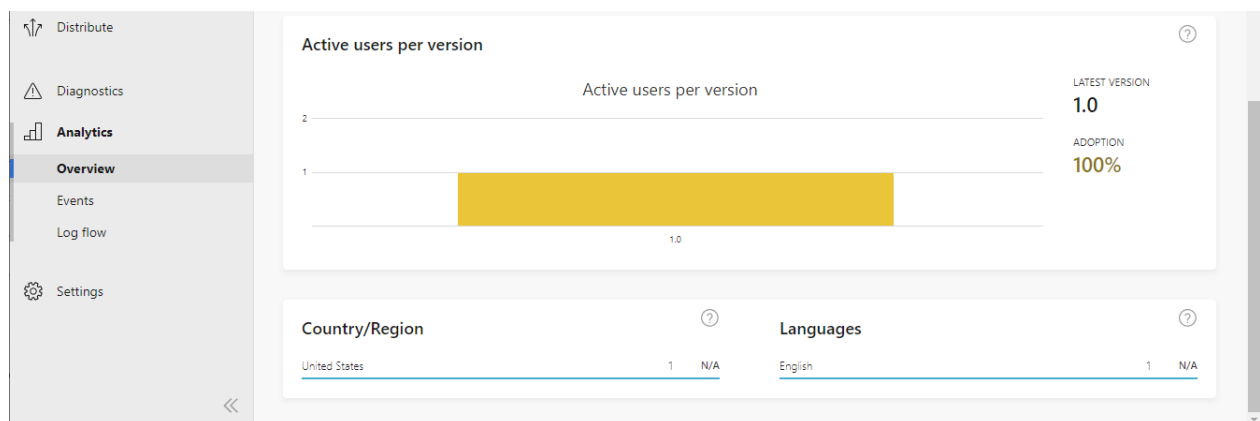


Figure 22: Countries and language information

If multiple versions of your app are currently available, the **Active users per session** area will show a report about how many users are using each version. The next group of information is about events and is probably the most interesting when it comes to deciding on which features to invest in more (or less).

Click **Events** in the toolbar at the left. You will see the full list of events that you tracked by invoking the **Analytics.TrackEvent** method, as shown in Figure 23.

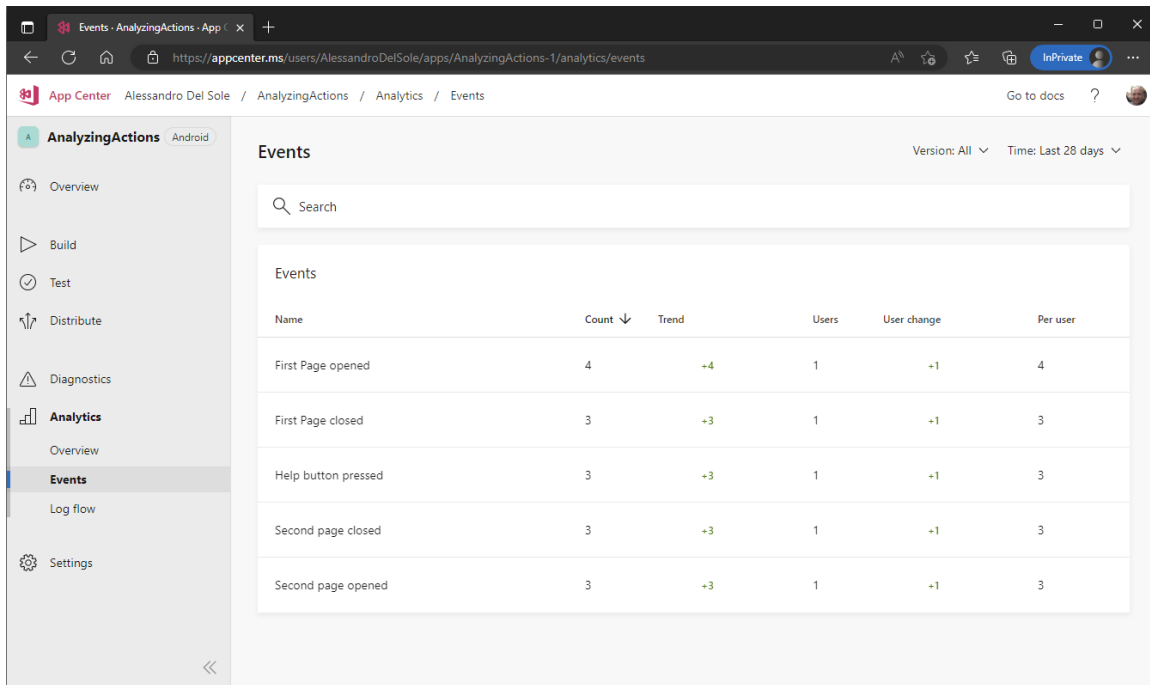


Figure 23: Analyzing events

For each event, you will see the total count, the positive or negative trend, how many users are involved, and the number of events per user. The default timeframe is the last 30 days, but you can select a different interval with the dropdown at the upper-right corner. Here you basically get a summary of how many features are used. You can also click individual events to see more detailed information, and Figure 24 shows an example.

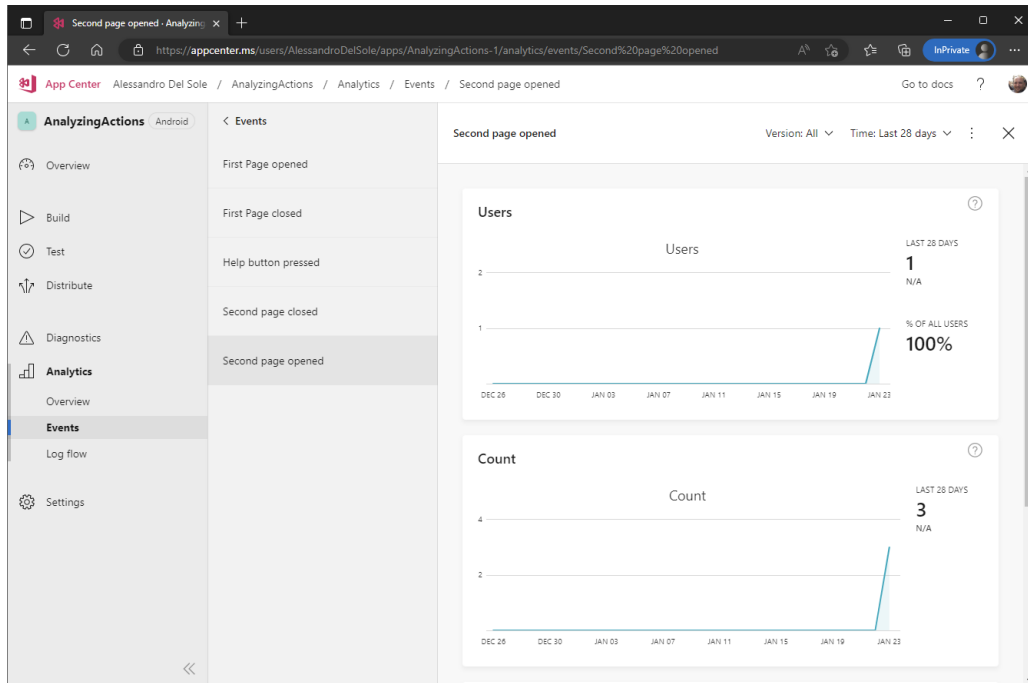


Figure 24: Event details per user

In this detailed view, you can see graphs that show how users interacted with the feature that raised the event over time. This is another important way to analyze the app usage; for example, it gives you an idea of how a feature has been used since its release, and it can help you understand if the feature was easy to find, easy to use, useful for the user, and so on.

The last analytic data to walk through is diagnostic data. Click **Diagnostics** in the toolbar at the left. When the page is opened, you will see issues grouped by crashes and errors. In the current example, there are errors raised intentionally by the app and tracked. Figure 25 shows how the page looks.

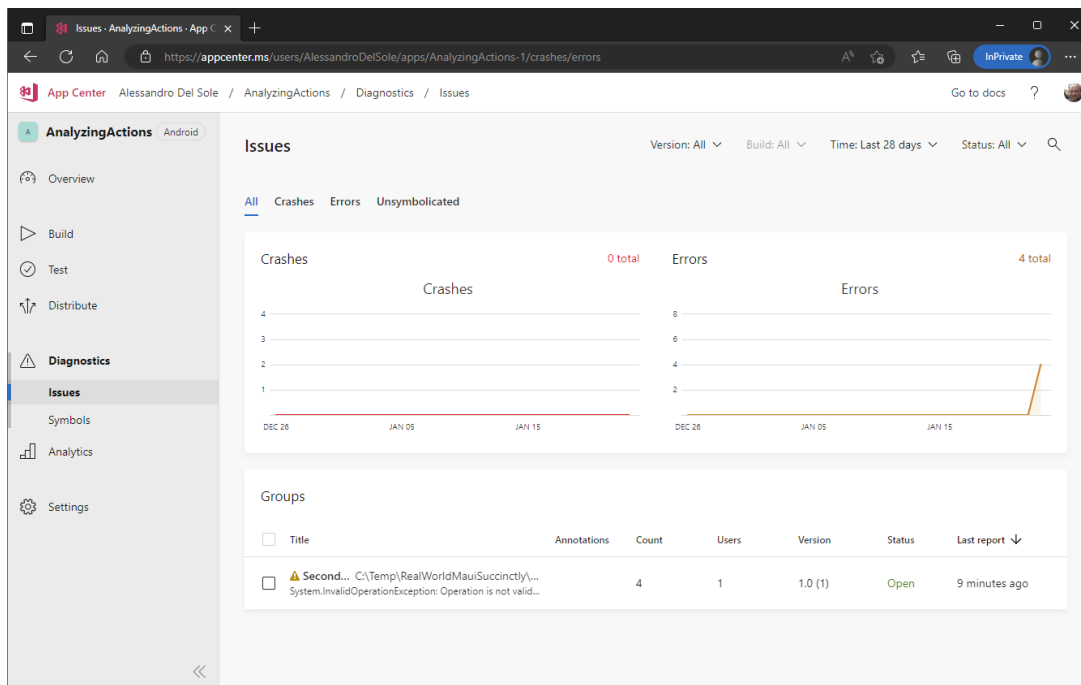


Figure 25: Diagnostic information

You have graphs to see crashes and errors over time, but the most useful information here is probably the list of .NET errors at the bottom of the page. In the current example, these have been raised intentionally to make the analytics engine generate reports, but in your apps, it will be important to add **Crashes.TrackError** invocations inside **try...catch** blocks. If you click on an error, you will get a detailed view that includes the stack trace, as shown in Figure 26.

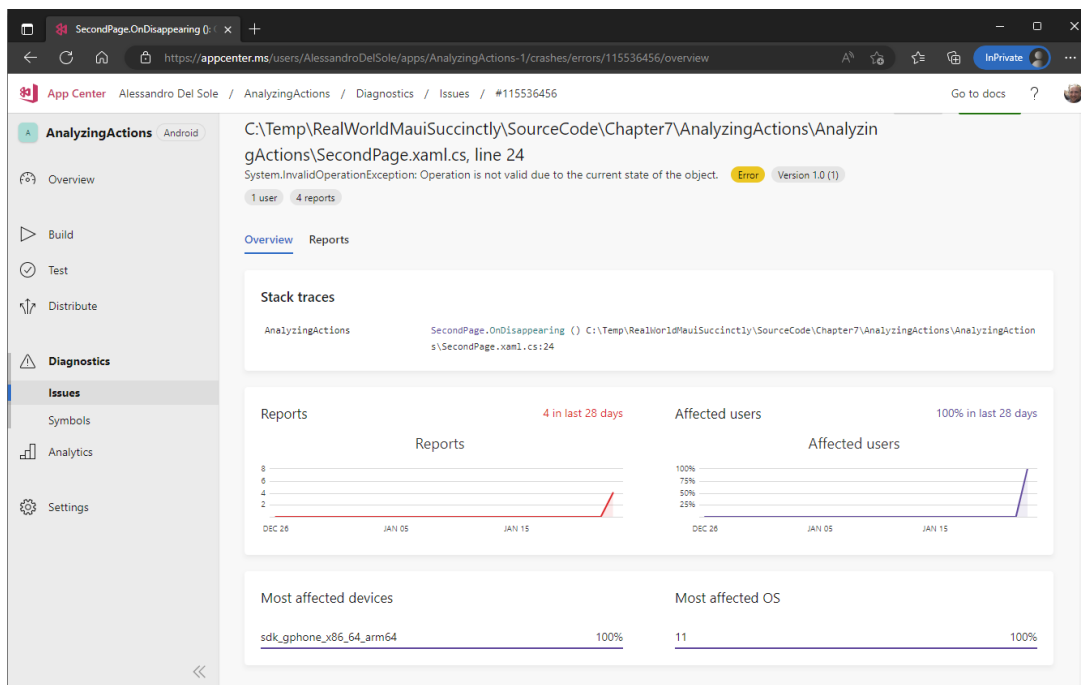


Figure 26: Detailed view of .NET exceptions

Diagnostic information here includes the stack trace, including the line of code and file that caused the error, a graph that shows how many times it happened, a graph that shows the number of users affected, and a summary of the most affected devices and OSes. In my experience, Android 9 is an operating system that needs more attention than its predecessors and successors due to its API, so having information on the OS version has been crucial.

With all this information in hand, all the roles involved in the project will have an option to increase the quality and success of an app, from stakeholders and investors who will raise the budget for features that are useful for the customers, to developers and designers who will understand better how to fix problems and improve the interaction.

Hints about Application Insights

The way the App Center collects information is certainly clear and useful for most of the roles in the project, but for further queries, data grouping, and advanced analysis, the best way is connecting your App Center account to Application Insights (also known as Azure Monitor) in Microsoft Azure, which also offers a structured query language to write powerful, custom queries. This is not within the scope of this chapter, but it is worth a mention. You can find more information [on this page](#) of the documentation.

Chapter summary

Decision-makers need to understand where to drive the budget for a project, and analytics help them do it. With analytics, you can track events and errors that happen in your app, understanding what features users like most, how much time they spend on a specific part, and more. In this way, all roles involved in the project can give their best contributions to improve the product.

As a developer, you are interested in writing code, and the **Analytics** and **Crashes** classes with their **TrackEvent** and **TrackError** methods, respectively, make it extremely easy to send analytics information. Last but not least, these classes also make it simple to handle users' explicit permission to collect analytics by passing true or false to the **SetEnabledAsync** method, which allows you to leave the rest of the code unchanged.

Improving the product quality is also possible by providing the user the best experience, which also comes through a nice user interface. This can be easily enriched with Lottie animations, which is discussed in the next chapter.

Chapter 8 Displaying Lottie Animations with SkiaSharp

Animated images and illustrations are an important way to enrich the look and feel of mobile applications and make them more appealing. In my daily work, I use animated illustrations connected to user interactions to provide educational content.

In the last few years, an interesting animation format has become extremely popular among designers and developers, produced by the famous Adobe After Effects design suite. This format is not actually a proprietary one; rather, it is JSON. Put succinctly, Adobe After Effects can produce files in a specialized JSON format containing the design for animated illustrations.

In order to render these JSON animations, several libraries exist, but the most important is called Lottie, and it is generally offered by Airbnb. However, for .NET MAUI, a Lottie implementation exists in the SkiaSharp graphic library, and it allows you to play with beautiful, animated JSON illustrations in your mobile apps. This is the topic of this chapter.

.NET MAUI and Lottie

For the next example, create a new .NET MAUI project, remove the unnecessary code from the MainPage.xaml and MainPage.xaml.cs files, and open the NuGet Package Manager user interface. The NuGet package you need to install is called **SkiaSharp.Extended.UI.Maui**, as demonstrated in Figure 27.

At the time of this writing, this package is not yet shipped to production, so I need to check the **Include prerelease** checkbox. Remember that SkiaSharp is also maintained by Microsoft, so a production release should be available soon.

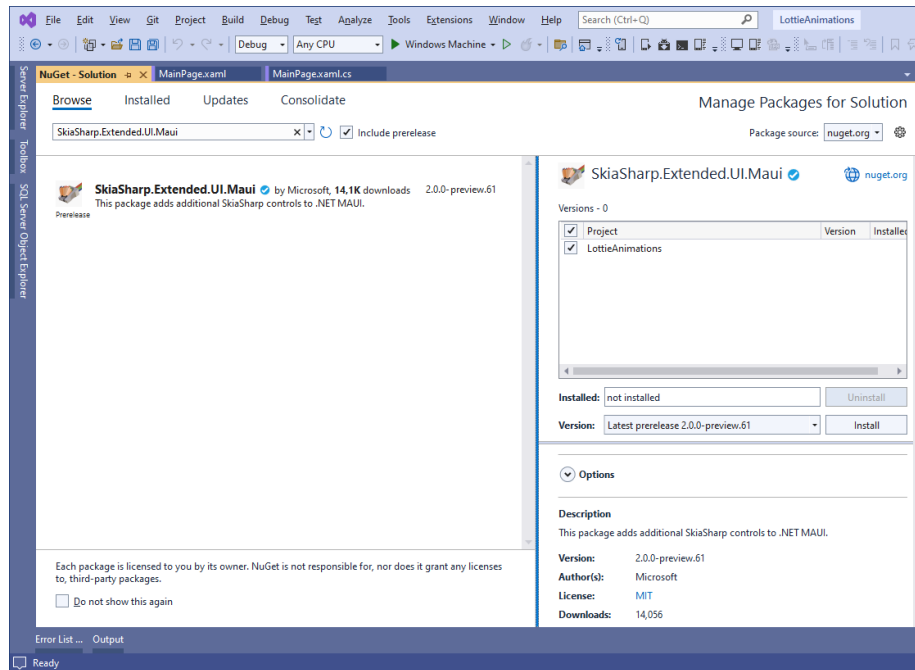


Figure 27: Installing the SkiaSharp NuGet package

As you will see shortly, the library offers a specific view that is very easy to use, but before using it, you need some sample animations.

Adding Lottie animations to the project

The way your app consumes Lottie animations depends on the source of the animations themselves. If your team has designers that produce Lottie JSON files, you will add such files to the Resources\Raw subfolder of the .NET MAUI project. If Lottie animations are instead hosted on a web server, either public or private, the animation can be simply consumed, passing its URI to the Lottie view.

A very nice and popular source for Lottie animation is the [Lottiefiles.com](https://lottiefiles.com) website. On this site, you can find both free and paid animations, illustrations, icons, and much more. For the current example, I will use the JSON animation included in the Lottie library repository, which is already included inside the companion code for your convenience.



Note: The JSON for Lottie animations can be very long and certainly complex. The goal of this chapter is not to explain how it is structured, so it is left to you for further studies.

You are certainly free to use a different animation, in which case you will decide how to consume it based on what you learn in the next sections.

Displaying Lottie animations

The SkiaSharp library provides a new view called **SKLottieView**. Before you can use it in your XAML, you need to add the following XML namespace declaration at the **ContentPage** level:

```
xmlns:skia="clr-namespace:SkiaSharp.Extended.UI.Controls;assembly=SkiaSharp.Extended.UI"
```

In its simplest form, the **SKLottieView** can be declared as follows:

```
<skia:SKLottieView HorizontalOptions="FillAndExpand"
    Source="lottielogo.json"
    RepeatCount="3" RepeatMode="Restart"
    VerticalOptions="FillAndExpand" />
```

The **Source** property specifies the animation file, which can be local or remote. In this example, the animation will restart (**RepeatMode**) three times (**RepeatCount**). The **RepeatMode** attribute can be also set with **Reverse**, whereas **RepeatCount** can be assigned with -1 if you want the animation to be played just once.

If you run the sample app, you will see the Lottie animation being played on your device. Figure 28 shows a static frame of the animation, but it will be actually played when running.

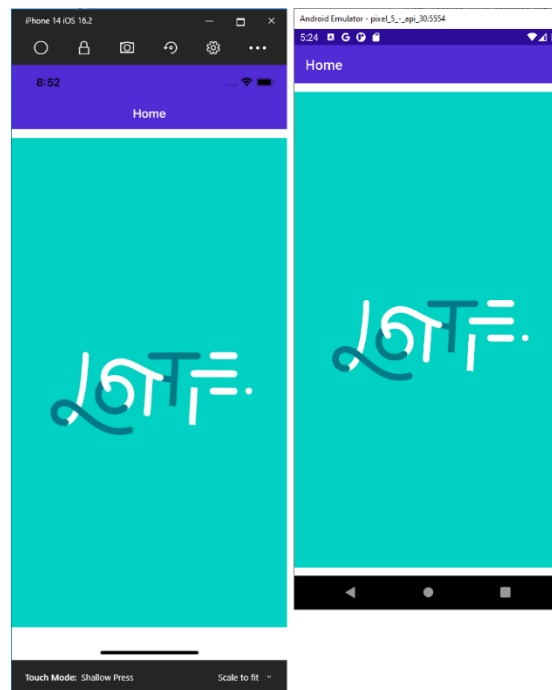


Figure 28: Lottie animation running in .NET MAUI

The **SKLottieView** object also exposes the following events:

- **AnimationLoaded**, raised when the animation has been loaded by the **SKLottieView**.
- **AnimationFailed**, raised when loading or playing the application fails.

Especially when your animations are loaded from a web resource via URI, failures in loading the animation can happen and the **AnimationFailed** event will be raised. Think of connectivity issues, or an animation that is not compatible with the After Effects JSON structure.

Finally, you can control the animation with the **IsAnimationEnabled** property. When **true**, the animation will be played, and it will be stopped when **false**.

Chapter summary

Lottie is a library from Airbnb that allows for rendering Adobe After Effects animations, created in a specialized JSON format. An implementation for .NET MAUI is provided by the SkiaSharp library and offers the **SKLottieView** control, which makes it easy to show animations with a basic implementation. Lottie animations are certainly an important addition to make mobile apps more beautiful, but they are not the only pieces of content your app might display. An important part is made up of documents, which is discussed in the next chapter.

Chapter 9 Displaying and Sharing Documents

Mobile apps offer the great benefit of bringing our documents everywhere with us, no matter whether they are stored locally on the device or on a cloud service. This opens the door to many possibilities, because you might need to receive, send, or share documents with other people even when you are not at your desk working on a computer. Obviously the most popular file format for sharing documents is PDF, so the focus of this chapter is going to be on this file format, but at least for the sharing part, you will be able to reuse the described techniques with any format.

In terms of mobile app development with .NET MAUI, there is no built-in view that allows for displaying PDF documents, but luckily there are third-party controls that make this easy. Sharing files is instead provided by the .NET MAUI code base, which simplifies development with yet another scenario. In the next paragraphs you will learn how to display PDF documents in your mobile apps using the Syncfusion® PDF Viewer and how to share PDF documents leveraging the MAUI API.



Note: A valid subscription for Syncfusion® Essential Studio® for .NET MAUI is required. Actually, you can use a trial version of the libraries described in this chapter by simply downloading the NuGet packages, and the app will show a message about the trial status. If you are an individual developer or work for a small business, you can also consider the [Community License](#). Make sure you read the related page to understand if you are eligible.

Chapter focus and sample project setup

The goal of this chapter is to explain how to display PDF documents in your apps built with .NET MAUI, and how to share them using the operating system's built-in sharing mechanism. In order to provide you with an example that is working locally, I have included a sample PDF document as an embedded resource of the companion project. In your real project, you will be free to choose from where the documents will be opened, and you will be able to use the techniques described in the “Working with files” section of Chapter 1.

Now, create a new, blank project. Displaying PDF documents is achieved using the **SfPdfViewer** control by Syncfusion®, which is part of the Essential Studio® for .NET MAUI suite available as a NuGet package. When you're ready, browse NuGet for the **Syncfusion.Maui.SfPdfViewer** package (see Figure 29) and install it in all the projects in the solution.

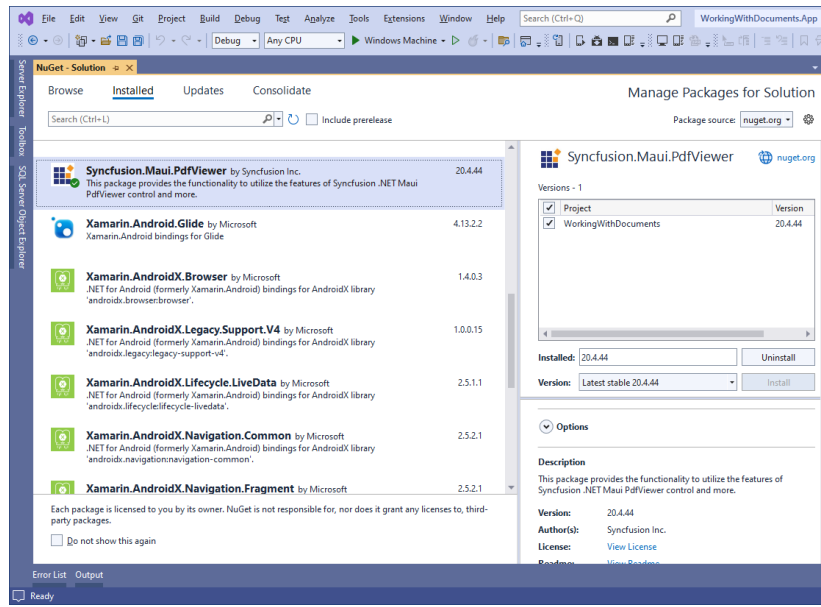


Figure 29: Installing the Syncfusion.Maui.PdfViewer NuGet package

When the package has been installed, you need an additional step. In the MauiProgram.cs file, you need to initialize the PDF Viewer with the following line that must precede the **return** statement of the **CreateMauiApp** method:

```
//Requires a using Syncfusion.Maui.Core.Hosting; directive
builder.ConfigureSyncfusionCore();
```

The user interface for the main page is very simple. It includes the PDF Viewer and two buttons: one for loading a document and one for sharing the document. Code Listing 21 contains the XAML code for this.

Code Listing 21

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:syncfusion="http://schemas.syncfusion.com/maui"
    x:Class="WorkingWithDocuments.MainPage">
    <Grid x:Name="PdfViewGrid">
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition Height="40" />
        </Grid.RowDefinitions>
        <syncfusion:SfPdfViewer
            x:Name="PdfViewerControl" />

        <StackLayout Grid.Row="1" Orientation="Horizontal">
            <Button x:Name="OpenButton"
                Text="Open">
```

```

        Clicked="OpenButton_Clicked" />
    <Button x:Name="ShareButton"
        Text="Share"
        Clicked="ShareButton_Clicked" />
</StackLayout>
</Grid>
</ContentPage>

```

For now, only focus on the XML namespace added to use the **SfPdfViewer** control. Specific considerations are coming in the next section, together with the C# code that enables the two buttons. As I mentioned previously, I added a sample PDF document in the companion solution, but if you are working on a brand new one, add a PDF file to the root folder of the project and set its **Build Action** as **EmbeddedResource**.

The last step for setup is specifying the Syncfusion license key, which you specify by adding the following line of code in the constructor of the App.Xaml.cs file, before the assignment of the **MainPage** property:

```

Syncfusion.Licensing.SyncfusionLicenseProvider.
    RegisterLicense("YOUR-LICENSE-KEY-GOES-HERE");

```

If you don't add the license key, libraries will work in trial mode (which is fine for the current example, but not for production purposes). In trial mode, the **SfPdfViewer** control will add a watermark to the view, saying that the document is being displayed with a trial version of the library.

Introducing the SfPdfViewer control

The Syncfusion® **SfPdfViewer** is a very powerful and versatile control that can be used to display, annotate, and edit PDF documents. In Code Listing 21, it is declared as follows:

```

<syncfusion:SfPdfViewer x:Name="PdfViewerControl" />

```

You load PDF documents into the **SfPdfViewer** via its **LoadDocument** method, which receives the file to open as a **Stream** object. You can also invoke the **LoadDocumentAsync** method for asynchronous loading. Both methods can optionally receive a **string** argument representing the document password, if the PDF is protected. With this in mind, the event handler for the **OpenButton** will look like the following:

```

private void OpenButton_Clicked(object sender, EventArgs e)
{
    var fileStream = typeof(App).GetTypeInfo().Assembly.
        GetManifestResourceStream("WorkingWithDocuments.SampleDoc.pdf");

    PdfViewerControl.LoadDocument(fileStream);
}

```

The first line of code reads a file from the embedded resources and returns a **Stream**, and the name has the following structure: `ProjectName.FileName.FileExtension`. If the file was placed in a project subfolder, the structure would be `ProjectName.FolderName.FileName.FileExtension`. In this case, the project is called `WorkingWithDocuments`. The second line loads the stream into the **SfPdfViewer** control. If you run the app as it is, you will see the result shown in Figure 30.

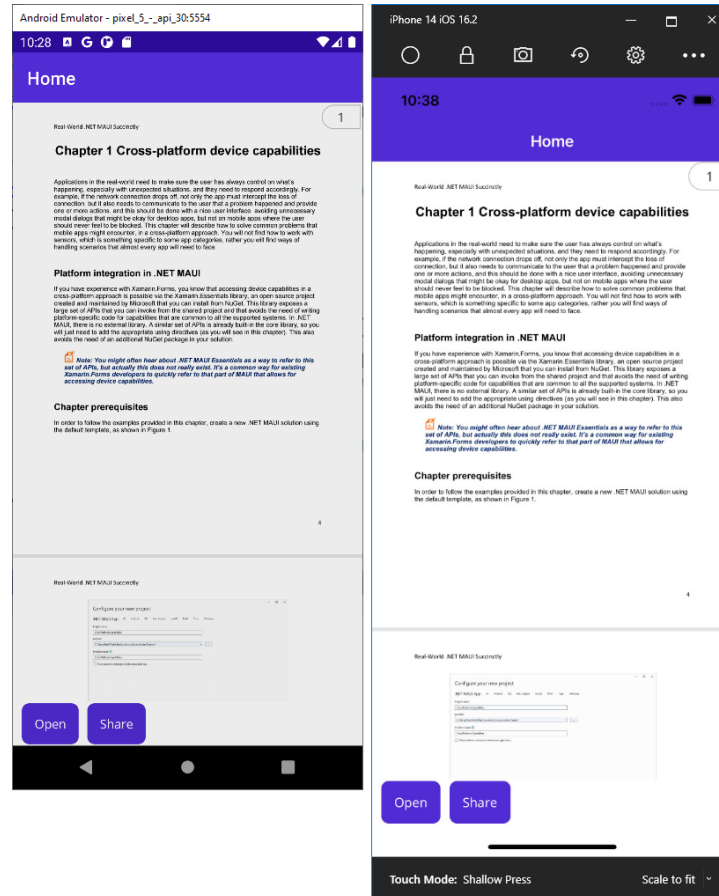


Figure 30: Displaying a PDF document in .NET MAUI

As you can see, the PDF document is shown in the user interface in a convenient way.

 **Note:** At the time of writing this chapter, the **SfPdfViewer** control is not feature complete yet. This means that it is reasonable to expect updates with new built-in tools, especially to achieve parity with the old **Xamarin.Forms** viewer.

The user interface of the **SfPdfViewer** is designed to be simple and easy to access, but powerful at the same time, and these features are those you would expect from a PDF viewer on a mobile app. Make sure you visit the [official documentation](#) for further information on the control's customization, localization, and other features.

Sharing files and documents

Sharing contents in mobile apps is one of the most important features in terms of app usability and goes beyond the concept of sharing on social media. For example, if you receive a PDF document via email and you want to open it inside a PDF viewer app like Adobe Reader, you are actually sharing the document from the mail client with the PDF viewer. Similarly, you can share files on your device, such as pictures and videos, in a Whatsapp chat. What you are doing is asking the operating system to open another app that is capable of receiving contents from other apps.

In a mobile app that works with data, a common scenario is to receive a document via Web API, display the document in-app, and have the option to send it via email to some recipients. So at a higher level, by sharing, we generally mean sharing contents with another app.

Sharing contents in .NET MAUI is made easy by the **Share** class. This class exposes a static method called **RequestAsync**, which receives an argument that can be of the following types:

- **string**, for sharing plain text.
- **ShareTextRequest**, for sharing URIs or text.
- **ShareFileRequest**, for sharing files.

Working with the first two is not covered in this chapter; we will focus on the third one. The **ShareFileRequest** class points to the local pathname of a file. In the current example, the PDF document is stored as an embedded resource, so an additional step is required to get the file from the resources, copying it into the app's directory and then pointing to the obtained pathname. This is accomplished by the following method called **ShareAsync**:

```
private async Task ShareAsync()
{
    var fileStream = typeof(App).GetTypeInfo().Assembly.
        GetManifestResourceStream("WorkingWithDocuments.SampleDoc.pdf");

    var cacheFile = Path.Combine(FileSystem.CacheDirectory,
        "SampleDoc.pdf");

    using (var file = new FileStream(cacheFile, FileMode.Create,
        FileAccess.Write))
    {
        fileStream.CopyTo(file);
    }
    var request = new ShareFileRequest();
    request.Title = "Share document";
    request.File = new ShareFile(cacheFile);
    await Share.RequestAsync(request);
}
```


Like in the first example about loading the document, the first line gets the file stored inside the embedded resources as a **Stream**. The **cacheFile** variable combines the path for the app's cache directory with the file name into one pathname. The **using** block creates an instance of the **FileStream** class pointing to the original file's stream, creating a copy in the target directory. The **using** block ensures the stream resources are disposed of after usage. The **ShareFileRequest** object can point to the local pathname via the **File** property. The **Title** property allows for specifying the title for the sharing user interface, if supported by the OS.

The **ShareAsync** method is invoked by the **Clicked** event handler for the **ShareButton** as follows:

```
private async void ShareButton_Clicked(object sender, EventArgs e)
{
    await ShareAsync();
}
```

If you now run the app and click the **Share** button after opening the document, you will get a result similar to Figure 31.

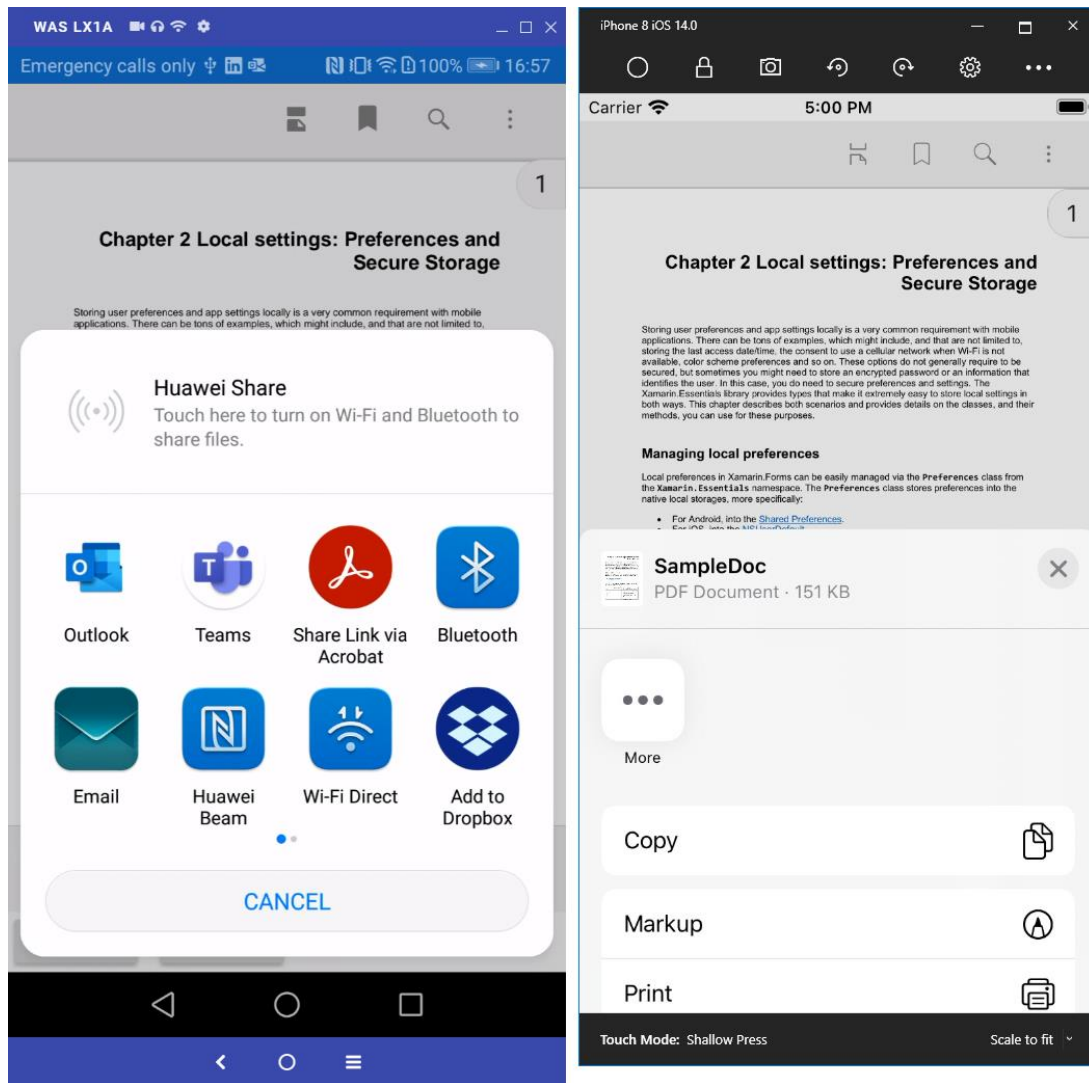


Figure 31: Sharing a document

The result will almost certainly vary on your device, because it depends on how many and which apps you have installed to support PDF documents (in this case, for iOS, Figure 31 is based on the simulator, which has no PDF apps installed, and therefore, it only shows device options).

You will just need to select an app enabled to receive PDF documents and the file will be shared, using the system interface and without the need for writing platform-specific code, which is an incredible benefit. The app that receives the document will then know how to handle it. For example, if you select an email client for sharing, the document will be automatically added as an attachment to a new email message.

Chapter summary

Applications might need to work with PDF documents for several reasons, such as for displaying reports, forms, or simply documentation. .NET MAUI does not have a built-in control to display PDF documents, but Syncfusion® provides the **SfPdfViewer** view, which is simple to use and includes a toolbar with all the commands and tools required to annotate, comment, and sign documents. When it comes to sharing, the **Share** class from the .NET MAUI library makes it easy to send files to other apps by simply invoking the **RequestAsync** method.

Conclusion

.NET MAUI offers everything you need to create powerful, native mobile applications. The purpose of this book is to provide guidance on how to get the most out of .NET MAUI to implement common requirements in modern mobile apps, from user experience requirements to runtime requirements.

In several cases, you have seen how to use the .NET MAUI library to satisfy basic device requirements in a cross-platform approach, such as checking for network connection and battery level, without the need to write platform-specific code. You have seen how to secure data locally and through communications over networks and also how to secure access to your app via biometric authentication. You have seen how analytics can improve business decisions to drive investments based on the app usage, which is critical to stakeholders. You have seen how to improve the look and feel of an app with Lottie animations, and you have also seen how to implement a PDF viewer to improve user productivity with documents.

You also got some tips about legal notices that you need to provide, after getting advice from a lawyer specialized in IT, because it is more important than the whole app content. In your daily job as a developer with .NET MAUI, there will certainly be more and different requirements and implementations, but for what is not included in this book, you can always bookmark the [official documentation](#).

In this book, I have shared practical solutions that I have been implementing in my daily job, with continuous updates as the technology evolves, and have discussed with other, very cool developers who share the office with me. Because sharing knowledge is the key to technical growth for every developer, I am happy to receive feedback on this book and on the techniques described here. If reading these chapters makes you think about more efficient solutions that you want to share, that would be beneficial to all the developer community.